VOLUME 02: TRACK 1 - HOST ENGINE & VIRTUAL MACHINE MANUAL

Chapter 12: Track 1 - Terminal, REPL & Hover Inspection

Interactive REPL, Task Scheduler, OmniStudio HUD & 3-Tier Hover Cards

> ***Witness:*** *Colossians 2:19 (KJV) - The whole body fitly joined together*

SYSTEM TRACK
<TRACK 1> : {Host Engine & Virtual Machine}

CHAPTER IDENTIFIER
CH 12

NOTEBOOKLM WORD COUNT
30,031 words

CATALOGED CHAPTER FILES
24 files (7,894 lines)

RECOMMENDED NOTEBOOKLM COGNITIVE QUERY

"How does the hover engine bridge tape execution rolls back to original .omni source specifications?"

CHAPTER MANIFEST & INDEX (24 FILES)

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
1	**<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/README.md**	MD	117	848	6.7
2	**<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/mod.rs**	RS	90	387	3.5
3	**<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/scheduler/mod.rs**	RS	46	191	1.9
4	**<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/scheduler/scheduler.rs**	RS	492	1,935	20.5
5	**<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/scheduler/tasks.rs**	RS	234	853	8.4
6	**<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/session/commands.rs**	RS	801	2,660	32.4
7	**<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/session/mod.rs**	RS	37	185	1.8
8	**<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/session/session.rs**	RS	481	1,647	19.5
9	**<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/studio/hud.rs**	RS	173	740	8.4
10	**<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/studio/kinematics.rs**	RS	104	456	4.9
11	**<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/studio/mod.rs**	RS	36	186	1.7
12	**<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/universal/mod.rs**	RS	54	197	2.0
13	**<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/universal/ports.rs**	RS	215	844	7.3
14	**<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/universal/call_response_engine/call_response.rs**	RS	443	1,816	17.6
15	**<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/universal/call_response_engine/echo.rs**	RS	155	656	6.1
16	**<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/universal/call_response_engine/mod.rs**	RS	27	74	0.8
17	**<TRACK 1> : {Host Engine & Virtual Machine}/src/cli/mod.rs**	RS	44	157	2.0
18	**<TRACK 1> : {Host Engine & Virtual Machine}/src/cli/repl.rs**	RS	427	1,585	21.6
19	**<TRACK 1> : {Host Engine & Virtual Machine}/src/cli/server.rs**	RS	576	1,855	23.0
20	**<TRACK 1> : {Host Engine & Virtual Machine}/src/cli/commands/assemble.rs**	RS	544	2,088	23.7
21	**<TRACK 1> : {Host Engine & Virtual Machine}/src/cli/commands/boot.rs**	RS	782	3,860	39.1
22	**<TRACK 1> : {Host Engine & Virtual Machine}/src/cli/commands/mod.rs**	RS	44	150	1.8
23	**<TRACK 1> : {Host Engine & Virtual Machine}/src/cli/commands/run.rs**	RS	926	2,972	34.8
24	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/hover/mod.rs**	RS	1,046	3,689	43.2

<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/README.md MARKDOWN / DOC 117 LINES 848 WORDS 6.7 KB

OmniCode Terminal Subsystem

```
================================================================================
BLOCK 1: METADATA & ARCHITECTURAL MANIFESTO
================================================================================
Subsystem: OmniCode Host Engine & Virtual Machine Terminal
Layer:     Interactive Shell, Multi-Track Scheduler & Native Character Echo
Path:      <TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/
================================================================================
```

> ***"And he gave unto Moses, when he had made an end of communing with him upon mount Sinai, two tables of testimony, tables of stone, written with the finger of God."***
> *— Exodus 31:18 (KJV)*
> *(The immutable inscription of atomic runes onto bounded physical tablets).*
>
> ***"Prepare thy work without, and make it fit for thyself in the field; and afterwards build thine house."***
> *— Proverbs 24:27 (KJV)*
> *(The discipline of proving atomic sub-engines and verification invariants before constructing complex systems).*
>
> ***"Remove not the ancient landmark, which thy fathers have set."***
> *— Proverbs 22:28 (KJV)*
> *(The preservation of historical memory boundaries, silicon addresses, and machine constants).*

1. Architectural Mission

The **Terminal Subsystem** serves as the sensory-motor cortex and interactive execution environment of OmniCode. It bridges the virtual machine runtime (`src/runtime/`) to external users, host environments, and bare-metal hypervisors.

Rather than relying on external host shell forks or raw byte pass-throughs, the OmniCode Terminal natively executes process ribbons in persistent silicon memory, schedules cooperative tasks across three execution channels, and drives the **Native Character Invariant Echo Protocol**:

1. **Native Character Echo (`universal/`)**: Proves that every keypress or character stream ingests via `FROM ::@ 0x03F8` , inscribes into an atomic `Cell` , transduces into 5 balanced trits, verifies single-cell capacity bounds ($\le 1\text{ Cell}$), and projects outward via `INTO ::@ 0x000B8000` with affirmative `+1:PROCEED` .
2. **Multi-Track Cooperative Scheduler (`scheduler/`)**: Organizes execution across Track 0 (Kernel), Track 1 (Terminal), and Track 2 (App), carving RAM partitions out of the `Root_Storehouse` .
3. **Interactive Session (`session/`)**: Provides direct in-memory roll evaluation and shell command routing.
4. **Cinematic Projector (`studio/`)**: Renders real-time visual HUD frames of 4-directional Memory Reel kinematics.

2. Topological Layer Structure

```
src/terminal/
├── universal/                <-- Tier 1: Hardware MMIO Ports & Native Character Echo
│   ├── ports.rs              <-- FROM (0x03F8), INTO (0x000B8000), Stream Buffers
│   ├── character_echo.rs     <-- 5-Trit Radix-3 Bijective Character Lifecycle & Echo Engine
│   └── mod.rs                <-- Re-exports Universal I/O & Echo
│
├── scheduler/                <-- Tier 2: Multi-Track Cooperative Process Dispatcher
│   ├── tasks.rs              <-- OmniTask, Track, TaskPriority, JSON Models
│   ├── scheduler.rs          <-- OmniScheduler, Partition Carving, Tick Loop
│   └── mod.rs                <-- Re-exports Scheduler & Models
│
├── session/                  <-- Tier 3: Interactive Shell & Upgraded Character I/O
```

```
│   ├── commands.rs            <-- Built-in Shell Command Dispatcher
│   ├── session.rs             <-- TerminalSession Coordinator with Native Character Echo
│   └── mod.rs                 <-- Re-exports Session Engine
│
├── studio/                    <-- Tier 4: Cinematic Projector & Memory HUD
│   ├── hud.rs                 <-- ANSI Box-Drawing Frame Renderer
│   ├── kinematics.rs          <-- Reel Transport & Spindle Depth Controls
│   └── mod.rs                 <-- Re-exports Studio Engine
│
├── mod.rs                     <-- Subsystem Public Facade (100% Backward Compatible)
└── README.md                  <-- Architectural Manifesto & Specification
```

3. Subsystem Breakdown

3.1 Universal (`universal/`)

- **Hardware MMIO Ports (`ports.rs`)**:
- `UART_COM1_INPUT_PORT = 0x03F8` : Input port for serial keyboard stream.
- `VGA_TEXT_OUTPUT_PORT = 0x000B8000` : Output port for video console display.
- `TerminalInputStream` / `TerminalOutputStream` : Bounded FIFO ring buffer queues.
- **Native Character Echo Engine (`character_echo.rs`)**:
- Bijective Radix-3 transduction: 8-bit ASCII $\iff$ 5 balanced trits ${-1, 0, +1}^5$.
- Inscription into 5-trit `Cell` and single-cell capacity verification under `! GUARD` .
- Methods: `echo_char(glyph) -> Result<CharacterEchoResult, String>` and `echo_stream(&str) -> Result<String, String>` .

3.2 Multi-Track Scheduler (`scheduler/`)

- **`tasks.rs`** : Defines execution channels (`Track::Kernel` , `Track::Terminal` , `Track::App`), priorities (`High` , `Normal` , `Low` , `Background`), and the `OmniTask` descriptor.
- **`scheduler.rs`** : Manages process lifecycles, advances the cooperative `tick` clock, carves partitions from `Root_Storehouse` , and supports manual tape stepping (`step_task`).

3.3 Interactive Session (`session/`)

- **`commands.rs`** : Implements built-in shell commands (`ps` , `tracks` , `mem` , `mount` , `run` , `step` , `rolls` , `spawn` , `pause` , `resume` , `kill` , `tick` , `vars` , `help` , `let`).
- **`session.rs`** : Integrates `Environment` , `OmniScheduler` , directional streams, and direct roll evaluation. Exposes `feed_input_char` , `read_output_char` , `process_character_echo` , and `process_character_stream_echo` .

3.4 Cinema Studio (`studio/`)

- **`hud.rs`** : ANSI box-drawing projector HUD frame showing active reel, clapper cues, and set props.
- **`kinematics.rs`** : Reel motion evaluator supporting `PLAY` , `PAUSE` , `STOP` , `v--` , `^--` , `<--` , and `-->` .

4. The Terminal Echo Protocol

```
graph LR
    A["Host Keypress ('A' = 65)"] --> B["FROM Port (0x03F8 / %from)"]
    B --> C["ROLL 1: INSCRIBE [ Cell ]"]
    C --> D["ROLL 2: TRANSDUCE 5 Trits [-1, 1, 1, -1, 1]"]
    D --> E["ROLL 3: GUARD (<= 1 Cell)"]
    E --> F["ROLL 4: PROJECT [ Target ]"]
    F --> G["INTO Port (0x000B8000 / %into)"]
    G --> H["Host Terminal Screen ('A')"]
```

5. Architectural Invariants

1. **Bijective Transduction Parity**: Every byte transduced into 5 balanced trits must reconstruct back to the identical byte ($\sum_{i=0}^4 t_i \cdot 3^i$).

2. **Single-Cell Footprint**: Any single character glyph must evaluate to $\le 1\text{ Cell}$.
3. **Channel Isolation**: Kernel tasks on Track 0 cannot be preempted or modified by unprivileged App tasks on Track 2.
4. **Affirmative Seal**: All terminal rolls must commit in `+1:PROCEED`.

<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/mod.rs RS | 90 LINES | 387 WORDS | 3.5 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Terminal Subsystem Identity, Public Facade & Architectural Invariants )
// ============================================================================
//! OmniCode Terminal Subsystem Facade
//!
//! # Architectural Scope & Purpose:
//! Provides the primary interactive userspace shell, cooperative process scheduler,
//! hardware MMIO directional stream buffers, native Character Echo Engine, and
//! cinematic HUD projector for the OmniCode platform and upcoming Operating System.
//!
//! Subsystems:
//! - `universal`: Hardware MMIO ports (`0x03F8`, `0x000B8000`), directional streams, and 5-trit Character Echo Engine.
//! - `scheduler`: Multi-Track cooperative task manager (`Track::Kernel`, `Track::Terminal`, `Track::App`).
//! - `session`: Interactive REPL shell, persistent variable state, and direct in-memory roll evaluator.
//! - `studio`: Cinematic HUD projector rendering 4-directional Memory Reel execution frames.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of physical data into bounded terminal tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic terminal I/O engines before constructing user interfaces).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical hardware ports and execution track landmarks).
// ============================================================================

pub mod universal;
pub mod scheduler;
pub mod session;
pub mod studio;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Re-export Declarations for Universal Access & 100% Backward Compatibility )
// ============================================================================

// Universal Terminal Substrate
pub use universal::{
    UART_COM1_INPUT_PORT,
    VGA_TEXT_OUTPUT_PORT,
    DEFAULT_TERMINAL_BUFFER_CAPACITY,
    TerminalInputStream,
    TerminalOutputStream,
    Call,
    Response,
    CharacterResponse,
    NumberResponse,
    StreamResponse,
    CallResponseEngine,
    EchoEngine,
    EchoResult,
};

// Multi-Track Scheduler
pub use scheduler::{
    OmniScheduler,
    OmniTask,
    SharedScheduler,
    TaskPriority,
    Track,
    create_shared_scheduler,
};

// Interactive Session
pub use session::{
    ExecutionResult,
    TerminalSession,
};

```

```
// Cinema Studio
pub use studio::OmniStudio;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Subsystem Proofs & Invariants )
// ============================================================================
// Terminal processes and character echo loops are coordinated cooperatively
// across the 3 execution channels with persistent state in the Environment.

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Verification & Module Closure )
// ============================================================================
// Terminal Subsystem facade verified and sealed in affirmative PROCEED.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/scheduler/mod.rs RS | 46 LINES | 191 WORDS | 1.9 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Multi-Track Scheduler Subsystem Identity & Public Exports )
// ============================================================================
//! OmniCode Terminal: Multi-Track Scheduler Subsystem Module
//!
//! Provides the primordial process and task scheduling engine,
//! 3-track execution channels, priority queues, and cooperative tick loops.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**: Tables of testimony, written with the finger of God.
//! - **Proverbs 24:27 (KJV)**: Prepare thy work without, and make it fit for thyself in the field.
//! - **Proverbs 22:28 (KJV)**: Remove not the ancient landmark, which thy fathers have set.
// ============================================================================

pub mod tasks;
pub mod scheduler;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Re-export Declarations for Scheduler & Tasks )
// ============================================================================
pub use tasks::{
    Track,
    TaskPriority,
    OmniTask,
};

pub use scheduler::{
    OmniScheduler,
    SharedScheduler,
    create_shared_scheduler,
};

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Subsystem Invariants & Cooperative Scheduling Guarantees )
// ============================================================================
// Task execution is isolated across designated channels (Kernel, Terminal, App).

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Verification & Module Closure )
// ============================================================================
// Multi-Track Scheduler subsystem verified and closed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/scheduler/scheduler.rs | RS | 492 LINES | 1935 WORDS | 20.5 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Multi-Track Cooperative Process Scheduler Engine & Lifecycle Dispatcher )
// ================================================================================
//! OmniCode Terminal: Multi-Track Process Scheduler Engine
//!
//! # Architectural Scope & Purpose:
//! Provides the primary process and task scheduling engine for the OmniCode
//! Native Terminal and OS userspace shell:
//! - Balanced Radix-3 process lifecycles (+1:Proceed, 0:Await, -1:Rollback, NULL:Halt).
//! - Multi-track execution: Track 0 (Kernel), Track 1 (Terminal), Track 2 (App).
//! - Root_Storehouse partition carving for mounted tape process ribbons.
//! - Cooperative `tick` clock and manual tape roll stepping (`step_task`).
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of process state onto physical tables of memory).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving cooperative task scheduling before dispatching shell interfaces).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical execution channel boundaries across kernel, terminal, and apps).
// ================================================================================

use std::path::Path;
use std::sync::{Arc, Mutex};
use std::time::{SystemTime, UNIX_EPOCH};

use crate::core::tape::Tape;
use crate::core::ternary::QuadState;
use crate::core::vessel::Vessel;
use crate::diagnostics::logger::DiagnosticLogger;

use super::tasks::{OmniTask, TaskPriority, Track};

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( OmniScheduler Struct Definition, Constructor & Defaults )
// ================================================================================

/// Primary multi-track cooperative process scheduler.
#[derive(Debug)]
pub struct OmniScheduler {
    tasks: Vec<OmniTask>,
    next_id: u64,
    /// Root storehouse from which task RAM partitions are carved.
    pub root_storehouse: Vessel,
}

impl Default for OmniScheduler {
    fn default() -> Self {
        let mut scheduler = Self {
            tasks: Vec::new(),
            next_id: 1,
            root_storehouse: Vessel::new_storehouse("Root_RAM", 4096),
        };
        // Initialize default OS background tasks on Track 0 (Kernel)
        scheduler.spawn_on_track("CoreDaemon", "omnicode kernel --daemon", TaskPriority::Background, Track::Kernel);
        scheduler.spawn_on_track("MemoryWatcher", "omnicode mem --watch", TaskPriority::Low, Track::Kernel);
        scheduler
    }
}

// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Scheduler Core: Process Dispatcher, Partition Carving & Tick Engine )
// ================================================================================

impl OmniScheduler {
```

```
    /// Constructs a new scheduler initialized with default kernel daemons.
    pub fn new() -> Self {
        Self::default()
    }

    /// Spawns a new task in the scheduler on Track 2 (App) by default.
    pub fn spawn(&mut self, name: &str, command: &str, priority: TaskPriority) -> u64 {
        self.spawn_on_track(name, command, priority, Track::App)
    }

    /// Spawns a new task on a designated execution track (Kernel, Terminal, or App).
    pub fn spawn_on_track(&mut self, name: &str, command: &str, priority: TaskPriority, track: Track) -> u64 {
        let id = self.next_id;
        self.next_id += 1;
        let mut task = OmniTask::with_track(id, name.to_string(), command.to_string(), priority, track);
        task.log_output.push(format!(
            "[*] Task spawned on {} with state +1:PROCEED (Priority: {})",
            track.as_str(),
            priority.as_str()
        ));
        self.tasks.push(task);
        id
    }

    /// Mounts a structured [`Tape`] onto an execution track, carving an isolated memory partition.
    pub fn mount_tape(&mut self, track: Track, mut tape: Tape) -> u64 {
        let id = self.next_id;
        self.next_id += 1;

        let part_size = tape.head.required_partition_size;
        let partition_res = self.root_storehouse.partition(&tape.head.name, part_size);

        tape.bound_partition = Some(tape.head.name.clone());
        let priority = match track {
            Track::Kernel => TaskPriority::High,
            Track::Terminal => TaskPriority::Normal,
            Track::App => TaskPriority::Normal,
        };

        let mut task = OmniTask::with_track(
            id,
            tape.head.name.clone(),
            format!("tape:{}", tape.head.name),
            priority,
            track,
        );

        let part_info = match partition_res {
            Ok(p) => {
                let msg = format!("carved partition '{}' ({} cells)", p.name, p.capacity);
                task.partition = Some(p);
                msg
            }
            Err(e) => format!("partition warning: {e}"),
        };

        task.log_output.push(format!(
            "[+] Mounted Tape '{}' on {} ({}, {} rolls)",
            tape.head.name,
            track.as_str(),
            part_info,
            tape.roll_count()
        ));

        task.tape = Some(tape);
        self.tasks.push(task);
        id
    }

    /// Mounts and parses a `.tape` file from disk onto the specified track.
    pub fn mount_tape_file(&mut self, track: Track, path: impl AsRef<Path>, logger: DiagnosticLogger) -> Result<u64,
String> {
        let tape = Tape::from_file(path, logger)?;
        Ok(self.mount_tape(track, tape))
    }

    /// Advances the scheduler clock by 1 tick, executing tasks in Proceed state.
    ///
```

```
    /// For tasks with mounted Tapes, steps the Tape Head forward roll-by-roll.
    pub fn tick(&mut self) -> usize {
        let mut active_executed = 0;
        let now = SystemTime::now()
            .duration_since(UNIX_EPOCH)
            .unwrap_or_default()
            .as_secs();

        for task in &mut self.tasks {
            if task.state == QuadState::Proceed {
                task.ticks += 1;
                task.updated_at = now;
                active_executed += 1;

                if let Some(ref mut tape) = task.tape {
                    if !tape.is_finished() {
                        if let Some(roll) = tape.active_roll() {
                            task.log_output.push(format!("==> [HEAD @ Roll {}] {}", tape.cursor + 1, roll.text));
                        }
                        tape.advance();
                        if tape.is_finished() {
                            task.log_output.push(format!(
                                "[√] Tape '{}' execution completed successfully (all {} rolls verified)",
                                tape.head.name,
                                tape.roll_count()
                            ));
                        }
                    }
                }
            }
        }
        active_executed
    }

    /// Manually steps the Tape Head on the given task by 1 roll, executing transport kinematics.
    pub fn step_task(&mut self, id: u64) -> Result<String, String> {
        if let Some(task) = self.tasks.iter_mut().find(|t| t.id == id) {
            if let Some(ref mut tape) = task.tape {
                if !tape.is_finished() {
                    let cur_idx = tape.cursor + 1;
                    let roll_text = tape.active_roll().map(|r| r.text.clone()).unwrap_or_default();

                    // Check for STOP transport action
                    let is_unconditional_stop = roll_text.contains("STOP") || (roll_text.contains("HALT") &&
!roll_text.contains("HALT WHEN"));
                    if is_unconditional_stop {
                        tape.cursor = tape.roll_count();
                        task.state = QuadState::Halt;
                        if let Some(ref mut reel) = task.reel {
                            reel.stop();
                        }
                        let msg = format!("[■ STOP] Task {id}: Execution stopped and sealed at Roll {cur_idx}:
{roll_text}");
                        task.log_output.push(msg.clone());
                        return Ok(msg);
                    }

                    // Check for REWIND transport loop action
                    if roll_text.contains("REWIND") {
                        let cue_opt = if let Some(start) = roll_text.find('<') {
                            if let Some(end) = roll_text[start..].find('>') {
                                Some(roll_text[start + 1..start + end].trim().to_string())
                            } else {
                                None
                            }
                        } else {
                            None
                        };

                        if let Some(cue) = cue_opt {
                            if task.loop_count < task.max_loops {
                                if let Some(target_idx) = tape.find_landmark_index(&cue) {
                                    task.loop_count += 1;
                                    tape.rewind_to(target_idx);
                                    if let Some(ref mut reel) = task.reel {
                                        let _ = reel.rewind_to_landmark(&cue);
                                    }
                                    let msg = format!(
```

```
                                        "[⏮ REWIND] Task {id}: Track rewound to Landmark <{cue}> at Roll {} (Iteration {}/{})",
                                        target_idx + 1, task.loop_count, task.max_loops
                                    );
                                    task.log_output.push(msg.clone());
                                    return Ok(msg);
                                }
                            }
                        }
                    }

                    // Check for PAUSE transport action
                    if roll_text.contains("PAUSE") {
                        tape.advance();
                        task.state = QuadState::Await;
                        if let Some(ref mut reel) = task.reel {
                            reel.pause();
                        }
                        let msg = format!("[⏸ PAUSE] Task {id}: Track paused at Roll {cur_idx}: {roll_text}");
                        task.log_output.push(msg.clone());
                        return Ok(msg);
                    }

                    // Default: PLAY (forward advance)
                    tape.advance();
                    if let Some(ref mut reel) = task.reel {
                        reel.play();
                    }
                    let msg = format!("[▶ PLAY] Step PID {id}: Advanced to Roll {cur_idx}/{}: {roll_text}", tape.roll_count());
                    task.log_output.push(msg.clone());
                    Ok(msg)
                } else {
                    Ok(format!("[√] Task {id} Tape '{}' has already executed all {} rolls", task.name, tape.roll_count()))
                }
            } else {
                Err(format!("Task {id} ('{}') has no active mounted Tape ribbon", task.name))
            }
        } else {
            Err(format!("Task ID {id} not found"))
        }
    }

    /// Displays all rolls of a task's tape with the live `==> [HEAD]` pointer.
    pub fn rolls_for_task(&self, id: u64) -> Option<String> {
        let task = self.tasks.iter().find(|t| t.id == id)?;
        let tape = task.tape.as_ref()?;

        let mut out = String::new();
        out.push_str(&format!("TAPE RIBBON: [{}] (PID: {})\n", tape.head.name, task.id));
        out.push_str(&format!("  Channel:   {}\n", task.track.as_str()));
        out.push_str(&format!("  Tape Head: {}\n", tape.head.raw_text));
        out.push_str("------------------------------------------------------------\n");

        for (i, roll) in tape.rolls.iter().enumerate() {
            if i == tape.cursor && !tape.is_finished() {
                out.push_str(&format!("==> [{:>2}] {}\n", i + 1, roll.text));
            } else if i < tape.cursor {
                out.push_str(&format!("  √ [{:>2}] {}\n", i + 1, roll.text));
            } else {
                out.push_str(&format!("    [{:>2}] {}\n", i + 1, roll.text));
            }
        }

        if tape.is_finished() {
            out.push_str("------------------------------------------------------------\n");
            out.push_str("  [√] TAPE EXECUTION COMPLETE (Sealed with PROCEED)\n");
        }
        Some(out)
    }

    /// Returns memory storehouse utilization and active partition breakdown.
    pub fn mem_summary(&self) -> String {
        let total = self.root_storehouse.capacity;
        let allocated = self.root_storehouse.count;
        let free = total.saturating_sub(allocated);
        let pct = if total > 0 { (allocated * 100) / total } else { 0 };
```

```
302
303         let mut out = String::new();
304         out.push_str("============================================================\n");
305         out.push_str("OMNICODE ROOT STOREHOUSE & TASK PARTITIONS\n");
306         out.push_str("============================================================\n");
307         out.push_str(&format!("Root_RAM:     {allocated} / {total} cells ({pct}% Allocated, {free} cells Free)\n"));
308
309         let filled_bars = pct / 5;
310         let empty_bars = 20 - filled_bars;
311         out.push_str(&format!("RAM Meter:    [{}{}] {}%\n", "".repeat(filled_bars), "".repeat(empty_bars), pct));
312         out.push_str("------------------------------------------------------------\n");
313         out.push_str("TASK PARTITION ALLOCATIONS:\n");
314
315         let mut found_part = false;
316         for t in &self.tasks {
317             if let Some(ref p) = t.partition {
318                 found_part = true;
319                 out.push_str(&format!("  • PID {:<3} [{:<16}] Capacity: {:>4} cells (Track: {})\n", t.id, p.name,
p.capacity, t.track.as_str()));
320             }
321         }
322         if !found_part {
323             out.push_str("  (No active task partitions carved yet)\n");
324         }
325         out.push_str("============================================================\n");
326         out
327     }
328
329     /// Returns a structured summary grouping tasks by Track.
330     pub fn track_summary(&self) -> String {
331         let mut out = String::new();
332         out.push_str("============================================================\n");
333         out.push_str("MULTI-TRACK EXECUTION CHANNELS\n");
334         out.push_str("============================================================\n");
335
336         for track in [Track::Kernel, Track::Terminal, Track::App] {
337             out.push_str(&format!("\n[ {} ]\n", track.as_str()));
338             let track_tasks: Vec<&OmniTask> = self.tasks.iter().filter(|t| t.track == track).collect();
339             if track_tasks.is_empty() {
340                 out.push_str("  • (No active processes on this track)\n");
341             } else {
342                 for t in track_tasks {
343                     let tape_tag = if t.tape.is_some() { "[TAPE]" } else { "      " };
344                     let state_badge = match t.state {
345                         QuadState::Proceed => "+1:PROCEED",
346                         QuadState::Await => " 0:AWAIT  ",
347                         QuadState::Rollback => "-1:ROLLBCK",
348                         QuadState::Halt => "NULL:HALT ",
349                     };
350                     out.push_str(&format!("  • PID {:<3} {} {:<10} {:<16} ticks: {}\n", t.id, tape_tag, state_badge,
t.name, t.ticks));
351                 }
352             }
353         }
354         out
355     }
356
357     /// Transitions a task to 0:AWAIT (Standby/Paused).
358     pub fn pause_task(&mut self, id: u64) -> Result<(), String> {
359         if let Some(task) = self.tasks.iter_mut().find(|t| t.id == id) {
360             task.state = QuadState::Await;
361             task.log_output.push("[~] Task transitioned to 0:AWAIT (Suspended)".to_string());
362             Ok(())
363         } else {
364             Err(format!("Task ID {id} not found"))
365         }
366     }
367
368     /// Resumes a task back to +1:PROCEED (Running).
369     pub fn resume_task(&mut self, id: u64) -> Result<(), String> {
370         if let Some(task) = self.tasks.iter_mut().find(|t| t.id == id) {
371             task.state = QuadState::Proceed;
372             task.log_output.push("[+] Task resumed to +1:PROCEED".to_string());
373             Ok(())
374         } else {
375             Err(format!("Task ID {id} not found"))
376         }
377     }
```

```
378
379     /// Signals a rollback on a task (-1:ROLLBACK).
380     pub fn rollback_task(&mut self, id: u64, reason: &str) -> Result<(), String> {
381         if let Some(task) = self.tasks.iter_mut().find(|t| t.id == id) {
382             task.state = QuadState::Rollback;
383             task.log_output.push(format!("[-] Task rolled back to -1:ROLLBACK. Reason: {reason}"));
384             Ok(())
385         } else {
386             Err(format!("Task ID {id} not found"))
387         }
388     }
389
390     /// Halts and terminates a task (NULL:HALT).
391     pub fn halt_task(&mut self, id: u64) -> Result<(), String> {
392         if let Some(task) = self.tasks.iter_mut().find(|t| t.id == id) {
393             task.state = QuadState::Halt;
394             task.log_output.push("[!] Task terminated with state NULL:HALT".to_string());
395             Ok(())
396         } else {
397             Err(format!("Task ID {id} not found"))
398         }
399     }
400
401     /// Appends output log line to a task.
402     pub fn log_task(&mut self, id: u64, line: &str) {
403         if let Some(task) = self.tasks.iter_mut().find(|t| t.id == id) {
404             task.log_output.push(line.to_string());
405         }
406     }
407
408     /// Returns all tasks formatted as a JSON array.
409     pub fn to_json(&self) -> String {
410         let task_jsons: Vec<String> = self.tasks.iter().map(|t| t.to_json()).collect();
411         format!("[{}]", task_jsons.join(","))
412     }
413
414     /// Returns a human-readable table of all running processes.
415     pub fn format_table(&self) -> String {
416         let mut out = String::new();
417         out.push_str("PID   STATE        CHANNEL         PRIORITY    TICKS   COMMAND\n");
418         out.push_str("-------------------------------------------------------------------------------\n");
419         for t in &self.tasks {
420             let state_badge = match t.state {
421                 QuadState::Proceed => "+1 PROCEED",
422                 QuadState::Await => " 0 AWAIT  ",
423                 QuadState::Rollback => "-1 ROLLBCK",
424                 QuadState::Halt => "NULL HALT ",
425             };
426             out.push_str(&format!(
427                 "{:<5} {:<12} {:<15} {:<11} {:<7} {}\n",
428                 t.id,
429                 state_badge,
430                 t.track.as_str(),
431                 t.priority.as_str(),
432                 t.ticks,
433                 t.command
434             ));
435         }
436         out
437     }
438
439     /// Returns a reference to all tasks in the scheduler.
440     pub fn tasks(&self) -> &[OmniTask] {
441         &self.tasks
442     }
443
444     /// Finds a task by ID.
445     pub fn get_task(&self, id: u64) -> Option<&OmniTask> {
446         self.tasks.iter().find(|t| t.id == id)
447     }
448
449     /// Finds a mutable reference to a task by ID.
450     pub fn get_task_mut(&mut self, id: u64) -> Option<&mut OmniTask> {
451         self.tasks.iter_mut().find(|t| t.id == id)
452     }
453 }
454
455 // ============================================================================
```

```rust
// BLOCK 4: CLOSING BLOCK
// ( Global Thread-Safe Scheduler Handle & Verification Tests )
// ================================================================================

/// Thread-safe shared scheduler pointer wrapped in `Arc<Mutex<OmniScheduler>>`.
pub type SharedScheduler = Arc<Mutex<OmniScheduler>>;

/// Factory function creating a thread-safe shared scheduler instance.
pub fn create_shared_scheduler() -> SharedScheduler {
    Arc::new(Mutex::new(OmniScheduler::new()))
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_scheduler_lifecycle() {
        let mut sched = OmniScheduler::new();
        let pid = sched.spawn("TestProcess", "omnicode test", TaskPriority::High);

        assert_eq!(sched.tasks.iter().find(|t| t.id == pid).unwrap().state, QuadState::Proceed);

        sched.tick();
        assert_eq!(sched.tasks.iter().find(|t| t.id == pid).unwrap().ticks, 1);

        sched.pause_task(pid).unwrap();
        assert_eq!(sched.tasks.iter().find(|t| t.id == pid).unwrap().state, QuadState::Await);

        sched.resume_task(pid).unwrap();
        assert_eq!(sched.tasks.iter().find(|t| t.id == pid).unwrap().state, QuadState::Proceed);

        sched.halt_task(pid).unwrap();
        assert_eq!(sched.tasks.iter().find(|t| t.id == pid).unwrap().state, QuadState::Halt);
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/scheduler/tasks.rs RS 234 LINES 853 WORDS 8.4 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Multi-Track Execution Channels, Task Priorities & Process Descriptor )
// ================================================================================
//! OmniCode Terminal: Multi-Track Execution Channels & Task Models
//!
//! # Architectural Scope & Purpose:
//! Encapsulates process and task data models for the OmniCode Terminal Scheduler:
//! - 3 Physical execution channels: [`Track::Kernel`] (0), [`Track::Terminal`] (1), [`Track::App`] (2).
//! - 4 Priority tiers: [`TaskPriority::High`], [`TaskPriority::Normal`], [`TaskPriority::Low`],
[`TaskPriority::Background`].
//! - [`OmniTask`]: Persistent descriptor managing process state, bound tape, memory partition, and logs.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of process descriptors into bounded memory tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving process descriptor models before dispatching scheduler loops).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical execution channel boundaries across kernel, terminal, and apps).
// ================================================================================

use std::time::{SystemTime, UNIX_EPOCH};

use crate::core::reel::MemoryReel;
use crate::core::tape::Tape;
use crate::core::ternary::QuadState;
use crate::core::vessel::Vessel;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Track & TaskPriority Enums and OmniTask Definition )
// ================================================================================

/// The 3 physical execution channels of the Multi-Track Scheduler.
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub enum Track {
    /// Track 0: Hardware / Kernel Genesis / Core Daemons.
    Kernel = 0,
    /// Track 1: Terminal Shell & Console HUD.
    Terminal = 1,
    /// Track 2: Userspace & Worker Applications.
    App = 2,
}

impl Track {
    /// Returns the canonical display string for this execution channel.
    pub fn as_str(&self) -> &'static str {
        match self {
            Track::Kernel => "TRACK 0 [KERNEL]",
            Track::Terminal => "TRACK 1 [TERMINAL]",
            Track::App => "TRACK 2 [APP]",
        }
    }
}

/// Scheduling priority tier for active processes.
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub enum TaskPriority {
    High = 3,
    Normal = 2,
    Low = 1,
    Background = 0,
}

impl TaskPriority {
    /// Returns the uppercase text representation of this priority.
    pub fn as_str(&self) -> &'static str {
        match self {
```

```
            TaskPriority::High => "HIGH",
            TaskPriority::Normal => "NORMAL",
            TaskPriority::Low => "LOW",
            TaskPriority::Background => "BACKGROUND",
        }
    }

    /// Resolves a string into a TaskPriority variant.
    pub fn from_str(s: &str) -> Self {
        match s.trim().to_uppercase().as_str() {
            "HIGH" | "3" => TaskPriority::High,
            "LOW" | "1" => TaskPriority::Low,
            "BACKGROUND" | "IDLE" | "0" => TaskPriority::Background,
            _ => TaskPriority::Normal,
        }
    }
}

/// Descriptor representing an active, waiting, or halted process in the terminal.
#[derive(Debug, Clone)]
pub struct OmniTask {
    /// Unique process identifier (PID).
    pub id: u64,
    /// Human-readable process name.
    pub name: String,
    /// Shell command or entry point string.
    pub command: String,
    /// Scheduling priority tier.
    pub priority: TaskPriority,
    /// Designated execution channel.
    pub track: Track,
    /// Balanced ternary process state (+1:Proceed, 0:Await, -1:Rollback, NULL:Halt).
    pub state: QuadState,
    /// UNIX timestamp of process creation.
    pub created_at: u64,
    /// UNIX timestamp of last update or tick.
    pub updated_at: u64,
    /// Total cooperative scheduler ticks elapsed.
    pub ticks: u64,
    /// Log output ring buffer.
    pub log_output: Vec<String>,
    /// Mounted process tape ribbon, if executing tape-based logic.
    pub tape: Option<Tape>,
    /// Bounded RAM partition carved from Root_Storehouse.
    pub partition: Option<Vessel>,
    /// 4-directional memory reel for cinematic HUD stepping.
    pub reel: Option<MemoryReel>,
    /// Current loop counter.
    pub loop_count: usize,
    /// Maximum allowed loops before watchdog intervention.
    pub max_loops: usize,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( OmniTask Constructors & Serializers )
// ============================================================================

impl OmniTask {
    /// Constructs a new task on Track 2 (App) by default.
    pub fn new(id: u64, name: String, command: String, priority: TaskPriority) -> Self {
        Self::with_track(id, name, command, priority, Track::App)
    }

    /// Constructs a new task on the specified execution channel.
    pub fn with_track(id: u64, name: String, command: String, priority: TaskPriority, track: Track) -> Self {
        let now = SystemTime::now()
            .duration_since(UNIX_EPOCH)
            .unwrap_or_default()
            .as_secs();

        Self {
            id,
            name: name.clone(),
            command,
            priority,
            track,
            state: QuadState::Proceed,
```

```
            created_at: now,
            updated_at: now,
            ticks: 0,
            log_output: Vec::new(),
            tape: None,
            partition: None,
            reel: Some(MemoryReel::new(name, 64)),
            loop_count: 0,
            max_loops: 100,
        }
    }

    /// Serializes the task descriptor into a structured JSON string.
    pub fn to_json(&self) -> String {
        let state_str = match self.state {
            QuadState::Proceed => "+1:PROCEED",
            QuadState::Await => "0:AWAIT",
            QuadState::Rollback => "-1:ROLLBACK",
            QuadState::Halt => "NULL:HALT",
        };

        let logs = self.log_output.join("\\n").replace('"', "\\\"");

        format!(
            r#"{{"id":{},"name":"{}","command":"{}","priority":"{}","track":"{}","state":"{}","ticks":{},"created_at":{},"logs":"{}"}}"#,
            self.id,
            self.name,
            self.command.replace('"', "\\\""),
            self.priority.as_str(),
            self.track.as_str(),
            state_str,
            self.ticks,
            self.created_at,
            logs
        )
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Test Verification Proofs & Invariants )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_track_hierarchy_and_names() {
        assert_eq!(Track::Kernel.as_str(), "TRACK 0 [KERNEL]");
        assert_eq!(Track::Terminal.as_str(), "TRACK 1 [TERMINAL]");
        assert_eq!(Track::App.as_str(), "TRACK 2 [APP]");
    }

    #[test]
    fn test_task_priority_resolution() {
        assert_eq!(TaskPriority::from_str("high"), TaskPriority::High);
        assert_eq!(TaskPriority::from_str("1"), TaskPriority::Low);
        assert_eq!(TaskPriority::from_str("idle"), TaskPriority::Background);
        assert_eq!(TaskPriority::from_str("unknown"), TaskPriority::Normal);
    }

    #[test]
    fn test_omni_task_creation_and_json() {
        let task = OmniTask::with_track(
            42,
            "InitDaemon".into(),
            "boot --init".into(),
            TaskPriority::High,
            Track::Kernel,
        );
        assert_eq!(task.id, 42);
        assert_eq!(task.track, Track::Kernel);
        assert_eq!(task.state, QuadState::Proceed);

        let json = task.to_json();
        assert!(json.contains("\"id\":42"));
```

```
        assert!(json.contains("\"name\":\"InitDaemon\""));
        assert!(json.contains("\"priority\":\"HIGH\""));
        assert!(json.contains("\"track\":\"TRACK 0 [KERNEL]\""));
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/session/commands.rs RS 801 LINES 2660 WORDS 32.4 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Terminal Built-in Shell Command Dispatcher & Host OS Interlock )
// ================================================================================
//! OmniCode Terminal: Built-in Shell Command Dispatcher
//!
//! # Architectural Scope & Purpose:
//! Handles parsing and execution of native operating system shell commands:
//! - Process table inspection (`ps`, `tasks`).
//! - Multi-track execution channel inspection (`tracks`, `channels`).
//! - Storehouse RAM utilization meter (`mem`, `ram`).
//! - Tape mounting and step execution (`mount`, `run`, `step`, `rolls`).
//! - Cooperative scheduler clock stepping (`tick`).
//! - Process control (`spawn`, `pause`, `resume`, `kill`, `halt`).
//! - Variable declarations (`let x = 42`) and environment inspection (`vars`, `env`).
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**: Tables of testimony, written with the finger of God.
//! - **Proverbs 24:27 (KJV)**: Prepare thy work without, and make it fit for thyself in the field.
//! - **Proverbs 22:28 (KJV)**: Remove not the ancient landmark, which thy fathers have set.
// ================================================================================

use std::collections::HashMap;
use std::time::Instant;

use crate::cogs::tripwire::Tripwire;
use crate::core::filetype::resolve_track_path;
use crate::core::ternary::QuadState;
use crate::diagnostics::debugger::OmniDebugger;
use crate::diagnostics::logger::{DiagnosticLogger, FailpointType};
use crate::runtime::environment::Environment;
use crate::terminal::scheduler::{SharedScheduler, TaskPriority, Track};

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( ExecutionResult Container & Command Dispatch Function Signature )
// ================================================================================

/// Result of an interactive terminal command execution.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct ExecutionResult {
    /// Text emitted to stdout during execution.
    pub stdout: String,
    /// Balanced ternary state following execution (+1:Proceed, 0:Await, -1:Rollback, NULL:Halt).
    pub state: QuadState,
    /// Execution duration in microseconds.
    pub duration_us: u128,
}

impl ExecutionResult {
    /// Constructs a successful execution result with state +1:PROCEED.
    pub fn success(stdout: String, duration_us: u128) -> Self {
        Self {
            stdout,
            state: QuadState::Proceed,
            duration_us,
        }
    }

    /// Constructs an asynchronous wait or standby result with state 0:AWAIT.
    pub fn await_state(stdout: String, duration_us: u128) -> Self {
        Self {
            stdout,
            state: QuadState::Await,
            duration_us,
        }
    }

    /// Constructs a failed or rollback result with state -1:ROLLBACK.
    pub fn failure(stdout: String, duration_us: u128) -> Self {
        Self {
            stdout,
            state: QuadState::Rollback,
            duration_us,
```

```
        }
    }
}

// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Built-in Shell Command Dispatch Engine )
// ================================================================================

/// Dispatches built-in terminal shell commands. Returns `Some(ExecutionResult)` if
/// the command was handled, or `None` if it should fall through to roll evaluation.
pub fn dispatch_builtin_command(
    trimmed: &str,
    scheduler: &SharedScheduler,
    env: &Environment,
    variables: &mut HashMap<String, String>,
    logger: &DiagnosticLogger,
    _debugger: &OmniDebugger,
    start: Instant,
) -> Option<ExecutionResult> {
    // 0. Universal Call & Response Engine (ECHO command)
    if trimmed == "echo" || trimmed.starts_with("echo ") {
        let raw_payload = if trimmed == "echo" {
            ""
        } else {
            trimmed["echo ".len()..].trim()
        };
        let payload = if (raw_payload.starts_with('\'') && raw_payload.ends_with('\'') && raw_payload.len() >= 2)
            || (raw_payload.starts_with('"') && raw_payload.ends_with('"') && raw_payload.len() >= 2)
        {
            &raw_payload[1..raw_payload.len() - 1]
        } else {
            raw_payload
        };

        // Determine if payload is a numeric/binary scalar Call
        let num_parse = if payload.starts_with("0b") || payload.starts_with("0B") {
            i64::from_str_radix(&payload[2..], 2).ok()
        } else {
            payload.parse::<i64>().ok()
        };

        if let Some(n) = num_parse {
            match crate::terminal::universal::call_response_engine::CallResponseEngine::call_number(n) {
                Ok(resp) => {
                    let out = format!(
                        "Number: {} | Dual-Rail: 0x{:016X} | Pack(2-2-2-2): 0x{:02X} | Pack(2-1-2-1-2): 0x{:02X} |
Equilibrium: PROVED | State: +1:PROCEED",
                        resp.scalar, resp.packed_dual_rail, resp.byte_pack_2222, resp.byte_pack_21212
                    );
                    return Some(ExecutionResult::success(out, start.elapsed().as_micros()));
                }
                Err(e) => {
                    let err = Tripwire::trip(
                        logger,
                        "TERMINAL_ECHO",
                        FailpointType::SyntaxAnomaly,
                        format!("Number Call-Response failed: {e}"),
                    );
                    return Some(ExecutionResult::failure(
                        format!("[!] {err}"),
                        start.elapsed().as_micros(),
                    ));
                }
            }
        }

        // Otherwise execute Stream / Character Call-Response turn
        match crate::terminal::universal::call_response_engine::CallResponseEngine::call_stream(payload) {
            Ok(resp) => {
                return Some(ExecutionResult::success(resp.text, start.elapsed().as_micros()));
            }
            Err(e) => {
                let err = Tripwire::trip(
                    logger,
                    "TERMINAL_ECHO",
                    FailpointType::SyntaxAnomaly,
                    format!("Echo Call-Response failed: {e}"),
```

```
                );
                return Some(ExecutionResult::failure(
                    format!("[!] {err}"),
                    start.elapsed().as_micros(),
                ));
            }
        }
    }

    // 0b. Explicit number call (Convenience alias)
    if trimmed.starts_with("number ") || trimmed.starts_with("num ") {
        let raw = if trimmed.starts_with("number ") {
            trimmed["number ".len()..].trim()
        } else {
            trimmed["num ".len()..].trim()
        };
        if let Ok(n) = raw.parse::<i64>() {
            match crate::terminal::universal::call_response_engine::CallResponseEngine::call_number(n) {
                Ok(resp) => {
                    let out = format!(
                        "Number: {} | Dual-Rail: 0x{:016X} | Pack(2-2-2-2): 0x{:02X} | Pack(2-1-2-1-2): 0x{:02X} |
Equilibrium: PROVED | State: +1:PROCEED",
                        resp.scalar, resp.packed_dual_rail, resp.byte_pack_2222, resp.byte_pack_21212
                    );
                    return Some(ExecutionResult::success(out, start.elapsed().as_micros()));
                }
                Err(e) => {
                    return Some(ExecutionResult::failure(
                        format!("[!] Number call failed: {e}"),
                        start.elapsed().as_micros(),
                    ));
                }
            }
        }
    }

    // 1. Process table
    if trimmed == "ps" || trimmed == "tasks" {
        let sched = scheduler.lock().unwrap();
        let table = sched.format_table();
        return Some(ExecutionResult::success(table, start.elapsed().as_micros()));
    }

    // 2. Multi-track channel summary
    if trimmed == "tracks" || trimmed == "channels" {
        let sched = scheduler.lock().unwrap();
        let summary = sched.track_summary();
        return Some(ExecutionResult::success(summary, start.elapsed().as_micros()));
    }

    // 3. Storehouse memory meter
    if trimmed == "mem" || trimmed == "ram" {
        let sched = scheduler.lock().unwrap();
        let summary = sched.mem_summary();
        return Some(ExecutionResult::success(summary, start.elapsed().as_micros()));
    }

    // 4. Mount a tape file onto Track 2 (App) or Track 0 (Kernel)
    if trimmed.starts_with("mount ") {
        let raw_path = trimmed["mount ".len()..].trim();
        let resolved = resolve_track_path(raw_path);
        let path_str = resolved.to_string_lossy().to_string();
        let mut sched = scheduler.lock().unwrap();
        let track = if raw_path.contains("kernel") || raw_path.contains("boot") {
            Track::Kernel
        } else {
            Track::App
        };
        match sched.mount_tape_file(track, &path_str, logger.clone()) {
            Ok(pid) => return Some(ExecutionResult::success(
                format!("[+] Mounted Tape '{raw_path}' onto {} with PID {pid}", track.as_str()),
                start.elapsed().as_micros(),
            )),
            Err(e) => {
                let err = Tripwire::trip(
                    logger,
                    "TERMINAL_MOUNT",
                    FailpointType::SyntaxAnomaly,
```

```
                    format!("Failed to mount tape '{raw_path}': {e}"),
                );
                return Some(ExecutionResult::failure(
                    format!("[!] {err}"),
                    start.elapsed().as_micros(),
                ));
            }
        }
    }

    // 5. Mount and execute a tape, omni, C, or Lua file
    if trimmed.starts_with("run ") {
        let path = trimmed["run ".len()..].trim();
        let track = if path.contains("kernel") || path.contains("boot") {
            Track::Kernel
        } else {
            Track::App
        };

        // If it's a tape, preserve existing scheduler mount & step logs
        if path.ends_with(".tape") {
            let resolved = resolve_track_path(path);
            let path_str = resolved.to_string_lossy().to_string();
            let mut sched = scheduler.lock().unwrap();
            match sched.mount_tape_file(track, &path_str, logger.clone()) {
                Ok(pid) => {
                    let mut logs = Vec::new();
                    logs.push(format!("[+] Mounted Tape '{path}' onto {} (PID {pid})", track.as_str()));
                    loop {
                        match sched.step_task(pid) {
                            Ok(msg) => {
                                logs.push(msg.clone());
                                if msg.contains("completed") || msg.contains("already executed") || msg.contains("STOP")
{
                                    break;
                                }
                            }
                            Err(e) => {
                                let err = Tripwire::trip(
                                    logger,
                                    "TERMINAL_RUN",
                                    FailpointType::SyntaxAnomaly,
                                    format!("Task {pid} execution halted: {e}"),
                                );
                                logs.push(format!("[!] {err}"));
                                break;
                            }
                        }
                    }
                    return Some(ExecutionResult::success(logs.join("\n"), start.elapsed().as_micros()));
                }
                Err(e) => {
                    let err = Tripwire::trip(
                        logger,
                        "TERMINAL_RUN",
                        FailpointType::SyntaxAnomaly,
                        format!("Failed to run tape '{path}': {e}"),
                    );
                    return Some(ExecutionResult::failure(
                        format!("[!] {err}"),
                        start.elapsed().as_micros(),
                    ));
                }
            }
        } else if path.ends_with(".omni") || path.ends_with(".c") || path.ends_with(".lua") {
            let config = crate::cli::commands::run::RunConfig::new(path);
            let ctx = match config.open(logger) {
                Ok(c) => c,
                Err(e) => {
                    return Some(ExecutionResult::failure(
                        format!("[!] {e}"),
                        start.elapsed().as_micros(),
                    ));
                }
            };
            match crate::cli::commands::run::execute_run_pipeline(&ctx) {
                Ok(res) => {
                    return Some(ExecutionResult::success(res.output, start.elapsed().as_micros()));
```

```
                }
                Err(e) => {
                    return Some(ExecutionResult::failure(
                        format!("[!] {e}"),
                        start.elapsed().as_micros(),
                    ));
                }
            }
        }
    }

    // 6. Step Tape Head on a task
    if trimmed.starts_with("step ") {
        let pid_str = trimmed["step ".len()..].trim();
        if let Ok(pid) = pid_str.parse::<u64>() {
            let mut sched = scheduler.lock().unwrap();
            match sched.step_task(pid) {
                Ok(msg) => return Some(ExecutionResult::success(msg, start.elapsed().as_micros())),
                Err(e) => return Some(ExecutionResult::failure(format!("[!] Error: {e}"), start.elapsed().as_micros())),
            }
        } else {
            return Some(ExecutionResult::failure("Usage: step <pid>".to_string(), start.elapsed().as_micros()));
        }
    }

    // 7. Display rolls on a task
    if trimmed.starts_with("rolls ") {
        let pid_str = trimmed["rolls ".len()..].trim();
        if let Ok(pid) = pid_str.parse::<u64>() {
            let sched = scheduler.lock().unwrap();
            match sched.rolls_for_task(pid) {
                Some(rolls_str) => return Some(ExecutionResult::success(rolls_str, start.elapsed().as_micros())),
                None => return Some(ExecutionResult::failure(format!("[!] Task {pid} not found or has no mounted tape"),
start.elapsed().as_micros())),
            }
        } else {
            return Some(ExecutionResult::failure("Usage: rolls <pid>".to_string(), start.elapsed().as_micros()));
        }
    }

    // 8. General task spawning
    if trimmed.starts_with("spawn ") {
        let cmd = trimmed["spawn ".len()..].trim();
        let mut sched = scheduler.lock().unwrap();
        let pid = sched.spawn("UserTask", cmd, TaskPriority::Normal);
        return Some(ExecutionResult::success(
            format!("[+] Spawned process PID {pid}: '{cmd}' with state +1:PROCEED"),
            start.elapsed().as_micros(),
        ));
    }

    // 9. Task controls
    if trimmed.starts_with("pause ") || trimmed.starts_with("await ") {
        let pid_str = trimmed.split_whitespace().nth(1).unwrap_or("");
        if let Ok(pid) = pid_str.parse::<u64>() {
            let mut sched = scheduler.lock().unwrap();
            match sched.pause_task(pid) {
                Ok(()) => return Some(ExecutionResult::await_state(format!("[~] Task {pid} transitioned to 0:AWAIT"),
start.elapsed().as_micros())),
                Err(e) => return Some(ExecutionResult::failure(format!("[!] Error: {e}"), start.elapsed().as_micros())),
            }
        }
    }

    if trimmed.starts_with("resume ") {
        let pid_str = trimmed.split_whitespace().nth(1).unwrap_or("");
        if let Ok(pid) = pid_str.parse::<u64>() {
            let mut sched = scheduler.lock().unwrap();
            match sched.resume_task(pid) {
                Ok(()) => return Some(ExecutionResult::success(format!("[+] Task {pid} resumed to +1:PROCEED"),
start.elapsed().as_micros())),
                Err(e) => return Some(ExecutionResult::failure(format!("[!] Error: {e}"), start.elapsed().as_micros())),
            }
        }
    }

    if trimmed.starts_with("kill ") || trimmed.starts_with("halt ") {
        let pid_str = trimmed.split_whitespace().nth(1).unwrap_or("");
```

```
        if let Ok(pid) = pid_str.parse::<u64>() {
            let mut sched = scheduler.lock().unwrap();
            match sched.halt_task(pid) {
                Ok(()) => return Some(ExecutionResult::success(format!("[!] Task {pid} halted (NULL:HALT)"),
start.elapsed().as_micros())),
                Err(e) => return Some(ExecutionResult::failure(format!("[!] Error: {e}"), start.elapsed().as_micros())),
            }
        }
    }

    if trimmed == "tick" {
        let mut sched = scheduler.lock().unwrap();
        let count = sched.tick();
        return Some(ExecutionResult::success(format!("[*] Advanced scheduler clock 1 tick. Executed {count} active
tasks."), start.elapsed().as_micros()));
    }

    if trimmed == "vars" || trimmed == "env" {
        let mut out = String::from("ACTIVE SESSION STATE:\n");
        let mut has_entries = false;
        for (k, v) in env.bindings() {
            out.push_str(&format!("  {} = {:?}\n", k, v));
            has_entries = true;
        }
        for (k, v) in variables.iter() {
            out.push_str(&format!("  let {} = {}\n", k, v));
            has_entries = true;
        }
        if !has_entries {
            return Some(ExecutionResult::success("No active session bindings declared.".to_string(),
start.elapsed().as_micros()));
        }
        return Some(ExecutionResult::success(out, start.elapsed().as_micros()));
    }

    // 10. AI Model Inquiry & Cognitive Dialogue
    if trimmed.starts_with("ai ") {
        let raw_args = trimmed["ai ".len()..].trim();
        let (prompt_str, verbose) = if raw_args.starts_with("-v ") {
            (raw_args["-v ".len()..].trim(), true)
        } else if raw_args.starts_with("--verbose ") {
            (raw_args["--verbose ".len()..].trim(), true)
        } else {
            (raw_args, false)
        };

        let prompt = if (prompt_str.starts_with('"')
            && prompt_str.ends_with('"')
            && prompt_str.len() >= 2)
            || (prompt_str.starts_with('\'')
                && prompt_str.ends_with('\'')
                && prompt_str.len() >= 2)
        {
            &prompt_str[1..prompt_str.len() - 1]
        } else {
            prompt_str
        };

        let mut cortex = crate::ai::AICortex::new();
        match cortex.process_prompt(prompt) {
            Ok(res) => {
                let out = if verbose {
                    let mut s = format!(
                        "[CPI-SI Cognitive Thought Trace]\n\
                         • State: +1:{}\n\
                         • WHO:   {}\n\
                         • WHAT:  {}\n\
                         • WHY:   {}\n\
                         • HOW:   {}\n\n\
                         [Synthesized Reel]\n{}\n\n\
                         [Machine Layer Response]\n\"{}\"\n\n\
                         [Session: {} | Scroll: {:?}]",
                        res.execution_state.macro_op(),
                        res.thought.who,
                        res.thought.what,
                        res.thought.why,
                        res.thought.how,
                        res.synthesized_omnicode.trim(),
```

```
                        res.natural_response,
                        res.session_id,
                        res.scroll_path
                    );
                    if !res.repairs_applied.is_empty() {
                        s.push_str(&format!(
                            "\n[Restorator Repairs: {}]",
                            res.repairs_applied.len()
                        ));
                    }
                    s
                } else {
                    format!(
                        "[AI Model Response: CPI-SI Base]\n\"{}\"\n\n\
                        [Session: {}]",
                        res.natural_response, res.session_id
                    )
                };
                return Some(ExecutionResult::success(
                    out,
                    start.elapsed().as_micros(),
                ));
            }
            Err(e) => {
                return Some(ExecutionResult::failure(
                    format!("[!] AI Cortex Error: {e}"),
                    start.elapsed().as_micros(),
                ));
            }
        }
    }

    // 11. O(1) Episodic Session Query
    if trimmed.starts_with("query ") {
        let sid = trimmed["query ".len()..].trim();
        let cortex = crate::ai::AICortex::new();
        if let Some(s) = cortex.query_episode(sid) {
            let state_str = match s.final_state.as_int() {
                Some(i) => format!("{i:+}:{}", s.final_state.macro_op()),
                _ => format!("*:{}", s.final_state.macro_op()),
            };
            let paired_str = s
                .log_path
                .as_deref()
                .map(|p| p.to_string_lossy().into_owned())
                .unwrap_or_else(|| "(none)".to_string());
            let out = format!(
                "Session: {}\n  • Category:   {}\n  • State:      {}\n  • Prompt:     \"{}\"\n  • Failpoints: {}\n  • Repairs:    {}\n  • Scroll:     {:?}\n  • Telemetry:  {}",
                s.session_id,
                s.category,
                state_str,
                s.prompt,
                s.failpoint_count,
                s.repair_count,
                s.scroll_path,
                paired_str,
            );
            return Some(ExecutionResult::success(
                out,
                start.elapsed().as_micros(),
            ));
        } else {
            return Some(ExecutionResult::failure(
                format!("[!] Session '{sid}' not found in episodic storehouse"),
                start.elapsed().as_micros(),
            ));
        }
    }

    // 12. Failpoint & Anomaly Diagnostic Report
    if trimmed.starts_with("diagnose ") {
        let sid = trimmed["diagnose ".len()..].trim();
        let cortex = crate::ai::AICortex::new();
        if let Some(diag) = cortex.diagnose_failpoint(sid) {
            return Some(ExecutionResult::success(
                diag,
                start.elapsed().as_micros(),
```

```
            ));
        } else {
            return Some(ExecutionResult::failure(
                format!("[!] Session '{sid}' not found in episodic storehouse"),
                start.elapsed().as_micros(),
            ));
        }
    }

    // 13. Episodic Memory Listing
    if trimmed == "episodes" || trimmed.starts_with("episodes ")
        || trimmed == "scrolls" || trimmed.starts_with("scrolls ")
    {
        let count = if trimmed.starts_with("episodes ") {
            trimmed["episodes ".len()..]
                .trim()
                .parse::<usize>()
                .unwrap_or(10)
        } else if trimmed.starts_with("scrolls ") {
            trimmed["scrolls ".len()..]
                .trim()
                .parse::<usize>()
                .unwrap_or(10)
        } else {
            10
        };

        let cortex = crate::ai::AICortex::new();
        let recents = cortex.recent_episodes(count);
        if recents.is_empty() {
            return Some(ExecutionResult::success(
                "No episodes found in storehouse.".to_string(),
                start.elapsed().as_micros(),
            ));
        }

        let mut out = format!("RECENT EPISODES (Last {}):\n", recents.len());
        for (i, s) in recents.iter().enumerate() {
            let state_str = match s.final_state.as_int() {
                Some(val) => format!("{val:+}:{}", s.final_state.macro_op()),
                _ => format!("*:{}", s.final_state.macro_op()),
            };
            out.push_str(&format!(
                "  [{:>2}] {} | Cat: {:<8} | State: {:<11} | FP: {:>2} | Rep: {:>2}\n       Prompt: \"{}\"\n",
                i + 1,
                s.session_id,
                s.category,
                state_str,
                s.failpoint_count,
                s.repair_count,
                s.prompt,
            ));
        }
        return Some(ExecutionResult::success(
            out,
            start.elapsed().as_micros(),
        ));
    }

    // 14. Storehouse Vessel Capacity Rotation
    if trimmed == "rotate" || trimmed.starts_with("rotate ") {
        let cap = if trimmed.starts_with("rotate ") {
            trimmed["rotate ".len()..]
                .trim()
                .parse::<usize>()
                .unwrap_or(1000)
        } else {
            1000
        };
        let policy = crate::diagnostics::logger::RotationPolicy::new(
            cap, true,
        );
        let scroll_dir = crate::core::filetype::resolve_scroll_storage_dir();
        let s_logger = crate::ai::ScrollLogger::with_rotation_policy(
            &scroll_dir,
            policy,
        );
        match s_logger.rotate_storehouse() {
```

```
            Ok(n) => return Some(ExecutionResult::success(
                format!(
                    "[√] Rotated storehouse (cap={cap}): {n} excess archived"
                ),
                start.elapsed().as_micros(),
            )),
            Err(e) => return Some(ExecutionResult::failure(
                format!("[!] Rotation failed: {e}"),
                start.elapsed().as_micros(),
            )),
        }
    }

    // 14b. CPI-SI Discourse Prompt & Interaction
    if trimmed == "cpisi" || trimmed.starts_with("cpisi ")
        || trimmed == "interact" || trimmed.starts_with("interact ")
    {
        let prompt = if trimmed.contains(' ') {
            trimmed.split_once(' ').map(|x| x.1).unwrap_or("hello")
        } else {
            "hello"
        };
        let mut cortex = crate::ai::AICortex::new();
        match cortex.process_prompt(prompt) {
            Ok(res) => {
                let (exc, neu, inh) = cortex.brain.synaptic_counts();
                let out = format!(
                    "[√] CPI-SI Base Deliberation (+1:{}):\n  \
                     • Response: \"{}\"\n  \
                     • Synapse State: +1: {}, 0: {}, -1: {}",
                    res.execution_state.macro_op(),
                    res.natural_response,
                    exc, neu, inh
                );
                return Some(ExecutionResult::success(
                    out,
                    start.elapsed().as_micros(),
                ));
            }
            Err(e) => {
                return Some(ExecutionResult::failure(
                    format!("[!] CPI-SI error: {e}"),
                    start.elapsed().as_micros(),
                ));
            }
        }
    }

    // 15. Supervised Episodic Neural Training
    if trimmed == "train" || trimmed.starts_with("train ") {
        let scroll_dir = crate::core::filetype::resolve_scroll_storage_dir();
        let learner = crate::ai::AILearner::new(&scroll_dir);
        match learner.digest_experiences() {
            Ok(summary) => {
                let mut brain = crate::ai::NeuralBrain::new(
                    &[8, 16, 4],
                    1,
                    logger.clone(),
                );
                let trainer = crate::ai::NeuralTrainer::new();
                let report = trainer.train(&mut brain, &summary.experiences);
                let weights_path =
                    std::path::PathBuf::from(
                        "storage/weights/cpisi_base.weights"
                    );
                brain.save_weights(&weights_path).ok();
                let out = format!(
                    "[√] Neural Training Complete:\n  \
                     • Episodes Digested: {}\n  \
                     • Synapses Adapted:  {}\n  \
                     • Resilience Score:  {:.1}% -> {:.1}%\n  \
                     • Weights Saved:     {:?}",
                    report.total_episodes,
                    report.synapses_adapted,
                    report.noise_resilience_before * 100.0,
                    report.noise_resilience_after * 100.0,
                    weights_path
                );
```

```
                return Some(ExecutionResult::success(
                    out,
                    start.elapsed().as_micros(),
                ));
            }
            Err(e) => {
                return Some(ExecutionResult::failure(
                    format!("[!] Training failed: {e}"),
                    start.elapsed().as_micros(),
                ));
            }
        }
    }

    // 16. Structured Systemic Noise & Degradation Simulation
    if trimmed == "noise" || trimmed.starts_with("noise ") {
        let engine = crate::ai::NoiseEngine::default();
        let ideal = &[
            crate::core::ternary::QuadState::Proceed,
            crate::core::ternary::QuadState::Proceed,
            crate::core::ternary::QuadState::Proceed,
            crate::core::ternary::QuadState::Proceed,
        ];
        let brain = crate::ai::NeuralBrain::new(
            &[4, 8, 2],
            1,
            logger.clone(),
        );
        let ladder = engine.simulate_breakdown_ladder(ideal);
        let mut out = String::from(
            "DEGRADATION LADDER (Ideal -> Breakdown):\n"
        );
        for step in &ladder {
            let noisy_out = brain.think(&step.perturbed_vector);
            let mode_str = match step.dominant_mode {
                crate::ai::DegradationMode::Ideal => "Pristine",
                crate::ai::DegradationMode::TritDecay => "Trit Decay",
                crate::ai::DegradationMode::BoundaryLeak => "Boundary Leak",
                crate::ai::DegradationMode::ThermalInversion => "Inversion",
                crate::ai::DegradationMode::FailpointTrap => "Halt Trap",
            };
            out.push_str(&format!(
                "  [{:>3.0}%] {:<13} | Fidelity: {:>5.1}% | Out: {:?}\n",
                step.stress_level * 100.0,
                mode_str,
                step.signal_fidelity * 100.0,
                noisy_out
            ));
        }
        let resilience = brain.evaluate_noise_resilience(ideal, &engine);
        out.push_str(&format!(
            "Neural Resilience Score: {:.1}%",
            resilience * 100.0
        ));
        return Some(ExecutionResult::success(
            out,
            start.elapsed().as_micros(),
        ));
    }

    if trimmed == "help" {
        let help = "OMNICODE NATIVE OPERATING SYSTEM SHELL:
ps, tasks          List scheduled tasks and active processes
tracks             Display the 3 execution channels (Kernel, Terminal, Apps)
mem, ram           Display ANSI storehouse RAM meter and allocations
mount <path.tape>  Mount a tape ribbon onto an execution track
run <path.tape>    Mount and execute a process tape to completion
step <pid>         Manually step a Tape Head forward by 1 roll
rolls <pid>        View tape rolls and the live ==> [HEAD] pointer
spawn <cmd>        Spawn a new task under the task scheduler
pause <pid>        Transition a process to 0:AWAIT (standby)
resume <pid>       Resume a process to +1:PROCEED (running)
kill <pid>         Halt and terminate a process (NULL:HALT)
tick               Advance the cooperative scheduler clock
vars, env          Display in-memory declared variables and Vessels
ai <prompt>        Deliberate with the AI Model and synthesize reels
interact, cpisi    Engage in live discourse with CPI-SI Base model
query <session>    Query an episodic memory scroll in O(1) time
```

```
  diagnose <session> Inspect trapped failpoints and immune repairs
  episodes [N]       Display the N most recent execution scrolls
  rotate [capacity]  Enforce vessel capacity bounds (Occupancy <= Capacity)
  train              Train balanced-ternary weights on episodic scrolls
  noise              Run non-random structured degradation ladder
  clear              Clear the terminal screen
  help               Display this help reference

LIVE OMNICODE STATEMENTS & ROLLS:
  Type any valid roll (e.g. `A [ 10 ]`, `42 --> Buf @ 0`, `Storehouse v-- App [ 256 ]`)
  to execute directly in persistent silicon memory!
";
        return Some(ExecutionResult::success(help.to_string(), start.elapsed().as_micros()));
    }

    // Variable assignment: let x = 42
    if trimmed.starts_with("let ") && trimmed.contains('=') {
        let parts: Vec<&str> = trimmed["let ".len()..].splitn(2, '=').collect();
        if parts.len() == 2 {
            let name = parts[0].trim().to_string();
            let val = parts[1].trim().to_string();
            variables.insert(name.clone(), val.clone());
            return Some(ExecutionResult::success(format!("Variable '{name}' bound to: {val}"),
start.elapsed().as_micros()));
        }
    }

    None
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Verification & Invariant Proofs )
// ============================================================================
// Built-in command dispatcher is decoupled from session orchestration.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/session/mod.rs RS | 37 LINES | 185 WORDS | 1.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Terminal Interactive Session Subsystem Identity & Public Exports )
// ============================================================================
//! OmniCode Terminal: Interactive Session Subsystem Module
//!
//! Provides the interactive userspace shell, persistent variable state,
//! direct roll evaluation, and native character stream echo protocols.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**: Tables of testimony, written with the finger of God.
//! - **Proverbs 24:27 (KJV)**: Prepare thy work without, and make it fit for thyself in the field.
//! - **Proverbs 22:28 (KJV)**: Remove not the ancient landmark, which thy fathers have set.
// ============================================================================

pub mod commands;
pub mod session;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Re-export Declarations for Terminal Session & Execution Models )
// ============================================================================
pub use commands::ExecutionResult;
pub use session::TerminalSession;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Subsystem Invariants & Session Proofs )
// ============================================================================
// Terminal sessions evaluate rolls and process character echo natively.

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Verification & Module Closure )
// ============================================================================
// Interactive Terminal Session subsystem verified and closed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/session/session.rs RS 481 LINES 1647 WORDS 19.5 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Terminal Interactive Session State, Direct Roll Evaluator & Native Echo Engine )
// ================================================================================
//! OmniCode Terminal: Interactive Session Coordinator & Character I/O Engine
//!
//! # Architectural Scope & Purpose:
//! Coordinates the interactive userspace shell for the OmniCode platform:
//! 1. Maintains command history, variable declarations, and persistent [`Environment`] state.
//! 2. Routes shell commands through the Multi-Track [`OmniScheduler`].
//! 3. Directly evaluates live procedural rolls in memory without sub-process overhead.
//! 4. Houses native Character I/O streams and the 5-trit Radix-3 [`CharacterEchoEngine`].
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of atomic runes onto bounded physical tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types and substrates before constructing systems).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical memory boundaries, silicon addresses, and constants).
// ================================================================================

use std::collections::HashMap;
use std::sync::{Arc, Mutex};
use std::time::Instant;

use crate::cogs::tripwire::Tripwire;
use crate::core::ternary::QuadState;
use crate::diagnostics::debugger::OmniDebugger;
use crate::diagnostics::logger::{DiagnosticLogger, FailpointType};
use crate::parser::lexer::Lexer;
use crate::parser::parser::Parser;
use crate::runtime::environment::{Environment, RuntimeValue};
use crate::runtime::evaluator::Evaluator;
use crate::terminal::scheduler::{OmniScheduler, SharedScheduler};
use crate::terminal::universal::call_response_engine::{
    Call, Response, CharacterResponse, NumberResponse, CallResponseEngine, EchoEngine,
};
use crate::terminal::universal::ports::{TerminalInputStream, TerminalOutputStream};

use super::commands::{dispatch_builtin_command, ExecutionResult};

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( TerminalSession Struct Definition & Initializers )
// ================================================================================

/// Interactive shell session maintaining persistent memory bindings,
/// scheduler access, directional I/O streams, and native character echo.
pub struct TerminalSession {
    /// Command line history.
    pub history: Vec<String>,
    /// Script-level variable bindings.
    pub variables: HashMap<String, String>,
    /// Thread-safe handle to the cooperative Multi-Track scheduler.
    pub scheduler: SharedScheduler,
    /// Persistent virtual machine runtime environment.
    pub env: Environment,
    /// Attached telemetry and error interception logger.
    pub logger: DiagnosticLogger,
    /// Delimiter boundary stack tracking debugger.
    pub debugger: OmniDebugger,
    /// Directional input stream from hardware `FROM` port (0x03F8).
    pub input_stream: TerminalInputStream,
    /// Directional output stream to hardware `INTO` port (0x000B8000).
    pub output_stream: TerminalOutputStream,
    /// Universal Call-and-Response Echo Engine.
    pub echo_engine: EchoEngine,
}
```

```rust

impl Default for TerminalSession {
    fn default() -> Self {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 10);
        let env = Environment::new();
        Self {
            history: Vec::new(),
            variables: HashMap::new(),
            scheduler: Arc::new(Mutex::new(OmniScheduler::new())),
            env,
            logger,
            debugger,
            input_stream: TerminalInputStream::default(),
            output_stream: TerminalOutputStream::default(),
            echo_engine: EchoEngine,
        }
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Execution Pipeline, Direct Roll Lowering & Native Character I/O )
// ============================================================================

impl TerminalSession {
    /// Constructs a new terminal session bound to a shared multi-track scheduler.
    pub fn new(scheduler: SharedScheduler) -> Self {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 10);
        let env = Environment::new();
        Self {
            history: Vec::new(),
            variables: HashMap::new(),
            scheduler,
            env,
            logger,
            debugger,
            input_stream: TerminalInputStream::default(),
            output_stream: TerminalOutputStream::default(),
            echo_engine: EchoEngine,
        }
    }

    /// Dispatches a command line and returns an [`ExecutionResult`].
    pub fn execute(&mut self, line: &str) -> ExecutionResult {
        let trimmed = line.trim();
        if trimmed.is_empty() {
            return ExecutionResult::success(String::new(), 0);
        }

        self.history.push(trimmed.to_string());
        let start = Instant::now();

        // 1. Dispatch Built-in Shell Commands
        if let Some(res) = dispatch_builtin_command(
            trimmed,
            &self.scheduler,
            &self.env,
            &mut self.variables,
            &self.logger,
            &self.debugger,
            start,
        ) {
            for c in res.stdout.chars() {
                self.output_stream.push_char(c);
            }
            return res;
        }

        // 2. Direct In-Memory Roll Evaluation (OmniCode Native Execution)
        if trimmed.contains("-->") || trimmed.contains("<--") || trimmed.contains("v--") || trimmed.contains("--^")
            || trimmed.starts_with("Storehouse") || (trimmed.contains('[') && trimmed.contains(']'))
            || (trimmed.contains('<') && trimmed.contains('>'))
            || trimmed.starts_with("status")
        {
            let mut lexer = Lexer::new(trimmed, self.logger.clone());
            if let Ok(tokens) = lexer.tokenize() {
```

```
                let mut parser = Parser::new(tokens, self.debugger.clone());
                if let Ok(prog) = parser.parse_program() {
                    let mut evaluator = Evaluator::new(self.env.clone(), self.debugger.clone());
                    match evaluator.eval(&crate::ast::nodes::ASTNode::Program(prog)) {
                        Ok(val) => {
                            self.env = evaluator.env; // Persist environment state!
                            return ExecutionResult::success(
                                format!("=> {:?} (state: +1:PROCEED)", val),
                                start.elapsed().as_micros(),
                            );
                        }
                        Err(e) => {
                            let err = Tripwire::trip(
                                &self.logger,
                                "TERMINAL_EVAL",
                                FailpointType::SyntaxAnomaly,
                                format!("Evaluation Failpoint: {e}"),
                            );
                            return ExecutionResult::failure(
                                format!("[!] {err}"),
                                start.elapsed().as_micros(),
                            );
                        }
                    }
                }
            }
        }

        // 3. Fallback: Host Subcommand Execution
        let clean_cmd = if trimmed.starts_with("omnicode ") {
            trimmed.to_string()
        } else {
            format!("omnicode {trimmed}")
        };

        let exe = std::env::current_exe().unwrap_or_else(|_| std::path::PathBuf::from("omnicode"));
        let args: Vec<&str> = clean_cmd["omnicode ".len()..].split_whitespace().collect();
        let output = std::process::Command::new(&exe)
            .args(&args)
            .output();

        match output {
            Ok(out) => {
                let stdout = String::from_utf8_lossy(&out.stdout).to_string();
                let stderr = String::from_utf8_lossy(&out.stderr).to_string();
                let combined = if stderr.is_empty() {
                    stdout
                } else if stdout.is_empty() {
                    stderr
                } else {
                    format!("{stdout}\n{stderr}")
                };
                if out.status.success() {
                    ExecutionResult::success(combined, start.elapsed().as_micros())
                } else {
                    ExecutionResult::failure(combined, start.elapsed().as_micros())
                }
            }
            Err(e) => ExecutionResult::failure(
                format!("Failed to execute CLI subcommand: {e}"),
                start.elapsed().as_micros(),
            ),
        }
    }

    // ------------------------------------------------------------------------
    // Native Character I/O & Echo Stream Methods
    // ------------------------------------------------------------------------

    /// Feeds an input character into the terminal's `FROM` port stream (0x03F8)
    /// and synchronizes it into the virtual machine environment.
    pub fn feed_input_char(&mut self, c: char) {
        self.input_stream.push_char(c);
        self.env.write_from(RuntimeValue::from_char(c));
    }

    /// Reads the next available character from the terminal's `INTO` port stream (0x000B8000).
    pub fn read_output_char(&mut self) -> Option<char> {
```

```
        self.output_stream.pop_char()
    }

    /// Flushes all pending output characters into an allocated string.
    pub fn flush_output(&mut self) -> String {
        self.output_stream.flush_to_string()
    }

    /// Processes a single character glyph through the full 5-step Character tape lifecycle,
    /// verifying Radix-3 transduction, single-cell capacity bounds, and returning the native echo.
    pub fn process_character_echo(&mut self, glyph: char) -> Result<CharacterResponse, String> {
        // Feed into FROM port
        self.feed_input_char(glyph);

        // Run full Inscription -> Transduction -> Bound Guard -> Target Projection
        let res = EchoEngine::echo_char(glyph)?;

        // Write projected character into INTO port and output stream
        self.output_stream.push_char(res.glyph);
        self.env.write_into(RuntimeValue::from_char(res.glyph));

        // Inscribe RollFrame telemetry into in-memory flight recorder
        let frame = crate::diagnostics::logger::RollFrame::new(
            res.glyph as u128,
            1,
            5,
            "TRANSDUCE",
            (0, res.glyph as u64),
            (res.glyph as u64, 0),
            res.state,
            0,
            Some(format!("Glyph '{}'", res.glyph)),
        );
        self.logger.record_roll_frame(frame);

        Ok(res)
    }

    /// Processes an entire character stream through the native Character Echo protocol,
    /// returning the verified echoed string.
    pub fn process_character_stream_echo(&mut self, stream: &str) -> Result<String, String> {
        let mut echoed = String::with_capacity(stream.len());
        for c in stream.chars() {
            let res = self.process_character_echo(c)?;
            echoed.push(res.glyph);
        }
        Ok(echoed)
    }

    /// Processes any Call payload through the universal Call-Response model.
    pub fn process_call(&mut self, call: Call) -> Result<Response, String> {
        CallResponseEngine::execute(call)
    }

    /// Processes a scalar Number through the Call-Response model.
    pub fn process_number_call(&mut self, scalar: i64) -> Result<NumberResponse, String> {
        let res = CallResponseEngine::call_number(scalar)?;
        self.env.write_into(RuntimeValue::from_number(scalar as f64));

        let frame = crate::diagnostics::logger::RollFrame::new(
            scalar.unsigned_abs() as u128,
            1,
            3,
            "TRANSDUCE",
            (0, scalar.unsigned_abs()),
            (res.packed_dual_rail, scalar.unsigned_abs()),
            QuadState::Proceed,
            0,
            Some(format!("Number {scalar}")),
        );
        self.logger.record_roll_frame(frame);

        Ok(res)
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
```

```
// ( Unit Test Verification Proofs & Invariants )
// ==============================================================================

#[cfg(test)]
mod tests {
    use super::*;
    use crate::core::ternary::QuadState;

    #[test]
    fn test_terminal_session_builtin_commands() {
        let mut session = TerminalSession::default();
        let res_help = session.execute("help");
        assert_eq!(res_help.state, QuadState::Proceed);
        assert!(res_help.stdout.contains("OMNICODE NATIVE OPERATING SYSTEM SHELL"));

        let res_tasks = session.execute("tasks");
        assert_eq!(res_tasks.state, QuadState::Proceed);
        assert!(res_tasks.stdout.contains("kernel --daemon"));

        let res_tracks = session.execute("tracks");
        assert_eq!(res_tracks.state, QuadState::Proceed);
        assert!(res_tracks.stdout.contains("CoreDaemon"));
    }

    #[test]
    fn test_terminal_session_echo_uncommon_letters() {
        let mut session = TerminalSession::default();

        // 1. Echo uncommon uppercase letter 'Q' (ASCII 81)
        let res_q = session.execute("echo Q");
        assert_eq!(res_q.state, QuadState::Proceed);
        assert_eq!(res_q.stdout, "Q");
        assert_eq!(session.read_output_char(), Some('Q'));

        // 2. Echo uncommon letter 'Z' (ASCII 90)
        let res_z = session.execute("echo 'Z'");
        assert_eq!(res_z.state, QuadState::Proceed);
        assert_eq!(res_z.stdout, "Z");
        assert_eq!(session.read_output_char(), Some('Z'));

        // 3. Echo symbol '~' (ASCII 126)
        let res_tilde = session.execute("echo ~");
        assert_eq!(res_tilde.state, QuadState::Proceed);
        assert_eq!(res_tilde.stdout, "~");
        assert_eq!(session.read_output_char(), Some('~'));

        // 4. Echo string with mixed uncommon characters "QxZ~"
        let res_multi = session.execute("echo QxZ~");
        assert_eq!(res_multi.state, QuadState::Proceed);
        assert_eq!(res_multi.stdout, "QxZ~");
        let drained: String = std::iter::from_fn(|| session.read_output_char()).collect();
        assert_eq!(drained, "QxZ~");
    }

    #[test]
    fn test_terminal_session_run_character_tape_and_omni() {
        let mut session = TerminalSession::default();

        // 1. Run character process tape
        let res_tape = session.execute("run lexicon/01_nouns/ideas/types/character.tape");
        assert_eq!(res_tape.state, QuadState::Proceed);
        assert!(res_tape.stdout.contains("Mounted Tape"));
        assert!(res_tape.stdout.contains("Roll 1/6"));

        // 2. Run character 4-Block source reel
        let res_omni = session.execute("run lexicon/01_nouns/ideas/types/character.omni");
        assert_eq!(res_omni.state, QuadState::Proceed);
        assert!(res_omni.stdout.contains("OmniCode Source Reel Executed Successfully"));
    }

    #[test]
    fn test_terminal_session_direct_roll_evaluation() {
        let mut session = TerminalSession::default();
        let res = session.execute("roll_1 [ A = A ]");
        assert_eq!(res.state, QuadState::Proceed);
        assert!(res.stdout.contains("+1:PROCEED"));
    }

```

```
    #[test]
    fn test_terminal_session_native_character_echo() {
        let mut session = TerminalSession::default();
        let res = session.process_character_echo('K').expect("Native echo for 'K' must succeed");
        assert_eq!(res.glyph, 'K');
        assert_eq!(res.state, QuadState::Proceed);
        assert_eq!(res.cell_footprint, 1);
        assert_eq!(session.read_output_char(), Some('K'));
    }

    #[test]
    fn test_terminal_session_stream_echo() {
        let mut session = TerminalSession::default();
        let text = "OmniCode Native Terminal 2026!";
        let echoed = session.process_character_stream_echo(text).expect("Stream echo must succeed");
        assert_eq!(echoed, text);
    }

    #[test]
    fn test_terminal_exhaustive_character_echo_no_valid_character_omitted() {
        let mut session = TerminalSession::default();

        // 1. Test every standard ASCII character (0 through 127)
        // Including control characters (\0, \t, \n, \r), printable symbols, letters, and numbers
        let mut omitted_count = 0;
        let mut tested_count = 0;

        for byte in 0u8..=127u8 {
            let ch = byte as char;
            tested_count += 1;

            match session.process_character_echo(ch) {
                Ok(res) => {
                    assert_eq!(res.glyph, ch, "Identity violation for character code {byte}");
                    assert_eq!(res.state, QuadState::Proceed, "State violation for character code {byte}");
                    assert_eq!(res.cell_footprint, 1, "Cell footprint must be 1 for character code {byte}");

                    // Read from INTO port output stream
                    let read_back = session.read_output_char();
                    assert_eq!(read_back, Some(ch), "Output port character mismatch for {byte}");
                }
                Err(err) => {
                    eprintln!("Omission error for character {byte}: {err}");
                    omitted_count += 1;
                }
            }
        }

        assert_eq!(omitted_count, 0, "No valid ASCII character should be omitted");
        assert_eq!(tested_count, 128, "All 128 standard ASCII characters must be tested");

        // 2. Test extended single-cell character capacity up to 242 (3^5 = 243 discrete values)
        for byte in 128u8..=242u8 {
            let ch = byte as char;
            let res = session.process_character_echo(ch)
                .unwrap_or_else(|e| panic!("Failed echo for extended character {byte}: {e}"));
            assert_eq!(res.glyph, ch);
            assert_eq!(session.read_output_char(), Some(ch));
        }

        // 3. Test comprehensive printable ASCII stream
        let all_printable: String = (32u8..=126u8).map(|b| b as char).collect();
        let stream_echoed = session.process_character_stream_echo(&all_printable)
            .expect("Stream echo for all printable characters must succeed");
        assert_eq!(stream_echoed, all_printable, "Every single printable character must match identically without
omission");
    }

    #[test]
    fn test_terminal_session_call_response_number() {
        let mut session = TerminalSession::default();

        // 1. Process number call for unities and partition
        let r0 = session.process_number_call(0).expect("Call 0");
        assert_eq!(r0.scalar, 0);
        assert!(r0.equilibrium_proved);

        let r1 = session.process_number_call(1).expect("Call 1");
```

```
        assert_eq!(r1.scalar, 1);

        let r_neg1 = session.process_number_call(-1).expect("Call -1");
        assert_eq!(r_neg1.scalar, -1);

        let r4096 = session.process_number_call(4096).expect("Call 4096");
        assert_eq!(r4096.scalar, 4096);
        assert_eq!(r4096.state, QuadState::Proceed);

        // 2. Process universal call
        let u_call = session.process_call(Call::Number(65536)).expect("Universal call");
        match u_call {
            Response::Number(n) => assert_eq!(n.scalar, 65536),
            _ => panic!("Expected Number response"),
        }
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/studio/hud.rs RS 173 LINES 740 WORDS 8.4 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( OmniStudio: Cinematic Projector, Set Props & Spindle HUD )
// ============================================================================
//! OmniCode Terminal: Cinematic Projector & Memory HUD
//!
//! # Architectural Scope & Purpose:
//! Provides a real-time visual projection HUD of the 4-directional Memory Reel,
//! showing active tracks, scenes, rolls, and set props (64-bit / 32-trit cells):
//! 1. **Projector Head:** Active Reel, Channel Track, Transport Status (`PLAY`, `PAUSE`, `REWIND`, `STOP`).
//! 2. **Spindle Depth Meter:** Current Scene nesting level (`v--` depth).
//! 3. **The Script (Active Roll):** Current instruction line rolling through the gate.
//! 4. **Set Props (Memory Grid):** Bounded cells holding physical 64-bit words.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of physical data into visual display volumes).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic projector frames before launching graphical studios).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of physical memory cell addresses and clapper cue landmarks).
// ============================================================================

use crate::core::mmio::VgaBuffer;
use crate::core::reel::MemoryReel;
use crate::core::tape::Tape;
use crate::core::ternary::TransportAction;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( OmniStudio Struct Definition and Configuration )
// ============================================================================

/// Cinematic Projector rendering interactive terminal HUD frames of memory reel execution.
pub struct OmniStudio {
    /// Active 4-directional memory reel.
    pub reel: MemoryReel,
    /// Canonical execution track display label.
    pub active_track: &'static str,
    /// Hardware MMIO VGA text buffer emulator.
    pub vga: VgaBuffer,
}

impl OmniStudio {
    /// Creates a new Studio Projector initialized with a named memory reel.
    pub fn new(reel_name: impl Into<String>, capacity: usize) -> Self {
        Self {
            reel: MemoryReel::new(reel_name, capacity),
            active_track: "TRACK 2 [APP]",
            vga: VgaBuffer::new(),
        }
    }

    /// Sets the active channel track display label.
    pub fn set_track(&mut self, track: &'static str) {
        self.active_track = track;
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Projector Frame Rendering & HUD Formatting )
// ============================================================================

impl OmniStudio {
    /// Renders a complete cinematic projector HUD frame for a tape and its active roll.
    pub fn render_frame(&self, tape: &Tape, roll_idx: usize, message: &str) -> String {
        let mut out = String::new();
        let transport_badge = match self.reel.transport {
```

```
            TransportAction::Play => "\x1b[1;32m[▶ PLAY]\x1b[0m",
            TransportAction::Pause => "\x1b[1;33m[▮▮ PAUSE]\x1b[0m",
            TransportAction::Rewind => "\x1b[1;35m[⏮ REWIND]\x1b[0m",
            TransportAction::Stop => "\x1b[1;31m[■ STOP]\x1b[0m",
        };

        let scene = self.reel.active_scene();
        let landmark_str = scene.landmark.as_deref().unwrap_or("none");


out.push_str("\x1b[1;36m╔══════════════════════════════════════════════════════════════════════════════╗\x1b[0m\n");
        out.push_str(&format!(
            "\x1b[1;36m║\x1b[0m  \x1b[1mOMNICODE CINEMA STUDIO\x1b[0m | REEL: \x1b[1;33m[{}]\x1b[0m | {}
\x1b[1;36m║\x1b[0m\n",
            self.reel.name, transport_badge
        ));
        out.push_str(&format!(
            "\x1b[1;36m║\x1b[0m  CHANNEL: {:<18} | SPINDLE DEPTH: {} | SCENE: <{}>  \x1b[1;36m║\x1b[0m\n",
            self.active_track, scene.depth, scene.name
        ));
        out.push_str(&format!(
            "\x1b[1;36m║\x1b[0m  CLAPPER CUE: <{:<16}> | HEAD POSITION: @0x{:04X}               \x1b[1;36m║\x1b[0m\n",
            landmark_str, scene.head_pos
        ));

out.push_str("\x1b[1;36m╠══════════════════════════════════════════════════════════════════════════════╣\x1b[0m\n");

        // Script Gate: Active Roll
        let roll_count = tape.roll_count();
        let roll_text = tape.rolls.get(roll_idx).map(|r| r.text.as_str()).unwrap_or("EOF");
        out.push_str(&format!(
            "\x1b[1;36m║\x1b[0m  \x1b[1;32mSCRIPT GATE [Roll {}/{}]:\x1b[0m {:<48} \x1b[1;36m║\x1b[0m\n",
            roll_idx + 1, roll_count, roll_text
        ));
        out.push_str(&format!(
            "\x1b[1;36m║\x1b[0m  {:<76} \x1b[1;36m║\x1b[0m\n",
            message
        ));

out.push_str("\x1b[1;36m╠══════════════════════════════════════════════════════════════════════════════╣\x1b[0m\n");

        // Set Props (Active Memory Cells)
        out.push_str("\x1b[1;36m║\x1b[0m  \x1b[1;33mSET PROPS (MEMORY CELL GRID):\x1b[0m
\x1b[1;36m║\x1b[0m\n");
        let display_cells = 4.min(scene.capacity);
        for i in 0..display_cells {
            if let Some(word) = scene.read(i) {
                let focus_ptr = if i == scene.head_pos { "▶" } else { " " };
                out.push_str(&format!(
                    "\x1b[1;36m║\x1b[0m   {} [0x{:04X}]: 0x{:016X} | Val: {:<5} | State: {:<7} \x1b[1;36m║\x1b[0m\n",
                    focus_ptr, i, word.raw, word.payload().to_i64(), word.state().macro_op()
                ));
            }
        }


out.push_str("\x1b[1;36m╚══════════════════════════════════════════════════════════════════════════════╝\x1b[0m\n");
        if self.vga.cursor_col > 0 || self.vga.cursor_row > 0 {
            out.push_str(&self.vga.render_screen(4));
        }
        out
    }

    /// Projects an entire tape sequentially with terminal cinematic frames.
    pub fn project_tape(&mut self, tape: &Tape) {
        println!("\x1b[1;32m▶ Mounting & Projecting Reel [{}] on {}\x1b[0m\n", self.reel.name, self.active_track);
        for (idx, roll) in tape.rolls.iter().enumerate() {
            let msg = self.step_roll(&roll.text);
            let frame = self.render_frame(tape, idx, &msg);
            print!("{frame}");
        }
        println!("\x1b[1;32m✓ Projection Complete: Tape sealed in {}\x1b[0m\n", tape.execution_state.macro_op());
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests )
```

```
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;
    use crate::diagnostics::logger::DiagnosticLogger;

    #[test]
    fn test_studio_render_frame() {
        let logger = DiagnosticLogger::new(false);
        let tape = Tape::from_source(
            r#"
            [Tape.TestProjection] : WHO {Kernel} WHAT {Test} --> DRIVE {Cinema}
            roll_1 [ 42 --> Cell_0 @ 0 ]
            roll_2 [ commit_status [ STOP ] ]
            "#,
            logger,
        ).expect("Parse tape");

        let studio = OmniStudio::new("TestReel", 16);
        let frame = studio.render_frame(&tape, 0, "Test kinematically advancing");
        assert!(frame.contains("OMNICODE CINEMA STUDIO"));
        assert!(frame.contains("TestReel"));
        assert!(frame.contains("SET PROPS"));
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/studio/kinematics.rs RS 104 LINES 456 WORDS 4.9 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Cinematic Reel Kinematics & Transport Instructions )
// ============================================================================
//! OmniCode Terminal: Cinematic Reel Kinematics
//!
//! # Architectural Scope & Purpose:
//! Evaluates procedural roll statements against the 4-directional [`MemoryReel`]:
//! - Transport Actions: `STOP`, `HALT`, `PAUSE`, `PLAY`.
//! - Spatial Spindle Kinematics: Sinking in (`v--`), Popping out (`^--`), Pulling back (`<--`).
//! - Hardware MMIO Video Projection: Screen / VGA writes at `0x000B8000`.
//! - Inscribing words forward into the reel (`-->`).
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**: Tables of testimony, written with the finger of God.
//! - **Proverbs 24:27 (KJV)**: Prepare thy work without, and make it fit for thyself in the field.
//! - **Proverbs 22:28 (KJV)**: Remove not the ancient landmark, which thy fathers have set.
// ============================================================================

use crate::core::mmio::VgaColor;
use crate::core::word::Word64;

use super::hud::OmniStudio;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Kinematic Step Methods on OmniStudio )
// ============================================================================

impl OmniStudio {
    /// Interprets a single tape roll instruction against the memory reel.
    pub fn step_roll(&mut self, roll_text: &str) -> String {
        // ====================================================================
        // BLOCK 3: BODY BLOCK
        // ( Motion Evaluation & Hardware MMIO Dispatch )
        // ====================================================================
        if roll_text.contains("STOP") || roll_text.contains("HALT") {
            self.reel.stop();
            return "Kinematics: Reel stopped and transport sealed (■ STOP)".to_string();
        }
        if roll_text.contains("PAUSE") {
            self.reel.pause();
            return "Kinematics: Track paused waiting on cue (⏸ PAUSE)".to_string();
        }
        if roll_text.contains("v--") {
            let scene_name = if let Some(pos) = roll_text.find("v--") {
                roll_text.get(pos + 3..).unwrap_or("").split_whitespace().next().unwrap_or("SubScene")
            } else {
                "SubScene"
            };
            self.reel.sink_in(scene_name, 16, None);
            return format!("Kinematics: Sunk into sub-scene <{}> (v-- Spindle Depth {})", scene_name,
self.reel.depth());
        }
        if roll_text.contains("^--") {
            let _ = self.reel.pop_out();
            return format!("Kinematics: Popped out to parent scene (^-- Spindle Depth {})", self.reel.depth());
        }
        if roll_text.contains("<--") {
            self.reel.pull_back();
            return "Kinematics: Pulled back track head (<--)".to_string();
        }
        if roll_text.contains("Screen") || roll_text.contains("VGA") || roll_text.contains("CONSOLE") ||
roll_text.contains("0x000B8000") {
            let extracted = if let Some(first_quote) = roll_text.find('\'') {
                if let Some(second_quote) = roll_text.get(first_quote + 1..).and_then(|s| s.find('\'')) {
                    roll_text.get(first_quote + 1..first_quote + 1 + second_quote)
                } else {
                    None
                }
            } else if let Some(first_quote) = roll_text.find('"') {
                if let Some(second_quote) = roll_text.get(first_quote + 1..).and_then(|s| s.find('"')) {
                    roll_text.get(first_quote + 1..first_quote + 1 + second_quote)
                } else {
```

```
                    None
                }
            } else {
                None
            };

            if let Some(screen_msg) = extracted {
                self.vga.write_str(screen_msg, VgaColor::LightGreen, VgaColor::Black);
                self.vga.new_line();
                return format!("Hardware MMIO: Wrote '{screen_msg}' to Bare-Metal Video Buffer @ 0x000B8000");
            }
        }
        if roll_text.contains("-->") {
            let val = roll_text
                .split_whitespace()
                .find_map(|s| s.parse::<i64>().ok())
                .unwrap_or(1);
            let word = Word64::from_signed_int(val);
            let _ = self.reel.push_forward(word);
            return format!("Kinematics: Inscribed word ({val}) and pushed forward (-->)");
        }
        self.reel.play();
        "Kinematics: Tape head advancing smoothly (▶ PLAY)".to_string()
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Kinematics Subsystem Verification )
// ============================================================================
// Reel kinematics layer verified and closed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/studio/mod.rs RS 36 LINES 186 WORDS 1.7 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Terminal Studio Subsystem Identity & Public Exports )
// ============================================================================
//! OmniCode Terminal: Cinema Studio Subsystem Module
//!
//! Provides the real-time visual projection HUD of the 4-directional Memory Reel,
//! showing active tracks, scenes, rolls, and set props (64-bit / 32-trit cells).
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**: Tables of testimony, written with the finger of God.
//! - **Proverbs 24:27 (KJV)**: Prepare thy work without, and make it fit for thyself in the field.
//! - **Proverbs 22:28 (KJV)**: Remove not the ancient landmark, which thy fathers have set.
// ============================================================================

pub mod hud;
pub mod kinematics;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Re-export Declarations for Cinema Studio )
// ============================================================================
pub use hud::OmniStudio;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Subsystem Invariants & Cinema Studio Proofs )
// ============================================================================
// Cinema Studio decouples visual ANSI HUD rendering from memory reel kinematics.

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Verification & Module Closure )
// ============================================================================
// Cinema Studio subsystem verified and closed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/universal/mod.rs RS 54 LINES 197 WORDS 2.0 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Terminal Universal Subsystem Identity & Public Exports )
// ============================================================================
//! OmniCode Terminal: Universal Subsystem Module
//!
//! Provides fundamental hardware MMIO ports (`0x03F8`, `0x000B8000`),
//! directional I/O stream buffers, and the universal Call-Response Engine.
//!
//! # Scripture Witness:
//! - **Job 14:15 (KJV)**: "Thou shalt call, and I will answer thee."
//! - **Exodus 31:18 (KJV)**: Tables of testimony, written with the finger of God.
//! - **Proverbs 24:27 (KJV)**: Prepare thy work without, and make it fit for thyself.
// ============================================================================

pub mod ports;
pub mod call_response_engine;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Re-export Declarations for Universal Terminal Substrates )
// ============================================================================
pub use ports::{
    UART_COM1_INPUT_PORT,
    VGA_TEXT_OUTPUT_PORT,
    DEFAULT_TERMINAL_BUFFER_CAPACITY,
    TerminalInputStream,
    TerminalOutputStream,
};

pub use call_response_engine::{
    Call,
    Response,
    CharacterResponse,
    NumberResponse,
    StreamResponse,
    CallResponseEngine,
    EchoEngine,
    EchoResult,
};

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Universal Invariants & Subsystem Proofs )
// ============================================================================
// Hardware MMIO ports and the Call-Response Engine are decoupled from
// interactive session shells and process dispatchers.

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Verification & Module Closure )
// ============================================================================
// Universal Terminal Subsystem verified and closed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/universal/ports.rs RS 215 LINES 844 WORDS 7.3 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Terminal Hardware MMIO Ports, Directional Streams & Invariant Constants )
// ================================================================================
//! OmniCode Terminal: Hardware MMIO Ports & Directional Stream Substrate
//!
//! # Architectural Scope & Purpose:
//! Defines the physical silicon MMIO port addresses and ring buffer streams
//! connecting the host hardware environment to the OmniCode Virtual Machine:
//! - `FROM ::@ 0x03F8`: COM1 UART Serial Input port (Host Keyboard / Stdin).
//! - `INTO ::@ 0x000B8000`: Video Text Buffer / Console Output port (Host Screen / Stdout).
//! - Directional FIFO streaming containers ([`TerminalInputStream`], [`TerminalOutputStream`]).
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of physical data across bounded hardware channels).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic hardware port substrates before constructing user interfaces).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical MMIO addresses: UART 0x03F8 and VGA 0x000B8000).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **MMIO Port Invariance**: Explicit mapping of physical hardware addresses guarantees
//!    isomorphic behavior whether executing on the Linux host CLI, inside QEMU bare-metal,
//!    or via the local Web GUI server.
// ================================================================================

use std::collections::VecDeque;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Hardware Port Constants & Stream Struct Definitions )
// ================================================================================

/// Hardware MMIO Address for COM1 UART Serial Input (`FROM ::@ 0x03F8`).
pub const UART_COM1_INPUT_PORT: usize = 0x03F8;

/// Hardware MMIO Address for Video Graphics Array Text Buffer (`INTO ::@ 0x000B8000`).
pub const VGA_TEXT_OUTPUT_PORT: usize = 0x000B8000;

/// Default ring buffer capacity for terminal stream queues (1,024 characters).
pub const DEFAULT_TERMINAL_BUFFER_CAPACITY: usize = 1024;

/// Directional FIFO input stream connected to the hardware `FROM` port (`0x03F8`).
#[derive(Debug, Clone)]
pub struct TerminalInputStream {
    buffer: VecDeque<char>,
    max_capacity: usize,
}

/// Directional FIFO output stream connected to the hardware `INTO` port (`0x000B8000`).
#[derive(Debug, Clone)]
pub struct TerminalOutputStream {
    buffer: VecDeque<char>,
    max_capacity: usize,
}

// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Directional Stream Methods & Buffer Operations )
// ================================================================================

impl Default for TerminalInputStream {
    fn default() -> Self {
        Self::new(DEFAULT_TERMINAL_BUFFER_CAPACITY)
    }
}

impl TerminalInputStream {
```

```
    /// Creates a new directional input stream with the designated buffer capacity.
    pub fn new(capacity: usize) -> Self {
        Self {
            buffer: VecDeque::with_capacity(capacity),
            max_capacity: capacity,
        }
    }

    /// Pushes a single character into the input buffer (simulating a hardware UART arrival).
    pub fn push_char(&mut self, c: char) {
        if self.buffer.len() >= self.max_capacity {
            self.buffer.pop_front();
        }
        self.buffer.push_back(c);
    }

    /// Pushes an entire string slice into the input buffer.
    pub fn push_str(&mut self, s: &str) {
        for c in s.chars() {
            self.push_char(c);
        }
    }

    /// Pops the next available character from the input queue.
    pub fn pop_char(&mut self) -> Option<char> {
        self.buffer.pop_front()
    }

    /// Returns `true` if the input buffer has no pending characters.
    pub fn is_empty(&self) -> bool {
        self.buffer.is_empty()
    }

    /// Returns the number of characters currently buffered.
    pub fn len(&self) -> usize {
        self.buffer.len()
    }

    /// Clears all pending characters from the stream.
    pub fn clear(&mut self) {
        self.buffer.clear();
    }
}

impl Default for TerminalOutputStream {
    fn default() -> Self {
        Self::new(DEFAULT_TERMINAL_BUFFER_CAPACITY)
    }
}

impl TerminalOutputStream {
    /// Creates a new directional output stream with the designated buffer capacity.
    pub fn new(capacity: usize) -> Self {
        Self {
            buffer: VecDeque::with_capacity(capacity),
            max_capacity: capacity,
        }
    }

    /// Pushes a projected character into the output buffer (simulating a VGA text write).
    pub fn push_char(&mut self, c: char) {
        if self.buffer.len() >= self.max_capacity {
            self.buffer.pop_front();
        }
        self.buffer.push_back(c);
    }

    /// Pushes an entire string slice into the output buffer.
    pub fn push_str(&mut self, s: &str) {
        for c in s.chars() {
            self.push_char(c);
        }
    }

    /// Pops the next projected character from the output queue.
    pub fn pop_char(&mut self) -> Option<char> {
        self.buffer.pop_front()
    }
```

```rust

    /// Flushes all pending output characters into a single allocated String.
    pub fn flush_to_string(&mut self) -> String {
        let s: String = self.buffer.iter().collect();
        self.buffer.clear();
        s
    }

    /// Returns `true` if the output buffer has no pending characters.
    pub fn is_empty(&self) -> bool {
        self.buffer.is_empty()
    }

    /// Returns the number of characters currently buffered.
    pub fn len(&self) -> usize {
        self.buffer.len()
    }

    /// Clears all pending characters from the stream.
    pub fn clear(&mut self) {
        self.buffer.clear();
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Test Verification Proofs & Stream Invariants )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_hardware_mmio_port_constants() {
        assert_eq!(UART_COM1_INPUT_PORT, 0x03F8);
        assert_eq!(VGA_TEXT_OUTPUT_PORT, 0x000B8000);
    }

    #[test]
    fn test_directional_input_stream_lifecycle() {
        let mut input = TerminalInputStream::new(16);
        assert!(input.is_empty());
        input.push_str("GLYPH");
        assert_eq!(input.len(), 5);
        assert_eq!(input.pop_char(), Some('G'));
        assert_eq!(input.pop_char(), Some('L'));
        assert_eq!(input.len(), 3);
        input.clear();
        assert!(input.is_empty());
    }

    #[test]
    fn test_directional_output_stream_flush() {
        let mut output = TerminalOutputStream::new(16);
        output.push_str("ECHO_OK");
        assert_eq!(output.len(), 7);
        let flushed = output.flush_to_string();
        assert_eq!(flushed, "ECHO_OK");
        assert!(output.is_empty());
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/universal/call_response_engine/call_response.rs

RS | 443 LINES | 1816 WORDS | 17.6 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Universal Call-Response Engine: Operator <-> Machine Bidirectional Ribbon )
// ================================================================================
//! OmniCode Terminal: Universal Call & Response Protocol Engine
//!
//! # Architectural Scope & Purpose:
//! Implements the universal **Call & Response** mathematical model of the
//! OmniCode framework:
//!
//! - **The Call (Caller / Sender Input):**
//!   Rank 5 outside-in asymmetric imposition into the silicon/virtual container:
//!   `{ Call ] --> INSCRIBE [ Container ] = [ Entity' ]`.
//! - **The Response (Via Caller / Sender Output):**
//!   Rank 4 inside-out emergence projecting verified state back to the sender:
//!   `[ Entity' ] --> PROJECT [ Response ] = [ Response' ]`.
//!
//! # Reversibility & AI Cognition:
//! Because OmniCode is designed for creating an AI model (where `.omni` and `.tape`
//! are the compiled and interpreted forms of the OmniCode framework rather than
//! passive data types), the Operator and the Machine (the computer) are symmetric
//! peers taking turns as Caller and Sender:
//! - **Operator Calls -> Machine Responds:** User/Agent commands, scalar inputs, glyphs.
//! - **Machine Calls -> Operator Responds:** Machine interrupts, hardware telemetry,
//!   predictive inference requests, and future networking packets across distributed nodes.
//!
//! An "echo" is simply the special case of Call-Response that repeats what was called
//! for after proving invariant bounds and ALU equilibrium.
//!
//! # Scripture Witness:
//! - **Job 14:15 (KJV)**:
//!   "Thou shalt call, and I will answer thee: thou wilt have a desire to the
//!    work of thine hands."
//!   (The primordial covenant of Call and Response between Creator and Creation).
//! - **Isaiah 65:24 (KJV)**:
//!   "And it shall come to pass, that before they call, I will answer;
//!    and while they are yet speaking, I will hear."
//!   (Zero-latency predictive Call-Response in equilibrium).
// ================================================================================

use crate::core::cell::{Cell, Trit};
use crate::core::ternary::QuadState;
use crate::core::word::{PackMode, Word32Trit};

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Types, Enums & Status Containers )
// ================================================================================

/// A Call payload transmitted by the Caller / Sender (Operator or Machine).
#[derive(Debug, Clone, PartialEq)]
pub enum Call {
    /// Inscription of an atomic symbolic rune / glyph (1 Cell capacity).
    Character(char),
    /// Inscription of a scalar magnitude / quantity (1 Word capacity).
    Number(i64),
    /// Inscription of a composite stream of characters (Text ribbon).
    Stream(String),
    /// Direct injection of a 64-bit dual-rail polar bit-plane register.
    DualRail(u64),
}

/// Result of a verified Character Call-Response turn.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct CharacterResponse {
    /// Echoed / verified character glyph.
    pub glyph: char,
    /// 5 balanced trits representing the character in virtual cell memory.
    pub trits: [i8; 5],
    /// Balanced ternary Quad-State (+1:PROCEED).
    pub state: QuadState,
    /// Verified cell capacity footprint (always 1 Cell).
    pub cell_footprint: usize,
```

```rust
}

/// Result of a verified Number Call-Response turn.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct NumberResponse {
    /// Outside-in scalar integer quantity.
    pub scalar: i64,
    /// 32 balanced trits representing the quantity in virtual word memory.
    pub trits: [i8; 32],
    /// Native 64-bit dual-rail polar bit-plane register (00=0, 01=+1, 10=-1, 11=Trap).
    pub packed_dual_rail: u64,
    /// Canonical dense 2-2-2-2 packing of least significant 4 trits (1 byte).
    pub byte_pack_2222: u8,
    /// Kinematic 2-1-2-1-2 resting equilibrium packing of least significant 3 trits (1 byte).
    pub byte_pack_21212: u8,
    /// Proof of ALU Zero-Sum Equilibrium Law: (+1) + (-1) <=> 0.
    pub equilibrium_proved: bool,
    /// Proof of SIMD rail-swap negation involution: -(-x) == x.
    pub rail_involution_proved: bool,
    /// Balanced ternary Quad-State (+1:PROCEED).
    pub state: QuadState,
    /// Verified word capacity footprint (always 1 Word).
    pub word_footprint: usize,
}

/// Result of a verified Stream Call-Response turn.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct StreamResponse {
    /// Echoed / verified text stream.
    pub echoed: String,
    /// Alias for echoed stream text.
    pub text: String,
    /// Total count of atomic character cells verified.
    pub cell_count: usize,
    /// Balanced ternary Quad-State (+1:PROCEED).
    pub state: QuadState,
}

/// Universal Response delivered to the Caller / Sender via the output channel.
#[derive(Debug, Clone, PartialEq)]
pub enum Response {
    /// Verified Character response.
    Character(CharacterResponse),
    /// Verified Number response.
    Number(NumberResponse),
    /// Verified Stream response.
    Stream(StreamResponse),
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Universal Call-Response Engine Implementation )
// ============================================================================

/// The Universal Call & Response Engine.
#[derive(Debug, Clone, Default)]
pub struct CallResponseEngine;

impl CallResponseEngine {
    /// Executes any Call, returning the corresponding verified Response.
    pub fn execute(call: Call) -> Result<Response, String> {
        match call {
            Call::Character(c) => Self::call_char(c).map(Response::Character),
            Call::Number(n) => Self::call_number(n).map(Response::Number),
            Call::Stream(s) => Self::call_stream(&s).map(Response::Stream),
            Call::DualRail(r) => Self::call_dual_rail(r).map(Response::Number),
        }
    }

    /// Executes the full Call-Response lifecycle for an atomic Character glyph:
    /// 1. Ingestion: Reads glyph payload from Caller.
    /// 2. Inscription: Allocates single 1-Cell container.
    /// 3. Transduction: Bijectively converts 8-bit byte <=> 5 balanced trits.
    /// 4. Guard: Checks identity, single-cell capacity, and ASCII boundary.
    /// 5. Response: Returns verified CharacterResponse.
    pub fn call_char(glyph: char) -> Result<CharacterResponse, String> {
        let byte = if (glyph as u32) <= 242 {
            glyph as u8
```

```
        } else {
            return Err(format!(
                "Character '{}' (U+{:04X}) exceeds single-cell capacity of 242 values",
                glyph, glyph as u32
            ));
        };

        let cell = Cell::from_natural(byte)?;
        let mut trits = [0i8; 5];
        for (i, t) in cell.trits.iter().enumerate() {
            trits[i] = t.as_i8();
        }

        let mut cell_trits = [Trit::Zero; 5];
        for (i, &t) in trits.iter().enumerate() {
            cell_trits[i] = Trit::from_i8(t)?;
        }
        let verified_cell = Cell::new(cell_trits, QuadState::Proceed);
        let reconstructed_byte = verified_cell
            .to_natural()
            .ok_or_else(|| "Failed to decode natural byte from cell".to_string())?;

        if reconstructed_byte != byte {
            return Err(format!(
                "Transduction anomaly: byte {byte} != reconstructed {reconstructed_byte}"
            ));
        }

        let reconstructed_char = reconstructed_byte as char;
        if reconstructed_char != glyph {
            return Err(format!(
                "Identity violation: glyph '{glyph}' != reconstructed '{reconstructed_char}'"
            ));
        }

        Ok(CharacterResponse {
            glyph: reconstructed_char,
            trits,
            state: QuadState::Proceed,
            cell_footprint: 1,
        })
    }

    /// Executes the full Call-Response lifecycle for an atomic Number scalar:
    /// 1. Ingestion: Captures scalar quantity from Caller.
    /// 2. Inscription: Allocates 1-Word memory container.
    /// 3. Transduction: Bijectively converts 64-bit integer <=> 32 balanced trits.
    /// 4. Polar Rail Packing: Maps 32 trits into 64-bit dual-rail polar planes.
    /// 5. Collision Guard: Ensures no `11` collision traps exist.
    /// 6. Equilibrium Proof: Proves (+1) + (-1) <=> 0 and SIMD rail-swap involution.
    /// 7. Response: Returns verified NumberResponse.
    pub fn call_number(scalar: i64) -> Result<NumberResponse, String> {
        // Step 1 & 2: Inscribe into 32-trit Word
        let word = Word32Trit::from_i64(scalar);

        // Step 3: Bijective round-trip validation
        let reconstructed_scalar = word.to_i64();
        if reconstructed_scalar != scalar {
            return Err(format!(
                "Number transduction overflow: input {scalar} != reconstructed {reconstructed_scalar}"
            ));
        }

        // Step 4: Dual-Rail polar bit-plane packing
        let packed_dual_rail = word.pack_dual_rail();

        // Step 5: Collision trap guard
        if Word32Trit::has_null_collision(packed_dual_rail) {
            return Err(format!(
                "Dual-rail collision trap: simultaneous activation on word for scalar {scalar}"
            ));
        }

        let unpacked_word = Word32Trit::unpack_dual_rail(packed_dual_rail)?;
        if unpacked_word != word {
            return Err(format!(
                "Dual-rail unpack parity mismatch on scalar {scalar}"
            ));
```

```
        }

        // Step 6: Zero-sum equilibrium proof (+1) + (-1) == 0
        let unity_pos = Word32Trit::from_i64(1);
        let unity_neg = Word32Trit::from_i64(-1);
        let (zero_sum, carry) = unity_pos.add(&unity_neg);
        let equilibrium_proved = (zero_sum.to_i64() == 0) && (carry == 0);

        // Step 7: SIMD rail-swap negation involution proof: -(-x) == x
        let negated = Word32Trit::simd_rail_swap_negate(packed_dual_rail);
        let double_negated = Word32Trit::simd_rail_swap_negate(negated);
        let rail_involution_proved = double_negated == packed_dual_rail;

        if !equilibrium_proved || !rail_involution_proved {
            return Err("ALU Zero-Sum Equilibrium or Rail-Swap Involution violated".to_string());
        }

        // Step 8: Dynamic kinematic byte topologies (least significant trits)
        let byte_pack_2222 = Word32Trit::pack_byte(&word.trits, PackMode::Dense2222);
        let byte_pack_21212 = Word32Trit::pack_byte(&word.trits, PackMode::Kinematic21212);

        Ok(NumberResponse {
            scalar,
            trits: word.trits,
            packed_dual_rail,
            byte_pack_2222,
            byte_pack_21212,
            equilibrium_proved,
            rail_involution_proved,
            state: QuadState::Proceed,
            word_footprint: 1,
        })
    }

    /// Executes Call-Response on a raw 64-bit dual-rail register word.
    pub fn call_dual_rail(packed: u64) -> Result<NumberResponse, String> {
        if Word32Trit::has_null_collision(packed) {
            return Err("Dual-rail collision trap detected in input register (code 11)".to_string());
        }
        let word = Word32Trit::unpack_dual_rail(packed)?;
        Self::call_number(word.to_i64())
    }

    /// Executes Call-Response across an entire text stream.
    pub fn call_stream(input: &str) -> Result<StreamResponse, String> {
        let mut echoed = String::with_capacity(input.len());
        let mut cell_count = 0;
        for c in input.chars() {
            let res = Self::call_char(c)?;
            echoed.push(res.glyph);
            cell_count += 1;
        }
        Ok(StreamResponse {
            echoed: echoed.clone(),
            text: echoed,
            cell_count,
            state: QuadState::Proceed,
        })
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Comprehensive Exhaustive Unit Tests: Character, Number Range & Reversibility )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_call_response_character_standard_ascii() {
        for byte in 0u8..=127u8 {
            let ch = byte as char;
            let resp = CallResponseEngine::call_char(ch)
                .unwrap_or_else(|e| panic!("Failed call_char for ASCII {byte}: {e}"));
            assert_eq!(resp.glyph, ch);
            assert_eq!(resp.state, QuadState::Proceed);
```

```
            assert_eq!(resp.cell_footprint, 1);
        }
    }

    #[test]
    fn test_call_response_number_zero_and_unities() {
        let unities = [-1i64, 0, 1];
        for &u in &unities {
            let resp = CallResponseEngine::call_number(u).expect("Unity must succeed");
            assert_eq!(resp.scalar, u);
            assert_eq!(resp.state, QuadState::Proceed);
            assert!(resp.equilibrium_proved);
            assert!(resp.rail_involution_proved);
            assert_eq!(resp.word_footprint, 1);
        }
    }

    #[test]
    fn test_call_response_number_byte_range() {
        // Exhaustively verify the entire shared 8-bit byte space (-128 through +127)
        for val in -128i64..=127i64 {
            let resp = CallResponseEngine::call_number(val)
                .unwrap_or_else(|e| panic!("Failed call_number for byte value {val}: {e}"));
            assert_eq!(resp.scalar, val);
            assert_eq!(resp.state, QuadState::Proceed);
            assert!(resp.equilibrium_proved);
            assert!(resp.rail_involution_proved);
        }
    }

    #[test]
    fn test_call_response_number_single_cell_trit_range() {
        // Single cell balanced ternary range: 3^5 = 243 values (-121 through +121)
        for val in -121i64..=121i64 {
            let resp = CallResponseEngine::call_number(val)
                .unwrap_or_else(|e| panic!("Failed call_number for 5-trit cell value {val}: {e}"));
            assert_eq!(resp.scalar, val);
        }
    }

    #[test]
    fn test_call_response_number_powers_of_3() {
        // Test powers of 3 from 3^0 up to 3^31 and negative powers
        let mut p: i64 = 1;
        for exponent in 0..=31 {
            let resp_pos = CallResponseEngine::call_number(p)
                .unwrap_or_else(|e| panic!("Failed call_number for 3^{exponent} = {p}: {e}"));
            assert_eq!(resp_pos.scalar, p);

            let resp_neg = CallResponseEngine::call_number(-p)
                .unwrap_or_else(|e| panic!("Failed call_number for -(3^{exponent}) = -{p}: {e}"));
            assert_eq!(resp_neg.scalar, -p);

            if exponent < 31 {
                p *= 3;
            }
        }
    }

    #[test]
    fn test_call_response_number_powers_of_2() {
        // Test boundary powers of 2 fitting within 32 balanced trits (max (3^32-1)/2 ~= 9.26e14, so 2^49)
        for bit in 0..=49 {
            let v = 1i64 << bit;
            let resp_pos = CallResponseEngine::call_number(v)
                .unwrap_or_else(|e| panic!("Failed call_number for 2^{bit} = {v}: {e}"));
            assert_eq!(resp_pos.scalar, v);

            let resp_neg = CallResponseEngine::call_number(-v)
                .unwrap_or_else(|e| panic!("Failed call_number for -(2^{bit}) = -{v}: {e}"));
            assert_eq!(resp_neg.scalar, -v);
        }

        // Verify that 2^50 exceeds 32-trit single-word capacity and returns an error
        let v50 = 1i64 << 50;
        assert!(CallResponseEngine::call_number(v50).is_err(), "2^50 must exceed 32-trit single word");
    }

```

```
    #[test]
    fn test_call_response_number_all_256_bytes_dynamic_topologies() {
        // Test all 256 byte values across all 7 kinematic topologies
        let modes = [
            PackMode::Dense2222,
            PackMode::Kinematic21212,
            PackMode::Kinematic12122,
            PackMode::Kinematic22121,
            PackMode::Directional332,
            PackMode::Directional323,
            PackMode::Directional233,
        ];

        for byte_val in 0u8..=255u8 {
            for &mode in &modes {
                if let Ok(unpacked_trits) = Word32Trit::unpack_byte(byte_val, mode) {
                    // Repack and ensure round trip
                    let repacked = Word32Trit::pack_byte(&unpacked_trits, mode);
                    let unpacked_again = Word32Trit::unpack_byte(repacked, mode)
                        .expect("Repacked byte must unpack cleanly");
                    assert_eq!(unpacked_again[0], unpacked_trits[0]);
                    assert_eq!(unpacked_again[1], unpacked_trits[1]);
                    assert_eq!(unpacked_again[2], unpacked_trits[2]);
                    if mode == PackMode::Dense2222 {
                        assert_eq!(unpacked_again[3], unpacked_trits[3]);
                    }
                }
            }
        }
    }

    #[test]
    fn test_call_response_number_null_collision_trap() {
        // Inject collision code 11 (3) into dual rail
        let corrupted_register: u64 = 0b11;
        let res = CallResponseEngine::call_dual_rail(corrupted_register);
        assert!(res.is_err(), "Collision trap must reject code 11");
    }

    #[test]
    fn test_call_response_reversibility_and_stream() {
        // Test caller (operator) stream call
        let stream = "OMNICODE 2026";
        let resp = CallResponseEngine::call_stream(stream).expect("Stream must succeed");
        assert_eq!(resp.echoed, stream);
        assert_eq!(resp.text, stream);
        assert_eq!(resp.cell_count, 13);
        assert_eq!(resp.state, QuadState::Proceed);

        // Test universal execute dispatch
        let r1 = CallResponseEngine::execute(Call::Character('Z')).unwrap();
        assert!(matches!(r1, Response::Character(_)));

        let r2 = CallResponseEngine::execute(Call::Number(4096)).unwrap();
        assert!(matches!(r2, Response::Number(_)));
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/universal/call_response_engine/echo.rs RS | 155 LINES | 656 WORDS | 6.1 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Universal Echo Engine: Call-and-Response Mechanism for Characters, Numbers & Streams )
// ============================================================================
//! OmniCode Terminal: Universal Echo Protocol Engine
//!
//! # Architectural Scope & Purpose:
//! In OmniCode, **AN ECHO IS A CALL AND RESPONSE THAT REPEATS WHAT WAS CALLED FOR**.
//! Since the language is also designed for creating an AI model (where `.omni` and
//! `.tape` are the compiled and interpreted forms of the OmniCode framework, rather
//! than passive data types), the Caller and Sender represent the Operator and the
//! Machine (the computer).
//!
//! Because they are reversible, the Echo Engine handles:
//! 1. **Character Echo:** Inscribes glyph into 1 Cell, bijectively transduces to 5 trits,
//!    guards single-cell bounds, and projects verified glyph to output.
//! 2. **Number Echo:** Inscribes scalar quantity into 1 Word (32 trits <=> 64 bits),
//!    packs dual-rail polar planes (00=0, 01=+1, 10=-1), guards against 11 collision trap,
//!    proves ALU zero-sum balance (+1) + (-1) <=> 0, and projects response.
//! 3. **Stream Echo:** Chains character cells into text ribbons.
//!
//! # Scripture Witness:
//! - **Job 14:15 (KJV)**: "Thou shalt call, and I will answer thee."
//! - **Exodus 31:18 (KJV)**: Written with the finger of God.
//! - **Proverbs 24:27 (KJV)**: Prepare thy work without, and make it fit for thyself.
// ============================================================================

use crate::core::cell::{Cell, Trit};
use crate::core::ternary::QuadState;
use crate::terminal::universal::call_response_engine::call_response::{
    Call, Response, CharacterResponse, NumberResponse, StreamResponse, CallResponseEngine,
};

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Types, Enums & Status Containers )
// ============================================================================

/// Universal Echo Result returned by [`EchoEngine::echo`].
#[derive(Debug, Clone, PartialEq)]
pub enum EchoResult {
    /// Atomic Character Echo.
    Character(CharacterResponse),
    /// Scalar Number Echo.
    Number(NumberResponse),
    /// Stream / Text Echo.
    Stream(StreamResponse),
}

/// The Universal Echo Protocol Engine.
#[derive(Debug, Clone, Default)]
pub struct EchoEngine;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Operational Echo Methods )
// ============================================================================

impl EchoEngine {
    /// Executes a universal Echo turn for any [`Call`] payload.
    pub fn echo(call: Call) -> Result<EchoResult, String> {
        match CallResponseEngine::execute(call)? {
            Response::Character(c) => Ok(EchoResult::Character(c)),
            Response::Number(n) => Ok(EchoResult::Number(n)),
            Response::Stream(s) => Ok(EchoResult::Stream(s)),
        }
    }

    /// Echoes a single character glyph through 5-trit radix-3 cell transduction.
    pub fn echo_char(glyph: char) -> Result<CharacterResponse, String> {
        CallResponseEngine::call_char(glyph)
    }

```

```
    /// Echoes a scalar number through 32-trit dual-rail polar bit-planes.
    pub fn echo_number(scalar: i64) -> Result<NumberResponse, String> {
        CallResponseEngine::call_number(scalar)
    }

    /// Echoes an entire text stream through continuous character cells.
    pub fn echo_stream(input: &str) -> Result<StreamResponse, String> {
        CallResponseEngine::call_stream(input)
    }

    /// Transduces an 8-bit ASCII byte into 5 balanced trits using canonical [`Cell`] natural encoding.
    pub fn byte_to_5_trits(byte: u8) -> Result<[i8; 5], String> {
        let cell = Cell::from_natural(byte)?;
        let mut trits = [0i8; 5];
        for (i, t) in cell.trits.iter().enumerate() {
            trits[i] = t.as_i8();
        }
        Ok(trits)
    }

    /// Reconstructs an 8-bit byte from 5 balanced trits using canonical [`Cell`] natural decoding.
    pub fn five_trits_to_byte(trits: &[i8; 5]) -> Result<u8, String> {
        let mut cell_trits = [Trit::Zero; 5];
        for (i, &t) in trits.iter().enumerate() {
            cell_trits[i] = Trit::from_i8(t)?;
        }
        let cell = Cell::new(cell_trits, QuadState::Proceed);
        cell.to_natural().ok_or_else(|| "Failed to decode natural byte from cell".to_string())
    }

    /// Verifies that a character fits strictly within single-cell bounds.
    pub fn verify_single_cell_bound(c: char) -> bool {
        c.len_utf8() <= 4
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests for Universal Echo Protocol Engine )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_echo_char_letters_and_symbols() {
        let test_chars = ['A', 'Z', 'a', 'z', '0', '9', '!', '@', '#', '$', ' '];
        for &c in &test_chars {
            let res = EchoEngine::echo_char(c).expect("Echo char must succeed");
            assert_eq!(res.glyph, c);
            assert_eq!(res.state, QuadState::Proceed);
            assert_eq!(res.cell_footprint, 1);
        }
    }

    #[test]
    fn test_echo_number_equilibrium() {
        let res = EchoEngine::echo_number(42).expect("Echo number must succeed");
        assert_eq!(res.scalar, 42);
        assert!(res.equilibrium_proved);
        assert!(res.rail_involution_proved);
    }

    #[test]
    fn test_echo_stream() {
        let text = "OmniCode Universal Echo OK";
        let res = EchoEngine::echo_stream(text).expect("Stream echo must succeed");
        assert_eq!(res.echoed, text);
        assert_eq!(res.text, text);
    }

    #[test]
    fn test_echo_universal_call_dispatch() {
        let res_char = EchoEngine::echo(Call::Character('K')).unwrap();
        assert!(matches!(res_char, EchoResult::Character(_)));

        let res_num = EchoEngine::echo(Call::Number(-1)).unwrap();
```

```
        assert!(matches!(res_num, EchoResult::Number(_)));
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/universal/call_response_engine/mod.rs

RS | 27 LINES | 74 WORDS | 0.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Call-Response Engine Subsystem Module )
// ============================================================================
//! OmniCode Terminal: Call & Response Engine Module
//!
//! Subsumes character echo and number execution into the unified Call-and-Response
//! protocol, defining Echo as the Call-Response mechanism that repeats what was called for.
// ============================================================================

pub mod call_response;
pub mod echo;

pub use call_response::{
    Call,
    Response,
    CharacterResponse,
    NumberResponse,
    StreamResponse,
    CallResponseEngine,
};

pub use echo::{
    EchoResult,
    EchoEngine,
};
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/cli/mod.rs RS 44 LINES 157 WORDS 2.0 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( CLI Subsystem Identity, Module Architecture & Core Re-exports )
// ============================================================================
//! OmniCode Command Line Subsystem
//!
//! Exposes interactive terminal REPL, script runners, structural 4-block linters,
//! and template scaffolding generators.

pub mod repl;
pub mod server;
pub mod commands;


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Re-export Declarations & Public Interface )
// ============================================================================
pub use repl::run_repl;
pub use server::run_gui_server;
pub use commands::{
    BootArch, BootConfig, BootContext, run_boot, run_boot_with_config,
    execute_kernel_pipeline, synthesize_hypervisor_command, execute_boot_pipeline, execute_boot_machine,
    DEFAULT_BOOT_ARCH, DEFAULT_GRAPHICAL_MODE, REAL_MODE_BOOT_VECTOR, VGA_TEXT_FRAMEBUFFER_BASE, COM1_SERIAL_IO_BASE,
    AssembleFormat, AssembleConfig, AssembleContext, run_assemble, run_assemble_with_config,
    execute_assemble_pipeline, serialize_assembled_output,
    RunFormat, RunConfig, RunContext, RunResult, run_with_config, execute_run_pipeline,
    run_on_virtual_terminal, run_omni_source, run_tape_ribbon, run_c_transpile, run_lua_transpile,
};


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( CLI Orchestration Framework )
// ============================================================================
// Command line entrypoints route to interactive REPL or single-file runners.


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Verification and Sealing )
// ============================================================================
// CLI subsystem verified and sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/cli/repl.rs RS 427 LINES 1585 WORDS 21.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Interactive Terminal Environment Identity, REPL Architecture & Dependencies )
// ============================================================================
//! OmniCode Terminal REPL: Interactive Ontological Shell
//!
//! Provides an interactive read-eval-print loop (REPL) enabling direct,
//! zero-latency experimentation with OmniCode's foundational primitives:
//!     - Balanced Ternary Logic (-1 Rollback, 0 Await, +1 Proceed, NULL Halt)
//!     - 6 Boundary Morphologies & Asymmetric Closures
//!     - First-Class Intent Asides (...)
//!     - 3D Coordinate Operators & Directional Vectors
//!     - Transactional Environment Snapshot & Rollback
//!     - Real-Time Diagnostic Failpoint Trapping

use std::fs;
use std::path::PathBuf;
use rustyline::error::ReadlineError;
use rustyline::DefaultEditor;

use crate::ast::nodes::ASTNode;
use crate::core::ternary::QuadState;
use crate::diagnostics::debugger::OmniDebugger;
use crate::diagnostics::logger::DiagnosticLogger;
use crate::diagnostics::linter::OmniLinter;
use crate::parser::lexer::Lexer;
use crate::parser::parser::Parser;
use crate::runtime::environment::{Environment, RuntimeValue};
use crate::runtime::evaluator::Evaluator;


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( REPL Configuration, Banner Display, and Context State Initialization )
// ============================================================================
/// Resolves the file path for persistent terminal command history.
fn get_history_path() -> Option<PathBuf> {
    std::env::var("HOME").ok().map(|h| PathBuf::from(h).join(".omnicode_history"))
}

/// Checks whether an interactive input buffer has balanced delimiters.
fn is_input_complete(input: &str) -> bool {
    let trimmed = input.trim_end();
    if trimmed.ends_with('\\') {
        return false;
    }

    let mut open_brackets = 0isize;
    let mut in_string = false;
    let mut quote_char = ' ';

    for line in input.lines() {
        let stripped = line.trim();
        if stripped.starts_with('#') {
            continue;
        }

        let chars: Vec<char> = line.chars().collect();
        let mut i = 0;
        while i < chars.len() {
            let c = chars[i];
            if in_string {
                if c == '\\' {
                    i += 2;
                    continue;
                }
                if c == quote_char {
                    in_string = false;
                }
            } else {
                match c {
                    '#' => break, // Comment extends to end of line
                    '"' | '\'' => {
                        in_string = true;
```

```
                        quote_char = c;
                    }
                    '[' | '{' | '<' | '(' => open_brackets += 1,
                    ']' | '}' | '>' | ')' => open_brackets -= 1,
                    _ => {}
                }
            }
            i += 1;
        }
    }

    open_brackets <= 0
}

/// Displays the welcome banner and prompt instructions.
fn print_welcome_banner() {
    println!("\x1b[1;36m================================================================================\x1b[0m");
    println!("\x1b[1;36m  OmniCode Interactive Shell (v0.1.0)\x1b[0m");
    println!("\x1b[1;36m  Core: Balanced Ternary & Boundary Physics (Rust Engine)\x1b[0m");
    println!("\x1b[0;37m  Type \x1b[1;33m:help\x1b[0m\x1b[0;37m or \x1b[1;33m:?\x1b[0m\x1b[0;37m for commands & syntax cheat sheet. Press \x1b[1;33mCtrl+D\x1b[0m\x1b[0;37m to exit.\x1b[0m");
    println!("\x1b[1;36m================================================================================\x1b[0m\n");
}

/// Displays the help guide and syntax reference.
fn print_help_guide() {
    println!("\x1b[1;33m--- OmniCode Interactive Commands ---\x1b[0m");
    println!("  \x1b[1;32m:help\x1b[0m, \x1b[1;32m:?\x1b[0m              Show this help reference");
    println!("  \x1b[1;32m:env\x1b[0m                  Inspect all active symbols and entity bindings");
    println!("  \x1b[1;32m:ast <expr>\x1b[0m           Display raw AST structure of an expression");
    println!("  \x1b[1;32m:load <path>\x1b[0m          Load, lint, parse, and evaluate an OmniCode file (.omni / .scroll)");
    println!("  \x1b[1;32m:lint <path>\x1b[0m          Validate file against 4-Block architecture (.omni / .scroll)");
    println!("  \x1b[1;32m:failpoints\x1b[0m           Review all failpoint incidents trapped in this session");
    println!("  \x1b[1;32m:verbose [on|off]\x1b[0m     Toggle real-time microsecond diagnostic logging");
    println!("  \x1b[1;32m:clear\x1b[0m                Clear the terminal screen");
    println!("  \x1b[1;32m:reset\x1b[0m                Reset the environment and diagnostic stack");
    println!("  \x1b[1;32m:exit\x1b[0m, \x1b[1;32m:quit\x1b[0m          Exit the OmniCode interactive shell\n");

    println!("\x1b[1;33m--- Notation & Physics Cheat Sheet ---\x1b[0m");
    println!("  \x1b[1;36mBoundaries:\x1b[0m   [...] mutable | <...> immutable axiom | {{...}} context");
    println!("                {{...] top-down asymmetric | [...}} bottom-up asymmetric | (...) intent aside");
    println!("  \x1b[1;36mTernary:\x1b[0m      +1 (Proceed) | 0 (Await) | -1 (Rollback) | NULL (Halt)");
    println!("  \x1b[1;36mRelations:\x1b[0m    = (Is Identity) | != (Is Not) | ~= (To Be Like)");
    println!("  \x1b[1;36mVectors:\x1b[0m      --> (Act Upon / Forward) | <-- (Act Again / Rollback)");
    println!("                ^-- (Builds To) | v-- (Breaks Down From)");
    println!("  \x1b[1;36mConnectives:\x1b[0m  ! (Adversative Caveat BUT) | :: (Conjunction AND) | | (Disjunction OR)\n");
}


// ==========================================================================
// BLOCK 3: BODY BLOCK
// ( Core REPL Execution Loop: Command Dispatch, Parser, Evaluator Pipeline )
// ==========================================================================
/// Launches and executes the OmniCode interactive REPL terminal session.
pub fn run_repl() -> Result<(), Box<dyn std::error::Error>> {
    print_welcome_banner();

    let mut rl = DefaultEditor::new()?;
    let history_file = get_history_path();
    if let Some(ref path) = history_file {
        let _ = rl.load_history(path);
    }

    let logger = DiagnosticLogger::new(false);
    let mut debugger = OmniDebugger::new(logger.clone(), 10);
    let mut env = Environment::new();
    let mut current_state = QuadState::Proceed;
    let mut verbose = false;
    let mut multiline_buffer = String::new();

    loop {
        let prompt = if multiline_buffer.is_empty() {
            match current_state {
                QuadState::Proceed => "\x1b[1;32momni[+1]>\x1b[0m ".to_string(),
                QuadState::Await => "\x1b[1;33momni[0]>\x1b[0m ".to_string(),
                QuadState::Rollback => "\x1b[1;31momni[-1]>\x1b[0m ".to_string(),
```

```
                QuadState::Halt => "\x1b[1;35momni[NULL]>\x1b[0m ".to_string(),
            }
        } else {
            "\x1b[1;30m   ...>\x1b[0m ".to_string()
        };

        let readline = rl.readline(&prompt);
        match readline {
            Ok(line) => {
                let trimmed = line.trim();

                // Multiline buffering logic
                if !multiline_buffer.is_empty() {
                    if trimmed.is_empty() {
                        // Empty line forces execution of buffered input
                    } else {
                        multiline_buffer.push('\n');
                        multiline_buffer.push_str(&line);
                        if !is_input_complete(&multiline_buffer) {
                            continue;
                        }
                    }
                } else if !trimmed.is_empty() && !trimmed.starts_with(':') && !is_input_complete(trimmed) {
                    multiline_buffer.push_str(&line);
                    continue;
                }

                let input = if !multiline_buffer.is_empty() {
                    let full = multiline_buffer.clone();
                    multiline_buffer.clear();
                    full
                } else {
                    line
                };

                let input_trimmed = input.trim();
                if input_trimmed.is_empty() {
                    continue;
                }

                let _ = rl.add_history_entry(input_trimmed);

                // Command Dispatch
                if input_trimmed.starts_with(':') || input_trimmed == "exit" || input_trimmed == "quit" {
                    let cmd_parts: Vec<&str> = input_trimmed.splitn(2, ' ').collect();
                    let cmd = cmd_parts[0];
                    let arg = if cmd_parts.len() > 1 { cmd_parts[1].trim() } else { "" };

                    match cmd {
                        ":help" | ":?" => print_help_guide(),
                        ":exit" | ":quit" | "exit" | "quit" => {
                            println!("\x1b[1;36mSealing OmniCode session. Goodbye.\x1b[0m");
                            break;
                        }
                        ":clear" => {
                            print!("\x1b[2J\x1b[1;1H");
                        }
                        ":reset" => {
                            env.clear();
                            logger.clear();
                            debugger = OmniDebugger::new(logger.clone(), 10);
                            current_state = QuadState::Proceed;
                            println!("\x1b[1;32m✓ Session reset: Environment and failpoints cleared.\x1b[0m");
                        }
                        ":env" => {
                            let bindings = env.bindings();
                            if bindings.is_empty() {
                                println!("\x1b[0;37m(Environment is empty)\x1b[0m");
                            } else {
                                println!("\x1b[1;33m--- Active Symbols & Bindings ({}) ---\x1b[0m", bindings.len());
                                for (k, v) in bindings {
                                    println!("  \x1b[1;36m{}\x1b[0m = {}", k, v.format_colored());
                                }
                            }
                        }
                        ":tokens" => {
                            if arg.is_empty() {
                                eprintln!("\x1b[1;31mUsage:\x1b[0m :tokens <expression>");
```

```
                                } else {
                                    let mut lex = Lexer::new(arg, logger.clone());
                                    match lex.tokenize() {
                                        Ok(toks) => {
                                            println!("\x1b[1;33mTokens ({}):\x1b[0m", toks.len());
                                            for t in toks {
                                                println!("  {:?}  \x1b[0;37m('{}' at L{}:C{})\x1b[0m", t.token_type,
t.lexeme, t.line, t.col);
                                            }
                                        }
                                        Err(e) => eprintln!("\x1b[1;31m✗ Lexer Error:\x1b[0m {}", e),
                                    }
                                }
                            }
                            ":ast" => {
                                if arg.is_empty() {
                                    eprintln!("\x1b[1;31mUsage:\x1b[0m :ast <expression>");
                                } else {
                                    let mut lex = Lexer::new(arg, logger.clone());
                                    match lex.tokenize() {
                                        Ok(toks) => {
                                            let mut p = Parser::new(toks, debugger.clone());
                                            match p.parse_program() {
                                                Ok(ast) => println!("{:#?}", ast),
                                                Err(e) => eprintln!("\x1b[1;31m✗ Parser Error:\x1b[0m {}", e),
                                            }
                                        }
                                        Err(e) => eprintln!("\x1b[1;31m✗ Lexer Error:\x1b[0m {}", e),
                                    }
                                }
                            }
                            ":failpoints" => {
                                let fps = logger.failpoints();
                                if fps.is_empty() {
                                    println!("\x1b[1;32m✓ No failpoint incidents recorded.\x1b[0m");
                                } else {
                                    println!("\x1b[1;33m--- Trapped Failpoint Incidents ({}) ---\x1b[0m", fps.len());
                                    for fp in fps {
                                        let fp_name = fp.failpoint_type.map(|t| t.as_str()).unwrap_or("GENERIC");
                                        println!("  * [{} µs] \x1b[1;31m[{}]\x1b[0m [{}] {}", fp.elapsed_micros, fp.channel,
fp_name, fp.message);
                                    }
                                }
                            }
                            ":verbose" => {
                                if arg == "on" {
                                    verbose = true;
                                    println!("\x1b[1;32m✓ Diagnostic verbose logging ENABLED.\x1b[0m");
                                } else if arg == "off" {
                                    verbose = false;
                                    println!("\x1b[1;33m✓ Diagnostic verbose logging DISABLED.\x1b[0m");
                                } else {
                                    verbose = !verbose;
                                    println!("Diagnostic verbose logging is now: \x1b[1;36m{}\x1b[0m", if verbose { "ON" }
else { "OFF" });
                                }
                            }
                            ":lint" => {
                                if arg.is_empty() {
                                    eprintln!("\x1b[1;31mUsage:\x1b[0m :lint <file.omni | file.scroll>");
                                } else {
                                    match fs::read_to_string(arg) {
                                        Ok(content) => {
                                            let linter = OmniLinter::new(logger.clone());
                                            let report = linter.lint_source(arg, &content);
                                            if report.is_valid {
                                                println!("\x1b[1;32m✓ 4-Block Structural Check Passed:\x1b[0m {} blocks
verified in canonical sequence.", report.blocks_found.len());
                                            } else {
                                                println!("\x1b[1;31m✗ 4-Block Structural Linter Violations:\x1b[0m");
                                                for err in report.errors {
                                                    println!("    \x1b[1;31m•\x1b[0m {}", err);
                                                }
                                            }
                                        }
                                        Err(e) => eprintln!("\x1b[1;31m✗ File read error:\x1b[0m {}", e),
                                    }
                                }
```

```
                        }
                        ":load" => {
                            if arg.is_empty() {
                                eprintln!("\x1b[1;31mUsage:\x1b[0m :load <file.omni | file.scroll>");
                            } else {
                                match fs::read_to_string(arg) {
                                    Ok(source) => {
                                        let prev_fps_len = logger.failpoints().len();
                                        let mut lex = Lexer::new(&source, logger.clone());
                                        match lex.tokenize() {
                                            Ok(toks) => {
                                                let mut p = Parser::new(toks, debugger.clone());
                                                match p.parse_program() {
                                                    Ok(prog) => {
                                                        let mut eval = Evaluator::new(env.clone(), p.debugger);
                                                        match eval.eval(&ASTNode::Program(prog)) {
                                                            Ok(res) => {
                                                                env = eval.env;
                                                                debugger = eval.debugger;
                                                                if let RuntimeValue::State(s) = res {
                                                                    current_state = s;
                                                                }
                                                                println!("\x1b[1;32m✓ Loaded and evaluated '{}':\x1b[0m
=> {}", arg, res.format_colored());
                                                            }
                                                            Err(e) => eprintln!("\x1b[1;31m✗ Evaluation Error in
'{}':\x1b[0m {}", arg, e),
                                                        }
                                                    }
                                                    Err(e) => eprintln!("\x1b[1;31m✗ Parser Error in '{}':\x1b[0m {}",
arg, e),
                                                }
                                            }
                                            Err(e) => eprintln!("\x1b[1;31m✗ Lexer Error in '{}':\x1b[0m {}", arg, e),
                                        }
                                        let new_fps = logger.failpoints();
                                        if new_fps.len() > prev_fps_len {
                                            for fp in &new_fps[prev_fps_len..] {
                                                let fp_name = fp.failpoint_type.map(|t|
t.as_str()).unwrap_or("FAILPOINT");
                                                println!("  \x1b[1;33m[FAILPOINT: {}]\x1b[0m {}", fp_name, fp.message);
                                            }
                                        }
                                    }
                                    Err(e) => eprintln!("\x1b[1;31m✗ File read error:\x1b[0m {}", e),
                                }
                            }
                        }
                        other => {
                            eprintln!("\x1b[1;31mUnknown command:\x1b[0m '{}'. Type :help for commands.", other);
                        }
                    }
                    continue;
                }

                // OmniCode Expression / Statement Evaluation Pipeline
                let prev_fps_len = logger.failpoints().len();
                let mut lexer = Lexer::new(input_trimmed, logger.clone());
                match lexer.tokenize() {
                    Ok(tokens) => {
                        let mut parser = Parser::new(tokens, debugger.clone());
                        match parser.parse_program() {
                            Ok(prog) => {
                                let mut evaluator = Evaluator::new(env.clone(), parser.debugger);
                                match evaluator.eval(&ASTNode::Program(prog)) {
                                    Ok(val) => {
                                        env = evaluator.env;
                                        debugger = evaluator.debugger;
                                        if let RuntimeValue::State(s) = val {
                                            current_state = s;
                                        }
                                        println!("=> {}", val.format_colored());
                                    }
                                    Err(err) => {
                                        eprintln!("\x1b[1;31m✗ Runtime Error:\x1b[0m {}", err);
                                    }
                                }
                            }
```

```
                            Err(parse_err) => {
                                eprintln!("\x1b[1;31m✗ Syntax Error:\x1b[0m {}", parse_err);
                            }
                        }
                    }
                    Err(lex_err) => {
                        eprintln!("\x1b[1;31m✗ Lexical Error:\x1b[0m {}", lex_err);
                    }
                }

                // Check and print any failpoint alerts triggered during this line
                let all_fps = logger.failpoints();
                if all_fps.len() > prev_fps_len {
                    for fp in &all_fps[prev_fps_len..] {
                        let fp_name = fp.failpoint_type.map(|t| t.as_str()).unwrap_or("FAILPOINT");
                        println!("  \x1b[1;33m[FAILPOINT: {}]\x1b[0m {}", fp_name, fp.message);
                    }
                }

                if verbose {
                    println!("\x1b[0;37m(Active bindings: {}, snapshot depth: {})\x1b[0m", env.bindings().len(),
debugger.boundary_stack.len());
                }
            }
            Err(ReadlineError::Interrupted) => {
                println!("\x1b[1;33m^C (Interrupted. Type :exit or press Ctrl+D to quit)\x1b[0m");
                multiline_buffer.clear();
            }
            Err(ReadlineError::Eof) => {
                println!("\x1b[1;36mSealing OmniCode session. Goodbye.\x1b[0m");
                break;
            }
            Err(err) => {
                eprintln!("\x1b[1;31mTerminal Error:\x1b[0m {:?}", err);
                break;
            }
        }
    }

    if let Some(ref path) = history_file {
        let _ = rl.save_history(path);
    }

    Ok(())
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Session Invariants, Teardown Proofs, and Terminal Teardown )
// ============================================================================
// Terminal REPL engine verified and sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/cli/server.rs RS | 576 LINES | 1855 WORDS | 23.0 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( OmniCode GUI Server Identity, Zero-Dependency Architecture & REST Invariants )
// ============================================================================
//! OmniCode Local Web GUI Server & Terminal Gateway
//!
//! Provides a pure standard-library HTTP server (0 external crates) hosting the
//! local OmniCode Studio GUI and Terminal Interface:
//!   - Static file delivery: `gui/index.html`, `gui/style.css`, `gui/app.js`
//!   - REST endpoints:
//!     * `POST /api/prompt`    -> Executes AICortex neural-symbolic cognitive pipeline
//!     * `POST /api/terminal`  -> Executes CLI commands directly (`omnicode run/lint/demo`)
//!     * `GET  /api/manifest`  -> Returns canonical `manifest.omni` covenant anchor
//!     * `GET  /api/lexicon`   -> Returns offline Biblical world model & passage citations
//!     * `GET  /api/scrolls`   -> Returns episodic memory learning statistics

use std::fs;
use std::io::{Read, Write};
use std::net::{TcpListener, TcpStream};
use std::path::Path;
use std::sync::{Arc, Mutex};
use std::thread;

use crate::ai::{AICortex, AILearner, BiblicalLexicon};
use crate::core::ternary::QuadState;
use crate::diagnostics::hover::HoverEngine;
use crate::diagnostics::logger::DiagnosticLogger;
use crate::terminal::{create_shared_scheduler, SharedScheduler, TerminalSession};


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Server Configuration, HTTP Parser & Response Structures )
// ============================================================================

pub struct GuiServerConfig {
    pub host: String,
    pub port: u16,
    pub gui_dir: String,
}

impl Default for GuiServerConfig {
    fn default() -> Self {
        Self {
            host: "127.0.0.1".to_string(),
            port: 8080,
            gui_dir: "gui".to_string(),
        }
    }
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( TCP Listener Loop, HTTP Routing, and REST API Handlers )
// ============================================================================

/// Starts the OmniCode GUI HTTP server on the specified port.
pub fn run_gui_server(port: u16) -> Result<(), String> {
    let host = std::env::var("OMNICODE_HOST").unwrap_or_else(|_| "0.0.0.0".to_string());
    let listener = TcpListener::bind(format!("{host}:{port}"))
        .map_err(|e| format!("Failed to bind TCP listener to {host}:{port}: {e}"))?;

    println!("\x1b[1;36m================================================================================\x1b[0m");
    println!("\x1b[1;36m  OmniCode GUI Studio & Terminal Server Active                                    \x1b[0m");
    println!("\x1b[1;36m================================================================================\x1b[0m");
    println!("\x1b[1;32m✓ Web Studio URL:\x1b[0m http://localhost:{port} (listening on {host}:{port})");
    println!("\x1b[1;33m✓ Backend Mode:\x1b[0m 100% Offline | Grounded in Biblical Lexicon & Covenant Manifest");
    println!("\x1b[1;35m✓ Terminal Gateway:\x1b[0m Active (CLI commands executable via GUI)\n");
    println!("Press Ctrl+C to terminate server.\n");

    let lexicon = Arc::new(BiblicalLexicon::new());
    let scheduler = create_shared_scheduler();
    let session = Arc::new(Mutex::new(TerminalSession::new(Arc::clone(&scheduler))));
```

```

    for stream in listener.incoming() {
        match stream {
            Ok(stream) => {
                let lex_clone = Arc::clone(&lexicon);
                let sched_clone = Arc::clone(&scheduler);
                let sess_clone = Arc::clone(&session);
                thread::spawn(move || {
                    handle_connection(stream, lex_clone, sched_clone, sess_clone);
                });
            }
            Err(e) => {
                eprintln!("Connection failed: {e}");
            }
        }
    }

    Ok(())
}

fn handle_connection(
    mut stream: TcpStream,
    lexicon: Arc<BiblicalLexicon>,
    scheduler: SharedScheduler,
    session: Arc<Mutex<TerminalSession>>,
) {
    let mut buffer = [0u8; 16384];
    let bytes_read = match stream.read(&mut buffer) {
        Ok(n) if n > 0 => n,
        _ => return,
    };

    let request_str = String::from_utf8_lossy(&buffer[..bytes_read]);
    let mut lines = request_str.lines();
    let request_line = match lines.next() {
        Some(line) => line,
        std::option::Option::None => return,
    };

    let parts: Vec<&str> = request_line.split_whitespace().collect();
    if parts.len() < 2 {
        return;
    }

    let method = parts[0];
    let raw_path = parts[1];
    let path = raw_path.split('?').next().unwrap_or(raw_path);
    let is_head = method == "HEAD";
    let route_method = if is_head { "GET" } else { method };

    // Extract body if POST
    let body = if method == "POST" {
        if let Some(pos) = request_str.find("\r\n\r\n") {
            request_str[pos + 4..].to_string()
        } else if let Some(pos) = request_str.find("\n\n") {
            request_str[pos + 2..].to_string()
        } else {
            String::new()
        }
    } else {
        String::new()
    };

    // Route Request
    match (route_method, path) {
        ("GET", "/") | ("GET", "/studio") | ("GET", "/index.html") | ("GET", "/studio.html") => {
            serve_file(&mut stream, "apps/gui/studio.html", "text/html; charset=utf-8", is_head);
        }
        ("GET", "/terminal") | ("GET", "/terminal.html") => {
            serve_file(&mut stream, "apps/gui/terminal.html", "text/html; charset=utf-8", is_head);
        }
        ("GET", "/specs") | ("GET", "/specs.html") => {
            serve_file(&mut stream, "apps/gui/specs.html", "text/html; charset=utf-8", is_head);
        }
        ("GET", "/model") | ("GET", "/model.html") => {
            serve_file(&mut stream, "apps/gui/model.html", "text/html; charset=utf-8", is_head);
        }
        ("GET", "/lexicon") | ("GET", "/lexicon.html") => {
```

```
                serve_file(&mut stream, "apps/gui/lexicon.html", "text/html; charset=utf-8", is_head);
            }
            ("GET", "/scrolls") | ("GET", "/scrolls.html") => {
                serve_file(&mut stream, "apps/gui/scrolls.html", "text/html; charset=utf-8", is_head);
            }
            ("GET", "/cloud") | ("GET", "/cloud.html") => {
                serve_file(&mut stream, "apps/gui/cloud.html", "text/html; charset=utf-8", is_head);
            }
            ("GET", "/download") | ("GET", "/download.html") => {
                serve_file(&mut stream, "apps/gui/download.html", "text/html; charset=utf-8", is_head);
            }
            ("GET", "/terminal.js") => {
                serve_file(&mut stream, "apps/gui/terminal.js", "application/javascript; charset=utf-8", is_head);
            }
            ("GET", "/style.css") => {
                serve_file(&mut stream, "apps/gui/style.css", "text/css; charset=utf-8", is_head);
            }
            ("GET", "/app.js") => {
                serve_file(&mut stream, "apps/gui/app.js", "application/javascript; charset=utf-8", is_head);
            }
            ("GET", "/api/health") => {
                let json = r#"{"status":"healthy","engine":"OmniCode","version":"0.1.0","state":"+1:PROCEED"}"#;
                send_json_response(&mut stream, 200, json, is_head);
            }
            ("GET", "/api/download") => {
                let archive_path = "dist/omnicode-terminal-linux-x86_64.tar.gz";
                if let Ok(bytes) = fs::read(archive_path) {
                    let header = format!(
                        "HTTP/1.1 200 OK\r\nContent-Type: application/gzip\r\nContent-Disposition: attachment;
filename=\"omnicode-terminal-linux-x86_64.tar.gz\"\r\nContent-Length: {}\r\nAccess-Control-Allow-Origin: *\r\nConnection:
close\r\n\r\n",
                        bytes.len()
                    );
                    let _ = stream.write_all(header.as_bytes());
                    if !is_head {
                        let _ = stream.write_all(&bytes);
                    }
                } else {
                    let err = r#"{"error":"Distribution archive not found. Run scripts/package_native.sh to build."}"#;
                    send_json_response(&mut stream, 404, err, is_head);
                }
            }
            ("GET", "/api/scheduler") => {
                let json = scheduler.lock().unwrap().to_json();
                send_json_response(&mut stream, 200, &json, is_head);
            }
            ("POST", "/api/scheduler/spawn") => {
                let cmd_str = extract_json_field(&body, "command").unwrap_or_else(|| "omnicode demo".to_string());
                let name_str = extract_json_field(&body, "name").unwrap_or_else(|| "UserTask".to_string());
                let prio_str = extract_json_field(&body, "priority").unwrap_or_else(|| "NORMAL".to_string());
                let priority = match prio_str.to_uppercase().as_str() {
                    "HIGH" => crate::terminal::TaskPriority::High,
                    "LOW" => crate::terminal::TaskPriority::Low,
                    "BACKGROUND" => crate::terminal::TaskPriority::Background,
                    _ => crate::terminal::TaskPriority::Normal,
                };
                let mut sched = scheduler.lock().unwrap();
                let pid = sched.spawn(&name_str, &cmd_str, priority);
                let json = format!(r#"{{"pid":{},"status":"spawned"}}"#, pid);
                send_json_response(&mut stream, 200, &json, is_head);
            }
            ("GET", "/api/manifest") => {
                let manifest_content = fs::read_to_string("manifest.omni")
                    .unwrap_or_else(|_| "# Manifest not found".to_string());
                let json = format!(
                    r#"{{"manifest": "{}"}}"#,
                    escape_json_string(&manifest_content)
                );
                send_json_response(&mut stream, 200, &json, is_head);
            }
            ("GET", "/api/lexicon") => {
                let mut entries_json = Vec::new();
                for e in lexicon.all_entries() {
                    entries_json.push(format!(
                        r#"{{"term":"{}","domain":"{}","definition":"{}","kjv":"{}","web":"{}","state":"{}","guidance":"
{}"}}"#,
                        escape_json_string(e.term),
                        escape_json_string(e.domain.as_str()),
```

```
                        escape_json_string(e.definition),
                        escape_json_string(e.kjv_passage),
                        escape_json_string(e.web_passage),
                        escape_json_string(e.quad_state_resonance.macro_op()),
                        escape_json_string(e.moral_guidance)
                    ));
                }
                let json = format!(r#"{{"count":{},"entries":[{}]}}"#, entries_json.len(), entries_json.join(","));
                send_json_response(&mut stream, 200, &json, is_head);
            }
            ("GET", "/api/scrolls") => {
                let scroll_dir = if Path::new("storage/scrolls").exists() {
                    "storage/scrolls"
                } else if Path::new("runtime/scrolls").exists() {
                    "runtime/scrolls"
                } else {
                    "scrolls"
                };
                let learner = AILearner::new(scroll_dir);
                let summary = learner.digest_experiences().unwrap_or_default();
                let mut exp_json = Vec::new();
                for e in &summary.experiences {
                    exp_json.push(format!(
                        r#"{{"session_id":"{}","prompt":"{}","state":"{}","repairs":{},"failpoints":{},"scroll":"{}"}}"#,
                        escape_json_string(&e.session_id),
                        escape_json_string(&e.prompt),
                        escape_json_string(e.final_state.macro_op()),
                        e.repair_count,
                        e.failpoint_count,
                        escape_json_string(&e.scroll_path.to_string_lossy())
                    ));
                }
                let json = format!(
                    r#"{{"total":{},"proceed":{},"rollback":{},"await":{},"halt":{},"repairs_learned":{},"episodes":
[{}]}}"#,
                    summary.total_episodes, summary.proceed_count, summary.rollback_count, summary.await_count,
summary.halt_count,
                    summary.total_repairs_learned, exp_json.join(",")
                );
                send_json_response(&mut stream, 200, &json, is_head);
            }
            ("POST", "/api/prompt") => {
                let prompt = extract_json_field(&body, "prompt").unwrap_or_else(|| body.clone());
                let mut cortex = AICortex::new();
                match cortex.process_prompt(&prompt) {
                    Ok(res) => {
                        let mut repairs_json = Vec::new();
                        for r in &res.repairs_applied {
                            repairs_json.push(format!(
                                r#"{{"line":{},"kind":"{:?}","desc":"{}"}}"#,
                                r.line, r.kind, escape_json_string(&r.description)
                            ));
                        }
                        let json = format!(
                            r#"{{"session_id":"{}","prompt":"{}","state":"{}","response":"{}","omnicode":"{}","scroll":"
{}","log":"{}","repairs":[{}]}}"#,
                            escape_json_string(&res.session_id),
                            escape_json_string(&res.prompt),
                            escape_json_string(res.execution_state.macro_op()),
                            escape_json_string(&res.natural_response),
                            escape_json_string(&res.synthesized_omnicode),
                            escape_json_string(&res.scroll_path.to_string_lossy()),
                            escape_json_string(&res.log_path.to_string_lossy()),
                            repairs_json.join(",")
                        );
                        send_json_response(&mut stream, 200, &json, is_head);
                    }
                    Err(e) => {
                        let json = format!(r#"{{"error":"{}"}}"#, escape_json_string(&e));
                        send_json_response(&mut stream, 500, &json, is_head);
                    }
                }
            }
            ("GET", "/api/system") => {
                let json = r#"{"project":"OmniCode","organization":"CreativeWorkzStudio
LLC","version":"0.1.0","offline":true,"engine":"Rust Core 2024 (Balanced Ternary Quad-State)"}"#;
                send_json_response(&mut stream, 200, json, is_head);
            }
```

```
        ("GET", "/api/hover") => {
            let file = extract_query_param(raw_path, "file");
            let symbol = extract_query_param(raw_path, "symbol");
            let line = extract_query_param(raw_path, "line")
                .and_then(|s| s.parse::<usize>().ok())
                .unwrap_or(1);
            let col = extract_query_param(raw_path, "col")
                .and_then(|s| s.parse::<usize>().ok())
                .unwrap_or(1);

            let mut engine = HoverEngine::new(DiagnosticLogger::new(false));
            engine.index_workspace(".");

            let card = if let Some(ref sym) = symbol {
                engine.document_symbol(sym, file.as_deref())
            } else if let Some(ref f) = file {
                engine.hover_at(f, line, col)
            } else {
                None
            };

            match card {
                Some(c) => send_json_response(&mut stream, 200, &c.to_json(), is_head),
                None => {
                    let err = r#"{"symbol":"","found":false,"error":"No symbol found or target could not be
resolved"}"#;
                    send_json_response(&mut stream, 404, err, is_head);
                }
            }
        }
        ("POST", "/api/hover") => {
            let file = extract_json_field(&body, "file");
            let symbol = extract_json_field(&body, "symbol");
            let line = extract_json_field(&body, "line")
                .and_then(|s| s.parse::<usize>().ok())
                .unwrap_or(1);
            let col = extract_json_field(&body, "col")
                .and_then(|s| s.parse::<usize>().ok())
                .unwrap_or(1);

            let mut engine = HoverEngine::new(DiagnosticLogger::new(false));
            engine.index_workspace(".");

            let card = if let Some(ref sym) = symbol {
                engine.document_symbol(sym, file.as_deref())
            } else if let Some(ref f) = file {
                engine.hover_at(f, line, col)
            } else {
                None
            };

            match card {
                Some(c) => send_json_response(&mut stream, 200, &c.to_json(), is_head),
                None => {
                    let err = r#"{"symbol":"","found":false,"error":"No symbol found or target could not be
resolved"}"#;
                    send_json_response(&mut stream, 404, err, is_head);
                }
            }
        }
        ("POST", "/api/terminal") => {
            let cmd_str = extract_json_field(&body, "command").unwrap_or_else(|| body.clone());
            let clean_cmd = cmd_str.trim();

            let res = {
                let mut s = session.lock().unwrap();
                s.execute(clean_cmd)
            };

            let state_str = match res.state {
                QuadState::Proceed => "+1:PROCEED",
                QuadState::Await => "0:AWAIT",
                QuadState::Rollback => "-1:ROLLBACK",
                QuadState::Halt => "NULL:HALT",
            };

            let json = format!(
                r#"{{"stdout":"{}","state":"{}","duration_us":{}}}"#,
```

```
                escape_json_string(&res.stdout),
                state_str,
                res.duration_us
            );
            send_json_response(&mut stream, 200, &json, is_head);
        }
        ("OPTIONS", _) => {
            // Handle CORS preflight
            let response = "HTTP/1.1 204 No Content\r\n\
                Access-Control-Allow-Origin: *\r\n\
                Access-Control-Allow-Methods: GET, POST, OPTIONS, HEAD\r\n\
                Access-Control-Allow-Headers: Content-Type\r\n\r\n";
            stream.write_all(response.as_bytes()).ok();
        }
        _ => {
            let response = "HTTP/1.1 404 Not Found\r\nContent-Length: 9\r\n\r\nNot Found";
            stream.write_all(response.as_bytes()).ok();
        }
    }
}

fn serve_file(stream: &mut TcpStream, file_path: &str, content_type: &str, is_head: bool) {
    let resolved_path = if Path::new(file_path).exists() {
        file_path.to_string()
    } else if file_path.starts_with("gui/") {
        let alt = format!("apps/{file_path}");
        if Path::new(&alt).exists() {
            alt
        } else {
            file_path.to_string()
        }
    } else {
        file_path.to_string()
    };

    if !Path::new(&resolved_path).exists() {
        let not_found = format!(
            "HTTP/1.1 404 Not Found\r\nContent-Type: text/plain\r\nContent-Length: {}\r\n\r\nFile Not Found: {}",
            resolved_path.len() + 16, resolved_path
        );
        stream.write_all(not_found.as_bytes()).ok();
        return;
    }

    match fs::read(&resolved_path) {
        Ok(data) => {
            let header = format!(
                "HTTP/1.1 200 OK\r\n\
                Content-Type: {}\r\n\
                Content-Length: {}\r\n\
                Access-Control-Allow-Origin: *\r\n\
                Connection: close\r\n\r\n",
                content_type, data.len()
            );
            stream.write_all(header.as_bytes()).ok();
            if !is_head {
                stream.write_all(&data).ok();
            }
        }
        Err(e) => {
            let err_body = format!("Error reading file: {e}");
            let header = format!(
                "HTTP/1.1 500 Internal Server Error\r\nContent-Length: {}\r\n\r\n{}",
                err_body.len(), err_body
            );
            stream.write_all(header.as_bytes()).ok();
        }
    }
}

fn send_json_response(stream: &mut TcpStream, status_code: u16, json: &str, is_head: bool) {
    let status_line = match status_code {
        200 => "200 OK",
        400 => "400 Bad Request",
        404 => "404 Not Found",
        500 => "500 Internal Server Error",
        _ => "200 OK",
    };
```

```
    let header = format!(
        "HTTP/1.1 {}\r\n\
        Content-Type: application/json; charset=utf-8\r\n\
        Content-Length: {}\r\n\
        Access-Control-Allow-Origin: *\r\n\
        Connection: close\r\n\r\n",
        status_line, json.as_bytes().len()
    );
    stream.write_all(header.as_bytes()).ok();
    if !is_head {
        stream.write_all(json.as_bytes()).ok();
    }
}

fn escape_json_string(s: &str) -> String {
    let mut out = String::with_capacity(s.len() + 10);
    for c in s.chars() {
        match c {
            '"' => out.push_str("\\\""),
            '\\' => out.push_str("\\\\"),
            '\n' => out.push_str("\\n"),
            '\r' => out.push_str("\\r"),
            '\t' => out.push_str("\\t"),
            '\u{0008}' => out.push_str("\\b"),
            '\u{000C}' => out.push_str("\\f"),
            c if c.is_control() => {
                out.push_str(&format!("\\u{:04x}", c as u32));
            }
            c => out.push(c),
        }
    }
    out
}

fn extract_json_field(json: &str, field_name: &str) -> Option<String> {
    let key = format!("\"{}\":", field_name);
    let key_pos = json.find(&key)?;
    let remainder = json[key_pos + key.len()..].trim_start();

    if remainder.starts_with('"') {
        let mut result = String::new();
        let chars = remainder[1..].chars();
        let mut escaped = false;
        for c in chars {
            if escaped {
                match c {
                    '"' => result.push('"'),
                    '\\' => result.push('\\'),
                    'n' => result.push('\n'),
                    'r' => result.push('\r'),
                    't' => result.push('\t'),
                    other => result.push(other),
                }
                escaped = false;
            } else if c == '\\' {
                escaped = true;
            } else if c == '"' {
                return Some(result);
            } else {
                result.push(c);
            }
        }
        Some(result)
    } else {
        None
    }
}

fn extract_query_param(raw_path: &str, param: &str) -> Option<String> {
    let query_str = raw_path.split('?').nth(1)?;
    for pair in query_str.split('&') {
        let mut kv = pair.split('=');
        let k = kv.next()?;
        if k == param {
            let v = kv.next().unwrap_or("");
            return Some(v.replace("%20", " ").replace('+', " "));
        }
```

```
    }
    None
}

#[allow(dead_code)]
fn tokenize_args(input: &str) -> Vec<String> {
    let mut args = Vec::new();
    let mut current = String::new();
    let mut in_quotes = false;
    let mut quote_char = '"';

    for c in input.chars() {
        if in_quotes {
            if c == quote_char {
                in_quotes = false;
            } else {
                current.push(c);
            }
        } else if c == '"' || c == '\'' {
            in_quotes = true;
            quote_char = c;
        } else if c.is_whitespace() {
            if !current.is_empty() {
                args.push(current);
                current = String::new();
            }
        } else {
            current.push(c);
        }
    }
    if !current.is_empty() {
        args.push(current);
    }
    args
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Server Module Status & Invariant Verification )
// ============================================================================
// OmniCode GUI Server module sealed and active.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/cli/commands/assemble.rs RS 544 LINES 2088 WORDS 23.7 KB

```
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Machine Manifest, Scheduling Priority, Dependency Contract & Tripwire Registry )
// ================================================================================
//! MODULE:              omnicode::cli::commands::assemble
//! IDENTITY:            Physical Assembly & Process Tape Lowering Gear
//! NATURE:              Lowering Engine from High-Level SVO Tapes to Low-Level Memory Layouts & Silicon Word Streams
//! VERSION:             0.3.0
//! CREATED:             2026-10-11
//! LAST_SEALED:         2026-10-11
//! AUTHOR:              CreativeWorkzStudio LLC / ProfessorSeanEX
//! COVENANT:            Public Domain / MIT
//!
//! SCHEDULER:
//!   TRACK_PRIMARY:     <TRACK 1> : {Host Engine & Virtual Machine}
//!   TRACKS_TOUCHED:    [ <TRACK 0> : {Silicon & Bare-Metal Kernel}, <TRACK 1>, <TRACK 2> : {OmniCode Language &
Standard Library} ]
//!   PRIORITY:          TaskPriority::High
//!   CHANNEL:           Track::Kernel
//!
//! CONTRACT / WHO-WHAT-WHERE-WHY-HOW:
//!   WHO:               OmniCode Host Assembler Gear & Transpilation Lowering Driver
//!   WHAT:              Transduce high-level procedural `.tape` process ribbons into low-level declarative
//!                      `.omni` compiled memory reels, physical symbol tables, and 64-bit balanced-ternary
//!                      machine words ([`crate::core::word::Word64`]).
//!   WHERE:             Across Track 2 (`lexicon/`, `lib/kernel/`) or caller paths -> silicon memory partitions ->
stdout / disk.
//!   WHEN:              Invoked via CLI command `omnicode assemble <file.tape> [--output <PATH>] [--format
<listing|words|json>] [--verbose] [--dry-run]`
//!   WHY:               Eliminate ungrounded "python-ese" scripting drift by enforcing physical memory partition
//!                      allocation (`@ offset`), hardware register binding (`%self`, `%caller`, `%acc`, `%swarm`),
//!                      pronoun capability verification (`my`, `your`, `their`), and formal logic proofs (`! [
Invariant ]`).
//!   HOW:               Balanced Ternary (+1:PROCEED sealed assembly, -1:ROLLBACK trapped failpoint, 0:AWAIT dry-run).
//!
//! THE TRANSPILER TRIAD PHILOSOPHY:
//!   OmniCode operates across three isomorphic manifestations of code:
//!   1. `.omni` (Low-Level Compiled Form / Spatial Architecture):
//!      Static, declarative, frozen memory layouts, concrete denotations `[...]`, and invariant boundaries `<...>`.
//!   2. `.tape` (Interpreted Form / Temporal Execution Ribbon):
//!      Dynamic, sequential, time-bound rolls stepping across the tape head in real time.
//!   3. `Rust` (Silicon Host Substrate):
//!      Bedrock systems language defining opcodes, bitfields, and zero-cost native execution.
//!   This gear serves as the mechanical spindle turning Time (`.tape`) into Space (`.omni`) and Silicon (`Word64`).
//!
//! GAME DEV IDEOLOGY (THE STAGE INCEPTION & SIMULATION TICK):
//!   - Trigger Volumes & Collision Bounds: Every file resolution, syntax parse, partition slice, and seal
//!     is guarded by [`crate::cogs::tripwire::Tripwire`] collision volumes.
//!   - Flight Recorder: Step counts, instruction rolls, and physical offsets are logged to
[`crate::diagnostics::logger::DiagnosticLogger`].
//!   - Respawn / Healing: Any syntax anomaly or invariant violation produces `#remediation_ready` telemetry
//!     for autonomous recovery by the [`crate::diagnostics::restorator::Restorator`].
//!
//! TRIPWIRE REGISTRY:
//!   - `TRIPWIRE_INPUT_RESOLVE`      (Resolution of input tape file across tracks via `resolve_track_path`)
//!   - `TRIPWIRE_FILETYPE_VALIDATION`(Enforcement of `.tape` or `.omni` extensions)
//!   - `TRIPWIRE_SYNTAX_TOKENIZATION`(Lexical analysis and token classification verification)
//!   - `TRIPWIRE_MEMORY_BOUNDS`      (Partition capacity and offset bounds verification)
//!   - `TRIPWIRE_COMMIT_INVARIANT`   (Assertion of final affirmative `+1:PROCEED` seal)
//!   - `TRIPWIRE_OUTPUT_SERIALIZE`   (File system permissions and disk serialization integrity)
//!
//! NAVIGATION GUIDE:
//!   - Block 1 (Lines 1-80): Manifest, Transpiler Triad Philosophy, Game Dev Ideology, Tripwire Registry.
//!   - Block 2 (Lines 81-220): Opening Block, Configurations (`AssembleConfig`), Context (`AssembleContext`), Enums.
//!   - Block 3 (Lines 221-420): Body Block & Execution Pipeline (`run_assemble_with_config`,
`execute_assemble_pipeline`).
//!   - Block 4 (Lines 421+): Unit tests validating flag parsing, path resolution, and physical machine word output.

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Scoped Imports, Environment Setup, Data Structures & Interface Definitions )
// ================================================================================

// --------------------------------------------------------------------------------
```

```rust
// 2.1. Scoped Namespace Ingestion & Multi-Platform Imports
// ---------------------------------------------------------------------------
use std::fmt;
use std::fs;
use std::path::PathBuf;

use crate::cogs::tripwire::{Tripwire, TripwireError};
use crate::core::filetype::resolve_track_path;
use crate::diagnostics::debugger::OmniDebugger;
use crate::diagnostics::logger::{DiagnosticLogger, FailpointType};
use crate::runtime::assembler::{AssembledTape, OmniAssembler};

// ---------------------------------------------------------------------------
// 2.2. Output Serialization Format Enumeration
// ---------------------------------------------------------------------------
/// Output format for the assembled process tape or spatial reel.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, Default)]
pub enum AssembleFormat {
    /// Human-readable annotated machine listing (Default).
    #[default]
    Listing,
    /// Raw physical 64-bit machine words (Hex dump).
    Words,
    /// Structured JSON representation for tooling, hovercards, and IDEs.
    Json,
    /// Declarative 4-Block spatial architecture reel (`.omni`).
    Omni,
    /// Bare-metal x86 bootstrap assembly (`.asm`) for MilleniumOS.
    Asm,
    /// Canonical C module source code (`.c`).
    C,
    /// Canonical Lua script code (`.lua`).
    Lua,
}

impl AssembleFormat {
    /// Resolves a CLI format string into an `AssembleFormat` variant.
    pub fn from_str_name(s: &str) -> Option<Self> {
        match s.to_lowercase().as_str() {
            "listing" | "txt" => Some(Self::Listing),
            "words" | "hex" | "bin" => Some(Self::Words),
            "json" => Some(Self::Json),
            "omni" | "reel" => Some(Self::Omni),
            "asm" | "assembly" | "nasm" => Some(Self::Asm),
            "c" => Some(Self::C),
            "lua" => Some(Self::Lua),
            _ => None,
        }
    }

    /// Canonical lowercase format identifier string.
    pub fn as_str(&self) -> &'static str {
        match self {
            Self::Listing => "listing",
            Self::Words => "words",
            Self::Json => "json",
            Self::Omni => "omni",
            Self::Asm => "asm",
            Self::C => "c",
            Self::Lua => "lua",
        }
    }
}

impl fmt::Display for AssembleFormat {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{}", self.as_str())
    }
}

// ---------------------------------------------------------------------------
// 2.3. Assembly Configuration Specification
// ---------------------------------------------------------------------------
/// Immutable configuration parameters for the assembly run.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct AssembleConfig {
    /// Relative or absolute path to the target process tape (`.tape`) or reel (`.omni`).
    pub input_path: PathBuf,
```

```
    /// Optional output file path for the assembled listing or binary artifact.
    pub output_path: Option<PathBuf>,
    /// Selected output serialization format.
    pub format: AssembleFormat,
    /// Enables verbose step-by-step diagnostic logging.
    pub verbose: bool,
    /// Strict mode: treat warnings or unspecified offsets as hard failpoints.
    pub strict: bool,
    /// Dry run: simulate lowering without writing output artifacts.
    pub dry_run: bool,
}

impl AssembleConfig {
    /// Constructs a default configuration for a given input path.
    pub fn new(input_path: impl Into<PathBuf>) -> Self {
        Self {
            input_path: input_path.into(),
            output_path: None,
            format: AssembleFormat::default(),
            verbose: false,
            strict: false,
            dry_run: false,
        }
    }

    /// Parses CLI arguments into an `AssembleConfig`.
    pub fn from_args(args: &[String]) -> Result<Self, String> {
        let mut input_path: Option<PathBuf> = None;
        let mut output_path: Option<PathBuf> = None;
        let mut format = AssembleFormat::Listing;
        let mut verbose = false;
        let mut strict = false;
        let mut dry_run = false;

        let mut idx = 0;
        while idx < args.len() {
            let arg = args[idx].as_str();
            match arg {
                "-o" | "--output" => {
                    idx += 1;
                    if idx >= args.len() {
                        return Err("FAILPOINT [MissingOutputArg]: Flag '--output' requires a file path
parameter".into());
                    }
                    output_path = Some(PathBuf::from(&args[idx]));
                }
                "-f" | "--format" => {
                    idx += 1;
                    if idx >= args.len() {
                        return Err("FAILPOINT [MissingFormatArg]: Flag '--format' requires a format name
(listing|words|json)".into());
                    }
                    let fmt_str = &args[idx];
                    format = AssembleFormat::from_str_name(fmt_str)
                        .ok_or_else(|| format!("FAILPOINT [InvalidFormat]: Unknown format '{fmt_str}' (expected:
listing, words, json)"))?;
                }
                "-v" | "--verbose" => {
                    verbose = true;
                }
                "-s" | "--strict" => {
                    strict = true;
                }
                "-n" | "--dry-run" => {
                    dry_run = true;
                }
                other => {
                    if !other.starts_with('-') && input_path.is_none() {
                        input_path = Some(PathBuf::from(other));
                    }
                }
            }
            idx += 1;
        }

        let input_path = input_path.ok_or_else(|| {
            "FAILPOINT [MissingInputPath]: No target tape file specified. Usage: omnicode assemble <file.tape> [--output
<PATH>] [--format <listing|words|json>]".to_string()
```

```
        })?;

        Ok(Self {
            input_path,
            output_path,
            format,
            verbose,
            strict,
            dry_run,
        })
    }

    /// Performs pre-flight verification, resolves paths across tracks, and constructs an `AssembleContext`.
    pub fn open(&self, logger: &DiagnosticLogger) -> Result<AssembleContext, TripwireError> {
        // Tripwire 1: Input file resolution across tracks
        let resolved = resolve_track_path(&self.input_path);
        Tripwire::assert_path_exists(
            logger,
            "TRIPWIRE_INPUT_RESOLVE",
            &resolved,
            "Target process tape or reel across Track roots",
        )?;

        // Tripwire 2: File extension check
        let ext = resolved.extension().and_then(|e| e.to_str()).unwrap_or("");
        if ext != "tape" && ext != "omni" {
            let msg = format!("File '{}' has unsupported extension '.{}' (expected .tape or .omni)", resolved.display(),
ext);
            return Err(Tripwire::trip(
                logger,
                "TRIPWIRE_FILETYPE_VALIDATION",
                FailpointType::SyntaxAnomaly,
                msg,
            ));
        }

        let debugger = OmniDebugger::new(logger.clone(), 100);
        let assembler = OmniAssembler::new(logger.clone(), debugger);

        Ok(AssembleContext {
            resolved_input: resolved,
            config: self.clone(),
            assembler,
            logger: logger.clone(),
        })
    }
}

// ---------------------------------------------------------------------------
// 2.4. Assembly Execution Context
// ---------------------------------------------------------------------------
/// Active runtime context holding resolved paths, tripwires, and the physical assembler.
#[derive(Debug)]
pub struct AssembleContext {
    /// Fully qualified path resolved across track roots.
    pub resolved_input: PathBuf,
    /// Immutable caller configuration.
    pub config: AssembleConfig,
    /// Physical assembler engine.
    pub assembler: OmniAssembler,
    /// Attached diagnostic logger for tripwires and telemetry.
    pub logger: DiagnosticLogger,
}

// ---------------------------------------------------------------------------
// 2.5. Initial Truth States & Equilibrium Defaults
// ---------------------------------------------------------------------------
/// Default output serialization format (human-readable annotated listing).
pub const DEFAULT_ASSEMBLE_FORMAT: AssembleFormat = AssembleFormat::Listing;
/// Baseline cell capacity reserved for unspecified memory partitions.
pub const DEFAULT_PARTITION_CELL_CAPACITY: usize = 64;
/// Maximum delimiter boundary nesting depth for structural invariant assertions.
pub const DEFAULT_ASSEMBLER_DEBUGGER_DEPTH: usize = 100;


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Assembly Execution Pipeline, Output Serialization & Tripwire Sealing )
```

```rust
// ============================================================================

// ----------------------------------------------------------------------------
// 3.1. Public CLI Dispatchers
// ----------------------------------------------------------------------------
/// Legacy single-path entrypoint maintaining backwards compatibility with CLI dispatchers.
pub fn run_assemble(path: &str) {
    let config = AssembleConfig::new(path);
    if let Err(e) = run_assemble_with_config(config) {
        eprintln!("\x1b[1;31mAssembly Failed:\x1b[0m {e}");
        std::process::exit(1);
    }
}

/// Executes the modular assembly pipeline with full configuration, tripwires, and formatting.
pub fn run_assemble_with_config(config: AssembleConfig) -> Result<AssembledTape, TripwireError> {
    let logger = DiagnosticLogger::new(config.verbose);
    println!("\x1b[1;36mAssembling OmniCode Process Tape:\x1b[0m {}", config.input_path.display());

    let mut ctx = config.open(&logger)?;
    let assembled = execute_assemble_pipeline(&mut ctx)?;

    // Handle output formatting and serialization
    serialize_assembled_output(&assembled, &ctx.config, &logger)?;

    println!(
        "\x1b[1;32m✓ Assembly Successful: Machine Tape Sealed in {}\x1b[0m ({} rolls, {} symbols, {} words)",
        assembled.commit_status.macro_op(),
        assembled.instructions.len(),
        assembled.symbols.len(),
        assembled.instructions.len()
    );

    Ok(assembled)
}

// ----------------------------------------------------------------------------
// 3.2. Core Lowering Pipeline
// ----------------------------------------------------------------------------
/// Invokes the lowering engine within the tripwire collision hull.
pub fn execute_assemble_pipeline(ctx: &mut AssembleContext) -> Result<AssembledTape, TripwireError> {
    let tape_path = ctx.resolved_input.clone();

    let assembled = match ctx.assembler.assemble_file(&tape_path) {
        Ok(t) => t,
        Err(e) => {
            return Err(Tripwire::trip(
                &ctx.logger,
                "TRIPWIRE_SYNTAX_TOKENIZATION",
                FailpointType::SyntaxAnomaly,
                format!("Tape assembly lowered into failpoint: {e}"),
            ));
        }
    };

    // Assert final commit status invariant
    Tripwire::assert_quad_state_proceed(
        &ctx.logger,
        "TRIPWIRE_COMMIT_INVARIANT",
        assembled.commit_status,
        "Process tape must conclude with 'commit_status [ PROCEED ]'",
    )?;

    Ok(assembled)
}

// ----------------------------------------------------------------------------
// 3.3. Output Serialization & Formatting
// ----------------------------------------------------------------------------
/// Renders or writes the assembled tape to disk or stdout according to the chosen format.
///
/// Consumes the universal serialization methods ([`AssembledTape::format_listing`],
/// [`AssembledTape::format_words`], [`AssembledTape::format_json`], [`AssembledTape::format_omni`])
/// extracted from the runtime lowering engine.
pub fn serialize_assembled_output(
    assembled: &AssembledTape,
    config: &AssembleConfig,
    logger: &DiagnosticLogger,
```

```
) -> Result<(), TripwireError> {
    let rendered = match config.format {
        AssembleFormat::Listing => assembled.format_listing(),
        AssembleFormat::Words => assembled.format_words(),
        AssembleFormat::Json => assembled.format_json(),
        AssembleFormat::Omni => assembled.format_omni(),
        AssembleFormat::Asm => assembled.format_asm(),
        AssembleFormat::C => assembled.format_c(),
        AssembleFormat::Lua => assembled.format_lua(),
    };

    if let Some(ref out_path) = config.output_path {
        if !config.dry_run {
            if let Err(e) = fs::write(out_path, &rendered) {
                return Err(Tripwire::trip(
                    logger,
                    "TRIPWIRE_OUTPUT_SERIALIZE",
                    FailpointType::SyntaxAnomaly,
                    format!("Failed to write assembled output to '{}': {e}", out_path.display()),
                ));
            }
            println!("\x1b[1;32m✓ Output Artifact Written:\x1b[0m {}", out_path.display());
        } else {
            println!("\x1b[1;33m[Dry Run] Output serialization skipped for:\x1b[0m {}", out_path.display());
        }
    } else {
        println!("{rendered}");
    }

    Ok(())
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Baton Pass Contract, Unit Tests & Verification Proof )
// ============================================================================

// ----------------------------------------------------------------------------
// 4.1. The Baton Pass Contract
// ----------------------------------------------------------------------------
// BATON PASS CONTRACT:
// 1. `AssembleConfig` parses and validates user CLI flags without side effects.
// 2. `open()` verifies that the process tape exists across Track roots and establishes tripwire bounds.
// 3. `execute_assemble_pipeline()` drives lowering into `OmniAssembler`, verifying `MachineOp`, registers, and
`Word64`.
// 4. `serialize_assembled_output()` formats or writes the output, yielding a sealed `AssembledTape`.
// 5. Downstream dispatchers in `main.rs` receive verified artifacts ready for scheduling or execution.

#[cfg(test)]
mod tests {
    use super::*;
    use crate::core::ternary::QuadState;

    #[test]
    fn test_assemble_config_default_and_new() {
        let config = AssembleConfig::new("lib/kernel/boot.tape");
        assert_eq!(config.input_path, PathBuf::from("lib/kernel/boot.tape"));
        assert_eq!(config.format, AssembleFormat::Listing);
        assert!(!config.verbose);
        assert!(!config.strict);
        assert!(!config.dry_run);
        assert_eq!(config.output_path, None);
    }

    #[test]
    fn test_assemble_config_argument_parsing() {
        let args = vec![
            "lexicon/operators/polarity.tape".to_string(),
            "--output".to_string(),
            "/tmp/polarity.asm".to_string(),
            "--format".to_string(),
            "words".to_string(),
            "--verbose".to_string(),
            "--dry-run".to_string(),
        ];
        let config = AssembleConfig::from_args(&args).expect("Parsing CLI args");
        assert_eq!(config.input_path, PathBuf::from("lexicon/operators/polarity.tape"));
```

```
        assert_eq!(config.output_path, Some(PathBuf::from("/tmp/polarity.asm")));
        assert_eq!(config.format, AssembleFormat::Words);
        assert!(config.verbose);
        assert!(config.dry_run);
    }

    #[test]
    fn test_assemble_config_missing_input_fails_gracefully() {
        let args: Vec<String> = vec!["--verbose".to_string()];
        let err = AssembleConfig::from_args(&args).unwrap_err();
        assert!(err.contains("MissingInputPath"));
    }

    #[test]
    fn test_assemble_format_conversions() {
        assert_eq!(AssembleFormat::from_str_name("listing"), Some(AssembleFormat::Listing));
        assert_eq!(AssembleFormat::from_str_name("words"), Some(AssembleFormat::Words));
        assert_eq!(AssembleFormat::from_str_name("hex"), Some(AssembleFormat::Words));
        assert_eq!(AssembleFormat::from_str_name("json"), Some(AssembleFormat::Json));
        assert_eq!(AssembleFormat::from_str_name("omni"), Some(AssembleFormat::Omni));
        assert_eq!(AssembleFormat::from_str_name("reel"), Some(AssembleFormat::Omni));
        assert_eq!(AssembleFormat::from_str_name("unknown"), None);
    }

    #[test]
    fn test_assemble_pipeline_lowers_boot_tape_and_transpiles_to_omni() {
        let tape_path = resolve_track_path("lib/kernel/boot.tape");
        if tape_path.exists() {
            let config = AssembleConfig {
                input_path: tape_path,
                output_path: None,
                format: AssembleFormat::Omni,
                verbose: false,
                strict: false,
                dry_run: true,
            };
            let logger = DiagnosticLogger::new(false);
            let mut ctx = config.open(&logger).expect("Opening assemble context");
            let assembled = execute_assemble_pipeline(&mut ctx).expect("Executing assemble pipeline");

            let omni_output = assembled.format_omni();
            assert!(omni_output.contains("Transpiled Spatial Reel"));
            assert!(omni_output.contains("BLOCK 1: METADATA BLOCK"));
            assert!(omni_output.contains("BLOCK 2: OPENING BLOCK"));
            assert!(omni_output.contains("BLOCK 3: BODY BLOCK"));
            assert!(omni_output.contains("BLOCK 4: CLOSING BLOCK"));
        }
    }

    #[test]
    fn test_assemble_pipeline_lowers_boot_tape_and_emits_words() {
        let tape_path = resolve_track_path("lib/kernel/boot.tape");
        if tape_path.exists() {
            let config = AssembleConfig {
                input_path: tape_path,
                output_path: None,
                format: AssembleFormat::Words,
                verbose: false,
                strict: false,
                dry_run: true,
            };
            let logger = DiagnosticLogger::new(false);
            let mut ctx = config.open(&logger).expect("Opening assemble context");
            let assembled = execute_assemble_pipeline(&mut ctx).expect("Executing assemble pipeline");

            assert_eq!(assembled.commit_status, QuadState::Proceed);
            assert!(!assembled.instructions.is_empty());

            // Assert that instructions carry real physical Word64 machine words
            for inst in &assembled.instructions {
                assert_ne!(inst.word.raw, 0xFFFFFFFF_FFFFFFFF);
            }
        }
    }

    #[test]
    fn test_assemble_preflight_missing_file_trips() {
        let config = AssembleConfig::new("nonexistent_directory/missing.tape");
```

```
        let logger = DiagnosticLogger::new(false);
        let err = config.open(&logger).unwrap_err();
        assert!(err.to_string().contains("TRIPWIRE_INPUT_RESOLVE"));
    }

    #[test]
    fn test_assemble_preflight_invalid_extension_trips() {
        let config = AssembleConfig::new("Cargo.toml");
        let logger = DiagnosticLogger::new(false);
        let err = config.open(&logger).unwrap_err();
        assert!(err.to_string().contains("TRIPWIRE_FILETYPE_VALIDATION"));
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/cli/commands/boot.rs RS 782 LINES 3860 WORDS 39.1 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Machine Manifest, Scheduling Priority, Dependency Contract & Tripwire Registry )
// ============================================================================
//! MODULE:              omnicode::cli::commands::boot
//! IDENTITY:            Bare-Metal Multi-Architecture Boot Gear
//! NATURE:              Cold-Start Machine Inception & Silicon Hardware Simulation Driver
//! VERSION:             0.3.0
//! CREATED:             2026-10-11
//! LAST_SEALED:         2026-10-11
//! AUTHOR:              CreativeWorkzStudio LLC / ProfessorSeanEX
//! COVENANT:            Public Domain / MIT
//!
//! SCHEDULER:
//!   TRACK_PRIMARY:     <TRACK 1> : {Host Engine & Virtual Machine}
//!   TRACKS_TOUCHED:    [ <TRACK 0> : {Silicon & Bare-Metal Kernel}, <TRACK 1>, <TRACK 2> : {Engine Runtime & SVO} ]
//!   PRIORITY:          TaskPriority::High
//!   CHANNEL:           Track::Kernel
//!
//! CONTRACT / WHO-WHAT-WHERE-WHY-HOW:
//!   WHO:               OmniCode Host Orchestrator Engine & Machine Inception Driver
//!   WHAT:              Assemble bare-metal operating system kernels in Track 0 and launch
//!                      the QEMU hypervisor across BIOS, UEFI, and Multiboot architectures.
//!   WHERE:             `<TRACK 0>/kernel` tree -> host QEMU hypervisor -> COM1 / VGA silicon endpoints
//!   WHEN:              Invoked via CLI command `omnicode boot [--bios|--uefi|--multiboot] [--gui] [--ovmf <PATH>]`
//!   WHY:               Verify silicon-level self-sufficiency, hardware invariants, and balanced-ternary machine
bootstrap.
//!   HOW:               Balanced Ternary (+1:PROCEED clean exit, -1:ROLLBACK trapped error, 0:AWAIT execution running)
//!
//! THE "IN-WAITING" MULTI-OS SYMBIOTIC MESH:
//!   OmniCode is fundamentally an autonomous bare-metal Operating System and Balanced-Ternary computing
//!   architecture. However, until custom silicon machines are ubiquitous, OmniCode incubates comfortably
//!   inside existing host operating systems (Linux, macOS, Windows).
//!   This gear delegates host environment discovery to [`crate::cogs::platform::HostPlatform`], ensuring:
//!   - Dynamic selection of hypervisor binaries (`qemu-system-x86_64` vs `qemu-system-x86_64.exe`).
//!   - Dynamic discovery of UEFI OVMF firmware blobs across Debian, Arch, Fedora, NixOS, macOS Homebrew, and Windows.
//!   - Dynamic probing of display servers to avoid graphical window crashes in headless containers or SSH sessions.
//!
//! GAME DEV IDEOLOGY (THE STAGE INCEPTION & SIMULATION TICK):
//!   In modern game engine architecture (e.g., Unreal, Unity, Godot):
//!   - Level Loading / Inception: Setting up world geometry, shaders, and player spawn points before rendering.
//!   - Collision Volumes & Triggers: Spatial boundaries that fire events when crossed or violated.
//!   - Flight Recorder & Telemetry HUD: Frame-by-frame debug logging with nanosecond timestamps.
//!   - Respawn / Immune Recovery: Resetting player coordinates or healing scene state after a collision failpoint.
//!   In OmniCode Bare-Metal Boot:
//!   - Stage Inception: `execute_kernel_pipeline` compiles the bare-metal kernel and binds MMIO registers.
//!   - Tripwires: Invariant collision sensors (`Tripwire::assert_*`) wrapping every file access, compiler invocation,
//!     and hypervisor process launch.
//!   - Flight Recorder: [`DiagnosticLogger`] and [`OmniDebugger`] track step counts, register cognitive notes,
//!     and serialize episodic scrolls.
//!   - Respawn / Healing: Any tripped invariant produces a [`FixSuggestion`] and `#remediation_ready` telemetry
//!     for autonomous recovery by the [`crate::diagnostics::restorator::Restorator`].
//!
//! TOUCHES / FILESYSTEM FOOTPRINT:
//!   - EXECUTABLE:      `<TRACK 0>/kernel/build_all_kernels.sh`
//!   - ARTIFACT:        `<TRACK 0>/kernel/omnicode_bios.img` (512-byte MBR boot sector)
//!   - ARTIFACT:        `<TRACK 0>/kernel/esp/EFI/BOOT/BOOTX64.EFI` (UEFI PE32+ application)
//!   - ARTIFACT:        `<TRACK 0>/kernel/omnicode_multiboot.elf` (ELF multiboot kernel)
//!   - FIRMWARE:        OVMF UEFI Code image (probed from host system via `HostPlatform`)
//!
//! TOUCHES / SILICON HARDWARE & MMIO ADDRESSES:
//!   - COM1 Serial Port (I/O Port `0x3F8`): Serial console telemetry output at 115200 baud 8N1.
//!   - VGA Text Buffer Base (Memory Address `0xB8000`): 80x25 monochrome/color text screen buffer.
//!   - Real Mode Reset Vector (Physical Memory `0x7C00`): IBM PC MBR bootstrap entrypoint.
//!
//! REQUIRES / HOST DEPENDENCIES:
//!   - `qemu-system-x86_64` (virtual machine emulator & silicon hypervisor)
//!   - `bash` (bare-metal kernel build pipeline driver)
//!   - `nasm` (16-bit real mode and multiboot x86 assembler)
//!   - `clang` + `lld-link` (x86_64-unknown-windows UEFI PE32+ compiler & linker)
//!   - `ld` (elf_i386 multiboot linker)
//!
//! TRIPWIRE REGISTRY:
//!   - `TRIPWIRE_KERNEL_BUILD_SCRIPT` (Track 0 build script accessibility and readability)
```

```
//!   - `TRIPWIRE_KERNEL_COMPILATION`  (Kernel compiler pipeline exit status code)
//!   - `TRIPWIRE_IMAGE_RESOLVE`       (Generated boot artifact existence after assembly)
//!   - `TRIPWIRE_HYPERVISOR_PATH`     (Availability of hypervisor binary on host PATH)
//!   - `TRIPWIRE_FIRMWARE_RESOLVE`    (Resolution of UEFI OVMF firmware blob)
//!   - `TRIPWIRE_DISPLAY_SERVER`      (Assertion of host display server when GUI mode is active)
//!   - `TRIPWIRE_QEMU_LAUNCH`         (Hypervisor execution status code and crash traps)
//!
//! NAVIGATION GUIDE (HOW TO NAVIGATE AND EXTEND THIS MODULE AS THE CODE GROWS):
//!   - Block 1 (Lines 1-95): Manifest, Multi-OS In-Waiting Philosophy, Game Dev Analogy, Tripwire Registry.
//!   - Block 2 (Lines 96-260): Opening Block, Domain Typology, Configuration & Equilibrium Constants.
//!     * To add a new boot architecture (e.g. RISC-V `RiscV64`): add a variant to `BootArch` in Section 2.2.
//!     * To inspect how options are parsed from CLI tokens: examine `BootConfig::from_args` in Section 2.3.
//!     * To view pre-flight invariant certification: examine `BootConfig::open` in Section 2.3.
//!     * To check silicon MMIO addresses: review equilibrium constants in Section 2.4.
//!   - Block 3 (Lines 261-530): Body Block & Bare-Metal Simulation Tick.
//!     * 3.1: Public dispatchers (`run_boot`, `run_boot_with_config`).
//!     * 3.2: Kernel compilation pipeline and build script execution (`execute_kernel_pipeline`).
//!     * 3.3: Silicon MMIO & hypervisor command synthesis (`synthesize_hypervisor_command`).
//!     * 3.4: Machine execution loop and telemetry sealing (`execute_boot_pipeline`, `execute_boot_machine`).
//!   - Block 4 (Lines 531+): Unit tests validating architecture names, argument parsing, command synthesis, and PATH
lookups.

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Scoped Imports, Environment Setup, Data Structures & Interface Definitions )
// ============================================================================

// ----------------------------------------------------------------------------
// 2.1. Scoped Namespace Ingestion & Multi-Platform Imports
// ----------------------------------------------------------------------------
use std::path::{Path, PathBuf};
use std::process::Command;

use crate::cogs::platform::HostPlatform;
use crate::cogs::tripwire::{Tripwire, TripwireError};
use crate::core::filetype::resolve_track_path;
use crate::core::ternary::QuadState;
use crate::diagnostics::debugger::OmniDebugger;
use crate::diagnostics::logger::DiagnosticLogger;

// ----------------------------------------------------------------------------
// 2.2. Domain Typology & Architectural Target Definitions
// ----------------------------------------------------------------------------
/// Supported bare-metal boot target architectures.
///
/// Each architecture represents a discrete physical execution plane into which
/// OmniCode can materialize directly from cold power-on.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum BootArch {
    /// 16-Bit Real Mode MBR (512-byte sector loaded at physical address `0x7C00`).
    /// Minimalist, self-contained hardware bootstrap with BIOS INT 10h / INT 13h / INT 16h.
    Bios,

    /// 64-Bit Native UEFI Long Mode (PE32+ EFI Application loaded by firmware boot manager).
    /// Modern hardware standard with graphical GOP framebuffer and EFI System Table.
    Uefi,

    /// 32-Bit Direct Kernel Handshake via Multiboot Specification (ELF executable).
    /// Used for direct hypervisor injection (QEMU `-kernel`, GRUB, Xen, KVM).
    Multiboot,
}

impl BootArch {
    /// Human and machine readable description of the architecture target.
    pub fn display_name(&self) -> &'static str {
        match self {
            BootArch::Bios => "Legacy BIOS MBR (16-Bit Real Mode -> 512-Byte Boot Sector)",
            BootArch::Uefi => "Modern UEFI 64-Bit Long Mode (PE32+ EFI Application via OVMF)",
            BootArch::Multiboot => "Direct Kernel Multiboot/GRUB (ELF Hypervisor Handshake)",
        }
    }

    /// Expected target artifact path relative to `<TRACK 0>/kernel`.
    pub fn target_artifact_relpath(&self) -> &'static str {
        match self {
            BootArch::Bios => "kernel/images/omnicode_bios.img",
            BootArch::Uefi => "kernel/images/esp",
            BootArch::Multiboot => "kernel/images/omnicode_multiboot.elf",
```

```
        }
    }
}

// ---------------------------------------------------------------------------
// 2.3. Boot Configuration Specification & Options Matrix
// ---------------------------------------------------------------------------
/// Concrete execution configuration for the bare-metal machine inception.
#[derive(Debug, Clone)]
pub struct BootConfig {
    /// Target execution architecture (BIOS, UEFI, or Multiboot).
    pub arch: BootArch,
    /// Whether to open an emulated graphical window or run headless over COM1 serial.
    pub graphical: bool,
    /// Host operating system plane under which execution is launched.
    pub host_platform: HostPlatform,
    /// Optional custom override path for UEFI OVMF firmware image.
    pub custom_ovmf: Option<PathBuf>,
}

/// Fully resolved, verified execution context prepared by the Opening Block.
///
/// Acts as the immutable "Setup Frame" handed to the Body Block's simulation tick.
/// Contains certified file paths, armed diagnostic sensors, and pre-resolved host binaries.
#[derive(Debug, Clone)]
pub struct BootContext {
    /// Initial boot configuration parameters.
    pub config: BootConfig,
    /// Attached telemetry logger for recording events and failpoints.
    pub logger: DiagnosticLogger,
    /// Interactive debugger tracking step count and QuadState transitions.
    pub debugger: OmniDebugger,
    /// Verified absolute path to `<TRACK 0>/kernel/build_all_kernels.sh`.
    pub build_script: PathBuf,
    /// Target boot artifact (MBR image, ESP directory, or ELF executable).
    pub boot_artifact: PathBuf,
    /// Pre-resolved UEFI OVMF firmware blob (if UEFI mode is selected).
    pub ovmf_firmware: Option<PathBuf>,
    /// Hypervisor executable name appropriate for the host platform.
    pub hypervisor_binary: &'static str,
}

impl BootConfig {
    /// Constructs a standard boot configuration with platform auto-detection.
    pub fn new(arch: BootArch, graphical: bool) -> Self {
        Self {
            arch,
            graphical,
            host_platform: HostPlatform::current(),
            custom_ovmf: None,
        }
    }

    /// Parses raw command-line tokens into a validated [`BootConfig`].
    pub fn from_args(args: &[String]) -> Self {
        let arch = if args.iter().any(|a| a == "--uefi" || a == "-u") {
            BootArch::Uefi
        } else if args.iter().any(|a| a == "--multiboot" || a == "-m" || a == "--direct") {
            BootArch::Multiboot
        } else {
            BootArch::Bios
        };

        let graphical = args.iter().any(|a| a == "--gui" || a == "-g" || a == "--window");

        let custom_ovmf = args.iter().position(|a| a == "--ovmf" || a == "-b").and_then(|idx| {
            args.get(idx + 1).map(PathBuf::from)
        });

        Self {
            arch,
            graphical,
            host_platform: HostPlatform::current(),
            custom_ovmf,
        }
    }

    /// Prepares and certifies the execution context before the Body touches the metal.
```

```
    ///
    /// # Precondition Tripwires & Environmental Probes:
    /// - Asserts that the Track 0 kernel build script is reachable on disk.
    /// - Asserts that the target boot artifact path resolves.
    /// - Pre-resolves firmware candidate and platform hypervisor binary.
    /// - Inspects host display server if graphical GUI window was requested.
    /// - Initializes an attached [`OmniDebugger`] to monitor step invariants.
    pub fn open(&self, logger: DiagnosticLogger) -> Result<BootContext, TripwireError> {
        let debugger = OmniDebugger::new(logger.clone(), 10);
        let build_script_path = resolve_track_path("kernel/build_all_kernels.sh");
        let build_script = Tripwire::assert_path_exists(
            &logger,
            "BOOT",
            build_script_path,
            "Track 0 kernel build pipeline script",
        )?;

        let boot_artifact = resolve_track_path(self.arch.target_artifact_relpath());

        // Probe host display environment if GUI window requested
        if self.graphical && !self.host_platform.has_display_server() {
            logger.record_ai_observation(
                "GUI_DISPLAY_HEADLESS",
                "Graphical window requested but host display server not detected",
                "Host environment lacks active DISPLAY or WAYLAND_DISPLAY session; window spawn may fail",
                ["gui", "display", "headless", "observation"].as_slice(),
                None,
            );
        }

        let ovmf_firmware = if self.arch == BootArch::Uefi {
            if let Some(custom) = &self.custom_ovmf {
                Some(Tripwire::assert_path_exists(&logger, "BOOT", custom, "Custom OVMF firmware")?)
            } else {
                self.host_platform.find_ovmf_firmware()
            }
        } else {
            None
        };

        let hypervisor_binary = self.host_platform.hypervisor_binary();

        Ok(BootContext {
            config: self.clone(),
            logger,
            debugger,
            build_script,
            boot_artifact,
            ovmf_firmware,
            hypervisor_binary,
        })
    }
}

// ---------------------------------------------------------------------------
// 2.4. Initial Truth States & Equilibrium Defaults
// ---------------------------------------------------------------------------
/// Canonical default boot architecture for minimal real-mode baseline.
pub const DEFAULT_BOOT_ARCH: BootArch = BootArch::Bios;

/// Default display mode: headless COM1 serial terminal output for pure dataflow.
pub const DEFAULT_GRAPHICAL_MODE: bool = false;

/// Real-mode bootstrap reset vector base in silicon RAM (`0x7C00`).
pub const REAL_MODE_BOOT_VECTOR: u64 = 0x7C00;

/// Standard VGA 80x25 text framebuffer memory-mapped base address (`0xB8000`).
pub const VGA_TEXT_FRAMEBUFFER_BASE: u64 = 0xB8000;

/// Standard IBM PC COM1 serial communications port I/O base address (`0x3F8`).
pub const COM1_SERIAL_IO_BASE: u16 = 0x3F8;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Kernel Compilation, Inception Tripwires & QEMU Execution Engine )
// ============================================================================

// ---------------------------------------------------------------------------
```

```rust
// 3.1. Public Dispatcher & Context Resolution
// ---------------------------------------------------------------------------
/// Builds the target kernels in Track 0 and launches the virtual machine in QEMU.
///
/// # Game Dev Loop & Cybernetic Architecture:
/// Functions as the primary command-line engine driver. Resolves ambient hardware
/// parameters, constructs the diagnostic logger and debugger sensors, and invokes
/// the self-healing execution pipeline.
///
/// # Returns
/// - `Ok(QuadState::Proceed)`: Hypervisor session completed cleanly without fatal invariant breach.
/// - `Err(TripwireError)`: Precondition, compilation, or hypervisor tripwire tripped with `-1:ROLLBACK`.
pub fn run_boot(arch: BootArch, graphical: bool) -> Result<QuadState, TripwireError> {
    let config = BootConfig::new(arch, graphical);
    run_boot_with_config(config)
}

/// Executes boot inception under a customized [`BootConfig`].
pub fn run_boot_with_config(config: BootConfig) -> Result<QuadState, TripwireError> {
    let logger = DiagnosticLogger::new(false);
    let ctx = match config.open(logger) {
        Ok(c) => c,
        Err(e) => {
            eprintln!("\x1b[1;31m[!] Precondition Tripwire Tripped during BootContext Open:\x1b[0m {e}");
            return Err(e);
        }
    };

    execute_boot_pipeline(ctx)
}

// ---------------------------------------------------------------------------
// 3.2. Kernel Compilation Pipeline & Build Tripwires
// ---------------------------------------------------------------------------
/// Orchestrates the execution of the Track 0 bare-metal kernel build pipeline.
///
/// # Silicon & Assembly Invariants:
/// - Reaches into `<TRACK 0>/kernel/build_all_kernels.sh`.
/// - Assembles 16-bit real-mode bootstrap vector (`0x7C00`) via NASM.
/// - Compiles 64-bit UEFI PE32+ application with Clang/LLD targeting GOP framebuffer.
/// - Links 32-bit ELF multiboot header with GNU LD for hypervisor direct handshakes.
///
/// # Sensory Tripwires:
/// - `TRIPWIRE_KERNEL_BUILD_SCRIPT`: Asserts build script is readable and executable.
/// - `TRIPWIRE_KERNEL_COMPILATION`: Asserts zero exit code from compilation subprocess.
/// - `TRIPWIRE_IMAGE_RESOLVE`: Asserts that target binary artifact exists on disk after assembly.
pub fn execute_kernel_pipeline(ctx: &mut BootContext) -> Result<(), TripwireError> {
    ctx.logger.log("KERNEL_PIPELINE", "Initiating Track 0 Bare-Metal Kernel Build Pipeline...");
    ctx.debugger.inspect_state(QuadState::Await, "kernel_build_started");
    ctx.debugger.step_count += 1;

    // 0. Transpile MilleniumOS boot architecture directly from OmniCode source reel
    let boot_omni_path = resolve_track_path("lib/kernel/boot.omni");
    if boot_omni_path.exists() {
        if let Ok(omni_content) = std::fs::read_to_string(&boot_omni_path) {
            if let Ok(generated_asm) = crate::runtime::assembler::transpile_omni_to_asm(&omni_content) {
                let bios_asm_path = resolve_track_path("kernel/bios/boot.asm");
                let _ = std::fs::write(&bios_asm_path, &generated_asm);
                let root_asm_path = resolve_track_path("kernel/boot.asm");
                let _ = std::fs::write(&root_asm_path, &generated_asm);
                ctx.logger.log("KERNEL_PIPELINE", "Transpiled lib/kernel/boot.omni -> bios/boot.asm for MilleniumOS");
            }
        }
    }

    let script_dir = ctx.build_script.parent().unwrap_or_else(|| Path::new("."));
    ctx.logger.log(
        "KERNEL_PIPELINE",
        format!("Executing driver script in directory: {:?}", script_dir),
    );

    println!("\x1b[1;34m[i] Executing bare-metal kernel build pipeline in {:?}...\x1b[0m", script_dir);
    let build_status = Command::new("bash")
        .arg(ctx.build_script.file_name().unwrap_or_default())
        .current_dir(script_dir)
        .status();

    match Tripwire::assert_command_success(&ctx.logger, "KERNEL_PIPELINE", build_status, "Kernel Build Pipeline") {
```

```
        Ok(()) => {
            println!("\x1b[1;32m✓ Track 0 Bare-Metal Kernel Pipeline Built Successfully\x1b[0m");
            ctx.logger.log("KERNEL_PIPELINE", "Kernel compilation subprocess returned exit code 0");
            ctx.logger.record_ai_observation(
                "KERNEL_BUILD_SUCCESS",
                "Bare-metal kernel compilation passed",
                "Track 0 build script executed cleanly with all artifacts generated",
                ["kernel", "build", "silicon", "verified"].as_slice(),
                Some("build_all_kernels.sh"),
            );
        }
        Err(e) => {
            ctx.debugger.inspect_state(QuadState::Rollback, "kernel_build_failed");
            ctx.logger.record_ai_observation(
                "KERNEL_BUILD_FAILED",
                "Bare-metal kernel compilation failed",
                &e.message,
                ["kernel", "build_failure", "rollback"].as_slice(),
                Some("build_all_kernels.sh"),
            );
            return Err(e);
        }
    }

    // Verify generated artifact presence post-build
    Tripwire::assert_path_exists(
        &ctx.logger,
        "KERNEL_PIPELINE",
        &ctx.boot_artifact,
        "Post-compilation target boot artifact",
    )?;

    ctx.debugger.inspect_state(QuadState::Proceed, "kernel_artifact_verified");
    Ok(())
}

// ---------------------------------------------------------------------------
// 3.3. Hypervisor Command Synthesis & Parameter Mapping
// ---------------------------------------------------------------------------
/// Constructs the platform-specific hypervisor launch command with exact MMIO,
/// serial redirection, and drive geometry parameters.
///
/// # Hardware & Architecture Mapping (Simulating Motherboard Buses):
/// - **BIOS**: Maps raw 512-byte image as primary boot drive (`format=raw,file=...`).
/// - **UEFI**: Maps OVMF firmware blob (`-bios ...`) and mounts ESP FAT partition.
/// - **Multiboot**: Direct hypervisor injection via QEMU `-kernel` flag.
/// - **Serial & VGA**: Multiplexes COM1 (`0x3F8`) and VGA text buffer (`0xB8000`)
///   to host standard I/O streams for headless evaluation or GUI windows.
///
/// # Sensory Tripwires:
/// - `TRIPWIRE_HYPERVISOR_PATH`: Asserts hypervisor binary (`qemu-system-x86_64`) exists on PATH.
/// - `TRIPWIRE_FIRMWARE_RESOLVE`: Asserts UEFI OVMF firmware blob exists when UEFI mode is active.
pub fn synthesize_hypervisor_command(ctx: &BootContext) -> Result<Command, TripwireError> {
    ctx.logger.log("HYPERVISOR", format!("Synthesizing launch command for {}", ctx.hypervisor_binary));

    // Assert that the hypervisor binary is present on host system PATH
    Tripwire::assert_binary_available(&ctx.logger, "HYPERVISOR", ctx.hypervisor_binary)?;

    let mut cmd = Command::new(ctx.hypervisor_binary);

    match ctx.config.arch {
        BootArch::Bios => {
            ctx.logger.log("HYPERVISOR", format!("Configuring BIOS MBR raw drive: {:?}", ctx.boot_artifact));
            let path_str = ctx.boot_artifact.to_string_lossy();
            let file_arg = if path_str.starts_with('/') || path_str.starts_with("./") {
                path_str.into_owned()
            } else {
                format!("./{path_str}")
            };
            cmd.arg("-drive").arg(format!("format=raw,file={file_arg}"));
        }
        BootArch::Uefi => {
            let ovmf = if let Some(ref fw) = ctx.ovmf_firmware {
                fw.to_string_lossy().into_owned()
            } else {
                let default_ovmf = "/usr/share/qemu/OVMF.fd".to_string();
                ctx.logger.log("HYPERVISOR", format!("Using fallback OVMF path: {}", default_ovmf));
                default_ovmf
```

```
            };
            ctx.logger.log("HYPERVISOR", format!("Configuring UEFI with OVMF firmware: {}", ovmf));
            cmd.arg("-bios").arg(ovmf);

            let esp_dir = if ctx.boot_artifact.to_string_lossy().contains(':') {
                let staged = PathBuf::from("target/esp");
                let staged_boot = staged.join("EFI").join("BOOT");
                let _ = std::fs::create_dir_all(&staged_boot);
                let efi_bin = ctx.boot_artifact.join("EFI").join("BOOT").join("BOOTX64.EFI");
                if efi_bin.exists() {
                    let _ = std::fs::copy(&efi_bin, staged_boot.join("BOOTX64.EFI"));
                }
                staged.to_string_lossy().into_owned()
            } else {
                ctx.boot_artifact.to_string_lossy().into_owned()
            };
            cmd.arg("-drive").arg(format!("file=fat:rw:{esp_dir},format=raw"));
            cmd.arg("-net").arg("none");
        }
        BootArch::Multiboot => {
            ctx.logger.log("HYPERVISOR", format!("Configuring direct multiboot ELF injection: {:?}",
ctx.boot_artifact));
            let path_str = ctx.boot_artifact.to_string_lossy();
            let kernel_arg = if path_str.starts_with('/') || path_str.starts_with("./") {
                path_str.into_owned()
            } else {
                format!("./{path_str}")
            };
            cmd.arg("-kernel").arg(kernel_arg);
        }
    }

    if ctx.config.graphical {
        ctx.logger.log("HYPERVISOR", "Display mode: Graphical Window with COM1 serial on stdio");
        cmd.arg("-serial").arg("stdio");
    } else {
        ctx.logger.log("HYPERVISOR", "Display mode: Headless with COM1 serial on stdio (display=none)");
        cmd.arg("-display").arg("none").arg("-serial").arg("stdio");
    }

    Ok(cmd)
}

// ---------------------------------------------------------------------------
// 3.4. Machine Execution Loop, Tripwire Interception & Telemetry Seal
// ---------------------------------------------------------------------------
/// Executes the complete bare-metal machine inception pipeline.
///
/// # Game Dev Loop Rationale:
/// Directly executes the simulation tick of the host engine:
/// 1. Emits visual inception banner with machine metadata.
/// 2. Executes kernel pipeline wrapped in build tripwires.
/// 3. Synthesizes hypervisor launch command with hardware mappings.
/// 4. Executes virtual machine, trapping non-zero exit codes.
/// 5. Inscribes telemetry into [`DiagnosticLogger`] and [`OmniDebugger`] for the
[`crate::diagnostics::restorator::Restorator`].
pub fn execute_boot_pipeline(mut ctx: BootContext) -> Result<QuadState, TripwireError> {
    println!("\x1b[1;36m================================================================================\x1b[0m");
    println!("\x1b[1;36m  OmniCode OS: Bare-Metal Multi-Architecture Machine Inception               \x1b[0m");
    println!("\x1b[1;36m  Mode: {}\x1b[0m", ctx.config.arch.display_name());
    println!("\x1b[1;36m  Host: {}\x1b[0m", ctx.config.host_platform.display_name());
    println!("\x1b[1;36m================================================================================\x1b[0m\n");

    ctx.logger.log(
        "BOOT",
        format!(
            "Beginning Machine Inception [{}] on [{}]",
            ctx.config.arch.display_name(),
            ctx.config.host_platform.display_name()
        ),
    );
    ctx.debugger.inspect_state(QuadState::Await, "machine_inception_begun");
    ctx.debugger.step_count += 1;

    // 1. Execute Bare-Metal Kernel Pipeline in Track 0
    execute_kernel_pipeline(&mut ctx)?;

    // 2. Configure Machine Emulator Command
```

```
    let mut cmd = synthesize_hypervisor_command(&ctx)?;

    if ctx.config.graphical {
        println!("\x1b[1;36m[i] Launching graphical virtual machine display with serial on stdio...\x1b[0m");
    } else {
        println!("\x1b[1;36m[i] Launching headless terminal machine with COM1 serial on stdio...\x1b[0m");
    }

    // 3. Launch Virtual Machine Inception
    ctx.logger.log("BOOT", format!("Spawning hypervisor child process: {:?}", cmd));
    ctx.debugger.inspect_state(QuadState::Await, "hypervisor_spawning");
    ctx.debugger.step_count += 1;

    let run_status = cmd.status();
    match Tripwire::assert_command_success(&ctx.logger, "BOOT", run_status, ctx.hypervisor_binary) {
        Ok(()) => {
            println!("\n\x1b[1;32m✓ QEMU Machine Inception Terminated Cleanly (+1:PROCEED)\x1b[0m");
            ctx.logger.log("BOOT", "Virtual machine exited with status code 0 (+1:PROCEED)");
            ctx.debugger.inspect_state(QuadState::Proceed, "machine_inception_clean_exit");
            ctx.logger.record_ai_observation(
                "BOOT_SUCCESS",
                "Machine inception ran to completion",
                "QEMU execution exited with success code 0",
                ["hypervisor", "boot", "proceed", "verified"].as_slice(),
                Some(ctx.hypervisor_binary),
            );
            Ok(QuadState::Proceed)
        }
        Err(e) => {
            ctx.debugger.inspect_state(QuadState::Rollback, "machine_inception_trapped");
            ctx.logger.record_ai_observation(
                "BOOT_FAILED",
                "Machine inception trapped failpoint",
                &e.message,
                ["hypervisor", "boot_failure", "rollback"].as_slice(),
                Some(ctx.hypervisor_binary),
            );
            eprintln!("  (Ensure {} is installed and accessible on your PATH)", ctx.hypervisor_binary);
            Err(e)
        }
    }
}

/// Backwards-compatible synchronous executor for machine inception.
pub fn execute_boot_machine(ctx: BootContext) {
    if let Err(e) = execute_boot_pipeline(ctx) {
        eprintln!("\x1b[1;31mMachine inception halted:\x1b[0m {e}");
        std::process::exit(1);
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Module Sealing, Invariant Verification & Downstream Baton Pass )
// ============================================================================

// ----------------------------------------------------------------------------
// 4.1. Module Sealing & Downstream Baton Pass Specification
// ----------------------------------------------------------------------------
// BATON PASS CONTRACT:
//   - Upstream Producer: CLI Command Dispatcher (`omnicode boot ...`).
//   - Downstream Receiver: `<TRACK 0>` Bare-Metal Kernel Pipeline & Host QEMU Hypervisor.
//   - Certified Invariants:
//     1. Target architecture variants (`Bios`, `Uefi`, `Multiboot`) map deterministically
//        to valid artifact targets in the `<TRACK 0>/kernel` tree.
//     2. Hardware MMIO base registers (COM1 `0x3F8`, VGA `0xB8000`, Real Mode `0x7C00`)
//        are immutable and verified.
//     3. Command synthesis accurately maps drives, firmware, and stdio multiplexing.
//     4. `BootContext::open` certifies all filesystem preconditions, initializes an
//        attached `OmniDebugger` and `DiagnosticLogger`, and leaves zero leaked state.
//     5. Any execution failure is intercepted by sensory tripwires and packaged with
//        remediation suggestions for the `Restorator`.

#[cfg(test)]
mod tests {
    use super::*;

    /// Verifies that all `BootArch` enum variants return non-empty, distinct, human-readable display titles.
```

```
    ///
    /// # Failure Mode Guarded Against:
    /// Prevents silent string regression or unmapped match arms when new architecture targets are added.
    #[test]
    fn test_boot_arch_display() {
        assert!(BootArch::Bios.display_name().contains("BIOS"), "BIOS display title must identify MBR mode");
        assert!(BootArch::Uefi.display_name().contains("UEFI"), "UEFI display title must identify Long Mode");
        assert!(BootArch::Multiboot.display_name().contains("Multiboot"), "Multiboot display title must identify ELF
mode");
    }

    /// Verifies that the physical kernel build script and default BIOS boot sector image resolve in `<TRACK 0>`.
    ///
    /// # Failure Mode Guarded Against:
    /// Guards against repository relocation anomalies or broken relative path traversal to Track 0.
    #[test]
    fn test_kernel_paths_resolve() {
        let build_script = resolve_track_path("kernel/build_all_kernels.sh");
        assert!(
            build_script.exists(),
            "Track 0 kernel build driver script must exist on disk"
        );
        let bios_img = resolve_track_path("kernel/images/omnicode_bios.img");
        assert!(
            bios_img.exists(),
            "Track 0 default BIOS binary artifact must exist on disk"
        );
    }

    /// Verifies that `HostPlatform::current()` correctly detects the active host environment
    /// and provides non-empty hypervisor and firmware candidate paths.
    ///
    /// # Failure Mode Guarded Against:
    /// Prevents null or uninitialized platform definitions from entering the boot pipeline.
    #[test]
    fn test_host_platform_detection() {
        let current = HostPlatform::current();
        assert!(!current.display_name().is_empty(), "Host platform must have a non-empty display name");
        assert!(!current.hypervisor_binary().is_empty(), "Host platform must provide a default hypervisor binary");
        assert!(!current.ovmf_search_paths().is_empty(), "Host platform must declare firmware candidate paths");
    }

    /// Verifies command-line argument parsing across short flags, long flags, and option permutations.
    ///
    /// # Invariants Tested:
    /// - `--uefi` and `-u` select `BootArch::Uefi`.
    /// - `--multiboot` and `-m` select `BootArch::Multiboot`.
    /// - Absence of arch flags defaults to `BootArch::Bios`.
    /// - `--gui` sets `graphical = true`.
    /// - `--ovmf <PATH>` extracts custom firmware path override.
    #[test]
    fn test_boot_config_from_args() {
        let args = vec!["omnicode".to_string(), "boot".to_string(), "--uefi".to_string(), "--gui".to_string()];
        let cfg = BootConfig::from_args(&args);
        assert_eq!(cfg.arch, BootArch::Uefi, "Flag --uefi must select BootArch::Uefi");
        assert!(cfg.graphical, "Flag --gui must set graphical mode to true");

        let args_mb = vec!["omnicode".to_string(), "boot".to_string(), "-m".to_string()];
        let cfg_mb = BootConfig::from_args(&args_mb);
        assert_eq!(cfg_mb.arch, BootArch::Multiboot, "Flag -m must select BootArch::Multiboot");
        assert!(!cfg_mb.graphical, "Default graphical mode must be headless (false)");

        let args_custom_ovmf = vec![
            "omnicode".to_string(),
            "boot".to_string(),
            "--uefi".to_string(),
            "--ovmf".to_string(),
            "/custom/ovmf.fd".to_string(),
        ];
        let cfg_custom = BootConfig::from_args(&args_custom_ovmf);
        assert_eq!(cfg_custom.custom_ovmf, Some(PathBuf::from("/custom/ovmf.fd")));
    }

    /// Verifies that `BootConfig::open` successfully certifies preconditions, resolves Track 0 scripts,
    /// attaches an armed `OmniDebugger`, and constructs an immutable `BootContext`.
    ///
    /// # Failure Mode Guarded Against:
    /// Prevents uncertified or partially initialized execution frames from reaching the hypervisor spawner.
```

```rust
    #[test]
    fn test_boot_context_open_preconditions() {
        let cfg = BootConfig::new(BootArch::Bios, false);
        let logger = DiagnosticLogger::new(false);
        let ctx_res = cfg.open(logger);
        assert!(ctx_res.is_ok(), "Opening BootContext must succeed when Track 0 workspace is valid");
        let ctx = ctx_res.unwrap();
        assert!(ctx.build_script.exists(), "Certified build script path must exist on disk");
        assert_eq!(ctx.config.arch, BootArch::Bios, "Context config arch must match specification");
        assert_eq!(ctx.debugger.step_count, 0, "Attached debugger must start at step count 0");
    }

    /// Verifies hypervisor command synthesis for BIOS MBR mode in headless serial operation.
    ///
    /// # Silicon Wiring Invariants:
    /// - Primary raw drive mapped to target artifact (`-drive format=raw,file=...`).
    /// - Display disabled (`-display none`).
    /// - COM1 serial port (`0x3F8`) multiplexed to standard I/O (`-serial stdio`).
    #[test]
    fn test_synthesize_hypervisor_command_bios_headless() {
        let cfg = BootConfig::new(BootArch::Bios, false);
        let logger = DiagnosticLogger::new(false);
        let ctx = cfg.open(logger).expect("Context open succeeds");
        let cmd = synthesize_hypervisor_command(&ctx).expect("Command synthesis succeeds");
        let args: Vec<String> = cmd.get_args().map(|s| s.to_string_lossy().into_owned()).collect();

        assert!(args.contains(&"-drive".to_string()), "Command must include -drive flag");
        assert!(args.iter().any(|a| a.contains("format=raw,file=")), "Must specify raw MBR disk format");
        assert!(args.contains(&"-display".to_string()), "Headless mode must include -display flag");
        assert!(args.contains(&"none".to_string()), "Headless mode must specify display none");
        assert!(args.contains(&"-serial".to_string()), "COM1 serial port must be redirected");
        assert!(args.contains(&"stdio".to_string()), "Serial redirection must target stdio");
    }

    /// Verifies hypervisor command synthesis for graphical window display mode.
    ///
    /// # Graphical Wiring Invariants:
    /// - Must NOT include `-display none`.
    /// - Serial COM1 must remain connected to `stdio` for background telemetry.
    #[test]
    fn test_synthesize_hypervisor_command_graphical() {
        let cfg = BootConfig::new(BootArch::Bios, true);
        let logger = DiagnosticLogger::new(false);
        let ctx = cfg.open(logger).expect("Context open succeeds");
        let cmd = synthesize_hypervisor_command(&ctx).expect("Command synthesis succeeds");
        let args: Vec<String> = cmd.get_args().map(|s| s.to_string_lossy().into_owned()).collect();

        assert!(!args.contains(&"-display".to_string()), "Graphical mode must not pass -display none");
        assert!(args.contains(&"-serial".to_string()), "Graphical mode must retain COM1 serial redirection");
        assert!(args.contains(&"stdio".to_string()), "Serial output must stream to host stdio");
    }

    /// Verifies hypervisor command synthesis for direct Multiboot kernel injection.
    ///
    /// # Hypervisor Direct Handshake Invariants:
    /// - ELF binary injected directly into RAM via QEMU `-kernel` parameter.
    #[test]
    fn test_synthesize_hypervisor_command_multiboot() {
        let cfg = BootConfig::new(BootArch::Multiboot, false);
        let logger = DiagnosticLogger::new(false);
        let ctx = cfg.open(logger).expect("Context open succeeds");
        let cmd = synthesize_hypervisor_command(&ctx).expect("Command synthesis succeeds");
        let args: Vec<String> = cmd.get_args().map(|s| s.to_string_lossy().into_owned()).collect();

        assert!(args.contains(&"-kernel".to_string()), "Multiboot must pass -kernel flag");
        assert!(args.iter().any(|a| a.contains("omnicode_multiboot.elf")), "Kernel path must point to multiboot ELF");
    }

    /// Verifies hypervisor command synthesis for UEFI Long Mode execution.
    ///
    /// # Firmware & ESP Wiring Invariants:
    /// - OVMF firmware image mapped via `-bios`.
    /// - ESP directory mapped as FAT drive via `file=fat:rw:...,format=raw`.
    #[test]
    fn test_synthesize_hypervisor_command_uefi() {
        let mut cfg = BootConfig::new(BootArch::Uefi, false);
        cfg.custom_ovmf = Some(PathBuf::from("/usr/share/qemu/OVMF.fd"));
        let logger = DiagnosticLogger::new(false);
```

```
        let ctx = cfg.open(logger).expect("Context open succeeds");
        let cmd = synthesize_hypervisor_command(&ctx).expect("Command synthesis succeeds");
        let args: Vec<String> = cmd.get_args().map(|s| s.to_string_lossy().into_owned()).collect();

        assert!(args.contains(&"-bios".to_string()), "UEFI mode must pass -bios flag");
        assert!(args.contains(&"/usr/share/qemu/OVMF.fd".to_string()), "Must wire specified OVMF firmware blob");
        assert!(args.contains(&"-drive".to_string()), "Must configure EFI System Partition drive");
        assert!(args.iter().any(|a| a.contains("file=fat:rw:")), "Must mount ESP as FAT filesystem");
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/cli/commands/mod.rs RS 44 LINES 150 WORDS 1.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( CLI Commands Subsystem Identity & Command Gear Registry )
// ============================================================================
//! OmniCode CLI Command Gears
//!
//! Encapsulates discrete CLI command implementations as modular mechanical gears
//! driven by the main orchestrator engine.

pub mod boot;
pub mod assemble;
pub mod run;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Command Re-exports )
// ============================================================================
pub use boot::{
    BootArch, BootConfig, BootContext, run_boot, run_boot_with_config,
    execute_kernel_pipeline, synthesize_hypervisor_command, execute_boot_pipeline, execute_boot_machine,
    DEFAULT_BOOT_ARCH, DEFAULT_GRAPHICAL_MODE, REAL_MODE_BOOT_VECTOR, VGA_TEXT_FRAMEBUFFER_BASE, COM1_SERIAL_IO_BASE,
};

pub use assemble::{
    AssembleFormat, AssembleConfig, AssembleContext, run_assemble, run_assemble_with_config,
    execute_assemble_pipeline, serialize_assembled_output,
};

pub use run::{
    RunFormat, RunConfig, RunContext, RunResult, run_with_config, execute_run_pipeline,
    run_on_virtual_terminal, run_omni_source, run_tape_ribbon, run_c_transpile, run_lua_transpile,
};

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Command Routing & Common Dispatch Logic )
// ============================================================================

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Verification and Sealing )
// ============================================================================
// Command gears verified and sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/cli/commands/run.rs RS 926 LINES 2972 WORDS 34.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Universal CLI Runner: Multi-Substrate Execution across Host & Virtual Terminal )
// ============================================================================
//! OmniCode CLI Unified Runner Command Gear
//!
//! # Architectural Scope & Purpose:
//! Orchestrates execution of all OmniCode code manifestations across both host terminals
//! and the virtual terminal substrate:
//! 1. Native Spatial Source Reels (`.omni`) & Episodic Scrolls (`.scroll`)
//! 2. Procedural Process Tapes (`.tape`)
//! 3. Transpiled C Backbones & Tests (`.c`)
//! 4. Transpiled Lua Process Tapes (`.lua`)
//! 5. Dynamic On-The-Fly Transpilation (`--transpile c` / `--transpile lua`)
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of physical data into bounded cell tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types across host and virtual substrates).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical memory boundaries, silicon addresses, and constants).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Substrate Transparency**: An OmniCode process behaves identically whether executed
//!    directly on a Linux host terminal via `omnicode run`, simulated in `TerminalSession`,
//!    transpiled to C for bare metal, or run via Lua for game engine scripting.
// ============================================================================

use std::fmt;
use std::fs;
use std::path::{Path, PathBuf};
use std::process::Command;
use std::time::Instant;

use crate::cogs::tripwire::{Tripwire, TripwireError};
use crate::core::filetype::{resolve_track_path, OmniFileType};
use crate::core::ternary::QuadState;
use crate::diagnostics::debugger::OmniDebugger;
use crate::diagnostics::logger::{DiagnosticLogger, FailpointType};
use crate::parser::lexer::Lexer;
use crate::parser::parser::Parser;
use crate::runtime::assembler::OmniAssembler;
use crate::runtime::environment::Environment;
use crate::runtime::evaluator::core::Evaluator;
use crate::terminal::scheduler::{OmniScheduler, Track};
use crate::terminal::session::TerminalSession;
use std::sync::atomic::{AtomicU64, Ordering};

/// Thread-safe counter guaranteeing unique temporary artifact names across concurrent test threads.
static TEMP_COUNTER: AtomicU64 = AtomicU64::new(0);

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Run Configurations, Substrate Format Enums & Context Definitions )
// ============================================================================

/// Target execution substrate format.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, Default)]
pub enum RunFormat {
    /// Auto-detect format from file extension and content.
    #[default]
    Auto,
    /// Native OmniCode 4-Block source reel (`.omni`) or episodic scroll (`.scroll`).
    Omni,
    /// Procedural process ribbon tape (`.tape`).
    Tape,
    /// Standard C transpile equivalent or unit test (`.c`).
    C,
```

```
    /// Standard Lua transpile equivalent or tape script (`.lua`).
    Lua,
}

impl RunFormat {
    /// Resolves format from a string name.
    pub fn from_str_name(s: &str) -> Option<Self> {
        match s.to_lowercase().as_str() {
            "omni" | "reel" | "scroll" => Some(Self::Omni),
            "tape" => Some(Self::Tape),
            "c" => Some(Self::C),
            "lua" => Some(Self::Lua),
            "auto" => Some(Self::Auto),
            _ => None,
        }
    }

    /// Resolves format from a file path extension.
    pub fn from_path(path: &Path) -> Self {
        let ext = path.extension().and_then(|e| e.to_str()).unwrap_or("").to_lowercase();
        match ext.as_str() {
            "omni" | "scroll" => Self::Omni,
            "tape" => Self::Tape,
            "c" | "h" => Self::C,
            "lua" => Self::Lua,
            _ => Self::Auto,
        }
    }

    /// Canonical lowercase format name.
    pub fn as_str(&self) -> &'static str {
        match self {
            Self::Auto => "auto",
            Self::Omni => "omni",
            Self::Tape => "tape",
            Self::C => "c",
            Self::Lua => "lua",
        }
    }
}

impl fmt::Display for RunFormat {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{}", self.as_str())
    }
}

/// Configuration options for the runner command.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct RunConfig {
    /// Target file path to execute.
    pub input_path: PathBuf,
    /// Substrate format.
    pub format: RunFormat,
    /// Optional transpile target on the fly (`"c"` or `"lua"`).
    pub transpile_target: Option<String>,
    /// Verbose diagnostic logging.
    pub verbose: bool,
    /// Dry run mode (validates and compiles without executing).
    pub dry_run: bool,
}

impl RunConfig {
    /// Creates a new `RunConfig` with default flags.
    pub fn new(input_path: impl Into<PathBuf>) -> Self {
        let path = input_path.into();
        let format = RunFormat::from_path(&path);
        Self {
            input_path: path,
            format,
            transpile_target: None,
            verbose: false,
            dry_run: false,
        }
    }

    /// Parses command line arguments for `omnicode run <file> [flags]`.
    pub fn from_args(args: &[String]) -> Result<Self, String> {
```

```
        let mut input_path: Option<PathBuf> = None;
        let mut format = RunFormat::Auto;
        let mut transpile_target: Option<String> = None;
        let mut verbose = false;
        let mut dry_run = false;

        let mut idx = 0;
        while idx < args.len() {
            match args[idx].as_str() {
                "--format" | "-f" => {
                    idx += 1;
                    if idx >= args.len() {
                        return Err("FAILPOINT [MissingFormat]: Option '--format' requires a format argument (omni|tape|c|lua)".into());
                    }
                    format = RunFormat::from_str_name(&args[idx])
                        .ok_or_else(|| format!("FAILPOINT [InvalidFormat]: Unknown format '{}'", args[idx]))?;
                }
                "--transpile" | "-t" => {
                    idx += 1;
                    if idx >= args.len() {
                        return Err("FAILPOINT [MissingTranspile]: Option '--transpile' requires a target (c|lua)".into());
                    }
                    transpile_target = Some(args[idx].to_lowercase());
                }
                "--verbose" | "-v" => {
                    verbose = true;
                }
                "--dry-run" | "-d" => {
                    dry_run = true;
                }
                other => {
                    if !other.starts_with('-') && input_path.is_none() {
                        input_path = Some(PathBuf::from(other));
                    }
                }
            }
            idx += 1;
        }

        let input_path = input_path.ok_or_else(|| {
            "FAILPOINT [MissingInputPath]: No target file specified. Usage: omnicode run <file> [--format omni|tape|c|lua] [--transpile c|lua]".to_string()
        })?;

        if format == RunFormat::Auto {
            format = RunFormat::from_path(&input_path);
        }

        Ok(Self {
            input_path,
            format,
            transpile_target,
            verbose,
            dry_run,
        })
    }

    /// Resolves target file across track roots and validates preconditions.
    pub fn open(&self, logger: &DiagnosticLogger) -> Result<RunContext, TripwireError> {
        let resolved = resolve_track_path(&self.input_path);
        Tripwire::assert_path_exists(
            logger,
            "TRIPWIRE_RUN_RESOLVE",
            &resolved,
            "Target script, process tape, or transpile source",
        )?;

        let format = if self.format == RunFormat::Auto {
            RunFormat::from_path(&resolved)
        } else {
            self.format
        };

        Ok(RunContext {
            resolved_input: resolved,
            format,
```

```
            config: self.clone(),
            logger: logger.clone(),
        })
    }
}

/// Context passed through execution stages.
#[derive(Debug, Clone)]
pub struct RunContext {
    /// Fully resolved target path.
    pub resolved_input: PathBuf,
    /// Target execution format.
    pub format: RunFormat,
    /// Immutable run configuration.
    pub config: RunConfig,
    /// System diagnostic flight recorder.
    pub logger: DiagnosticLogger,
}

/// Result of running a file.
#[derive(Debug, Clone, PartialEq)]
pub struct RunResult {
    /// Executed substrate format.
    pub format: RunFormat,
    /// Final balanced ternary state (+1:PROCEED).
    pub state: QuadState,
    /// Combined terminal output string.
    pub output: String,
    /// Execution duration in microseconds.
    pub duration_micros: u128,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Multi-Substrate Execution Pipelines & Virtual Terminal Integration )
// ============================================================================

/// Discovers Track 2 standard library paths for C and Lua transpile backbones.
pub fn find_track2_transpile_dirs() -> (PathBuf, PathBuf, PathBuf, PathBuf) {
    let base = resolve_track_path("lexicon/01_nouns/ideas/types/transpile variant");
    let c_include = base.join("c_backbone/include");
    let c_tests_include = base.join("tests/c/includes");
    let c_src = base.join("c_backbone/src");
    let lua_tapes = base.join("lua_tapes");
    (c_include, c_tests_include, c_src, lua_tapes)
}

/// Executes an OmniCode source reel (`.omni`) or episodic scroll (`.scroll`).
pub fn run_omni_source(ctx: &RunContext) -> Result<RunResult, TripwireError> {
    let start = Instant::now();
    let content = match fs::read_to_string(&ctx.resolved_input) {
        Ok(c) => c,
        Err(e) => {
            return Err(Tripwire::trip(
                &ctx.logger,
                "TRIPWIRE_OMNI_READ",
                FailpointType::SyntaxAnomaly,
                format!("Failed to read OmniCode file '{}': {e}", ctx.resolved_input.display()),
            ));
        }
    };

    let mut lexer = Lexer::new(&content, ctx.logger.clone());
    let tokens = match lexer.tokenize() {
        Ok(toks) => toks,
        Err(e) => {
            return Err(Tripwire::trip(
                &ctx.logger,
                "TRIPWIRE_OMNI_LEX",
                FailpointType::SyntaxAnomaly,
                format!("Lexical analysis failed for '{}': {e}", ctx.resolved_input.display()),
            ));
        }
    };

    let debugger = OmniDebugger::new(ctx.logger.clone(), 100);
    let mut parser = Parser::new(tokens, debugger);
    let prog = match parser.parse_program() {
```

```
        Ok(p) => p,
        Err(e) => {
            return Err(Tripwire::trip(
                &ctx.logger,
                "TRIPWIRE_OMNI_PARSE",
                FailpointType::SyntaxAnomaly,
                format!("Parsing failed for '{}': {e}", ctx.resolved_input.display()),
            ));
        }
    };

    let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
    let eval_val = match evaluator.eval(&crate::ast::nodes::ASTNode::Program(prog)) {
        Ok(v) => v,
        Err(e) => {
            return Err(Tripwire::trip(
                &ctx.logger,
                "TRIPWIRE_OMNI_EVAL",
                FailpointType::SyntaxAnomaly,
                format!("Evaluation failed for '{}': {e}", ctx.resolved_input.display()),
            ));
        }
    };

    let state = eval_val.to_quad_state();
    let output = format!(
        "[+] OmniCode Source Reel Executed Successfully: '{}'\n    • Substrate: Native OmniCode VM (Track 1 Rust
Engine)\n    • Result:    => {}\n    • State:     +1:PROCEED",
        ctx.resolved_input.display(),
        eval_val.format_colored()
    );

    // Inscribe persistent execution scroll and telemetry log
    let session_id = format!("run_omni_{}", std::process::id());
    let timestamp_str = crate::ai::model::scroll::generate_timestamp_id();
    let scroll = crate::ai::model::scroll::ExecutionScroll {
        session_id,
        timestamp_str,
        prompt: format!("Execute OmniCode Source Reel: {}", ctx.resolved_input.display()),
        synthesized_omnicode: content,
        execution_state: state,
        natural_response: output.clone(),
        failpoints: ctx.logger.failpoints(),
        repairs_applied: Vec::new(),
        remediations_suggested: ctx.logger.remediations(),
        elapsed_micros: start.elapsed().as_micros(),
    };
    let _ = crate::ai::model::scroll::ScrollLogger::default().save_dual_logs(&scroll);

    Ok(RunResult {
        format: RunFormat::Omni,
        state,
        output,
        duration_micros: start.elapsed().as_micros(),
    })
}

/// Executes a sequential process tape (`.tape`) via the OmniScheduler.
pub fn run_tape_ribbon(ctx: &RunContext) -> Result<RunResult, TripwireError> {
    let start = Instant::now();
    let mut scheduler = OmniScheduler::new();

    let path_str = ctx.resolved_input.to_string_lossy().to_string();
    let track = if path_str.contains("kernel") || path_str.contains("boot") {
        Track::Kernel
    } else {
        Track::App
    };

    let pid = match scheduler.mount_tape_file(track, &path_str, ctx.logger.clone()) {
        Ok(id) => id,
        Err(e) => {
            return Err(Tripwire::trip(
                &ctx.logger,
                "TRIPWIRE_TAPE_MOUNT",
                FailpointType::SyntaxAnomaly,
                format!("Failed to mount process tape '{}': {e}", ctx.resolved_input.display()),
            ));
```

```
        }
    };

    let mut logs = Vec::new();
    logs.push(format!("[+] Mounted Tape '{}' onto {} (PID {pid})", path_str, track.as_str()));

    let mut steps = 0;
    loop {
        match scheduler.step_task(pid) {
            Ok(msg) => {
                logs.push(msg.clone());
                if msg.contains("completed") || msg.contains("already executed") || msg.contains("STOP") {
                    break;
                }
            }
            Err(e) => {
                return Err(Tripwire::trip(
                    &ctx.logger,
                    "TRIPWIRE_TAPE_STEP",
                    FailpointType::SyntaxAnomaly,
                    format!("Task {pid} step halted: {e}"),
                ));
            }
        }
        steps += 1;
        if steps > 1000 {
            logs.push("[!] Transport kinematic loop limit reached (1000 steps)".to_string());
            break;
        }
    }

    let output = logs.join("\n");

    // Inscribe persistent execution scroll and telemetry log
    let session_id = format!("run_tape_{}", std::process::id());
    let timestamp_str = crate::ai::model::scroll::generate_timestamp_id();
    let scroll = crate::ai::model::scroll::ExecutionScroll {
        session_id,
        timestamp_str,
        prompt: format!("Run OmniCode Process Tape: {}", ctx.resolved_input.display()),
        synthesized_omnicode: output.clone(),
        execution_state: QuadState::Proceed,
        natural_response: "Tape execution complete: All rolls verified in Proceed (+1)".to_string(),
        failpoints: ctx.logger.failpoints(),
        repairs_applied: Vec::new(),
        remediations_suggested: ctx.logger.remediations(),
        elapsed_micros: start.elapsed().as_micros(),
    };
    let _ = crate::ai::model::scroll::ScrollLogger::default().save_dual_logs(&scroll);

    Ok(RunResult {
        format: RunFormat::Tape,
        state: QuadState::Proceed,
        output,
        duration_micros: start.elapsed().as_micros(),
    })
}

/// Compiles and executes a C transpile equivalent (`.c`).
pub fn run_c_transpile(ctx: &RunContext) -> Result<RunResult, TripwireError> {
    let start = Instant::now();
    let (c_inc, c_tests_inc, c_src, _) = find_track2_transpile_dirs();

    let target_path = &ctx.resolved_input;
    let target_content = fs::read_to_string(target_path).unwrap_or_default();
    let has_main = target_content.contains("int main(") || target_content.contains("int main (");

    // Output binary in temp dir with thread-unique identifier
    let seq = TEMP_COUNTER.fetch_add(1, Ordering::SeqCst);
    let temp_bin = std::env::temp_dir().join(format!("omnicode_c_bin_{}_{}", std::process::id(), seq));

    let mut cmd = Command::new("gcc");
    cmd.arg("-O2");

    if c_inc.exists() {
        cmd.arg("-I").arg(&c_inc);
    }
    if c_tests_inc.exists() {
```

```
        cmd.arg("-I").arg(&c_tests_inc);
    }

    cmd.arg(target_path);

    // Link all supporting C sources from c_backbone/src if present
    if c_src.exists() {
        if let Ok(entries) = fs::read_dir(&c_src) {
            for entry in entries.flatten() {
                let p = entry.path();
                if p.extension().and_then(|e| e.to_str()) == Some("c") && p != *target_path {
                    cmd.arg(p);
                }
            }
        }
    }

    // If compiling the multi-file testbed (test_omni_lib.c), link unit test companions
    if target_path.to_string_lossy().contains("test_omni_lib") {
        let unit_dir = c_tests_inc.parent().map(|p| p.join("unit"));
        if let Some(ref udir) = unit_dir {
            if udir.exists() {
                if let Ok(entries) = fs::read_dir(udir) {
                    for entry in entries.flatten() {
                        let p = entry.path();
                        if p.extension().and_then(|e| e.to_str()) == Some("c") && p != *target_path {
                            cmd.arg(p);
                        }
                    }
                }
            }
        }
    }

    // If unit test without main, link with testbed integrated runner
    if !has_main {
        let testbed = c_tests_inc.parent().map(|p| p.join("integrated/test_omni_lib.c"));
        if let Some(tb) = testbed {
            if tb.exists() && tb != *target_path {
                cmd.arg(&tb);
            }
        }
    }

    cmd.arg("-o").arg(&temp_bin);

    let compile_output = match cmd.output() {
        Ok(out) => out,
        Err(e) => {
            return Err(Tripwire::trip(
                &ctx.logger,
                "TRIPWIRE_C_COMPILE",
                FailpointType::SyntaxAnomaly,
                format!("Failed to invoke C compiler (gcc): {e}"),
            ));
        }
    };

    if !compile_output.status.success() {
        let stderr = String::from_utf8_lossy(&compile_output.stderr);
        return Err(Tripwire::trip(
            &ctx.logger,
            "TRIPWIRE_C_BUILD",
            FailpointType::SyntaxAnomaly,
            format!("C compilation failed:\n{stderr}"),
        ));
    }

    if ctx.config.dry_run {
        let _ = fs::remove_file(&temp_bin);
        return Ok(RunResult {
            format: RunFormat::C,
            state: QuadState::Proceed,
            output: format!("[√ Dry Run] C compilation verified for '{}'", target_path.display()),
            duration_micros: start.elapsed().as_micros(),
        });
    }

```

```
    // Execute compiled binary
    let exec_output = match Command::new(&temp_bin).output() {
        Ok(out) => out,
        Err(e) => {
            let _ = fs::remove_file(&temp_bin);
            return Err(Tripwire::trip(
                &ctx.logger,
                "TRIPWIRE_C_EXEC",
                FailpointType::SyntaxAnomaly,
                format!("Failed to execute compiled C binary: {e}"),
            ));
        }
    };

    let _ = fs::remove_file(&temp_bin);

    let stdout = String::from_utf8_lossy(&exec_output.stdout).to_string();
    let stderr = String::from_utf8_lossy(&exec_output.stderr).to_string();
    let combined = if stderr.is_empty() {
        stdout
    } else {
        format!("{stdout}\n{stderr}")
    };

    let state = if exec_output.status.success() {
        QuadState::Proceed
    } else {
        QuadState::Rollback
    };

    Ok(RunResult {
        format: RunFormat::C,
        state,
        output: combined,
        duration_micros: start.elapsed().as_micros(),
    })
}

/// Executes a Lua transpile tape (`.lua`) via the host Lua engine.
pub fn run_lua_transpile(ctx: &RunContext) -> Result<RunResult, TripwireError> {
    let start = Instant::now();
    let (_, _, _, lua_tapes_dir) = find_track2_transpile_dirs();

    let target_path = &ctx.resolved_input;
    let target_dir = target_path.parent().unwrap_or_else(|| Path::new("."));

    // Build LUA_PATH to find tape_engine.lua, test_common.lua, and local modules
    let lua_path_env = format!(
        "{}/?.lua;{}/?.lua;{}/?.lua;{}/tests/lua/unit/?.lua;;",
        lua_tapes_dir.display(),
        target_dir.display(),
        lua_tapes_dir.parent().unwrap_or(&lua_tapes_dir).display(),
        lua_tapes_dir.parent().unwrap_or(&lua_tapes_dir).display()
    );

    let mut cmd = Command::new("lua");
    cmd.env("LUA_PATH", &lua_path_env);
    cmd.arg(target_path);

    if ctx.config.dry_run {
        return Ok(RunResult {
            format: RunFormat::Lua,
            state: QuadState::Proceed,
            output: format!("[√ Dry Run] Lua target validated: '{}'", target_path.display()),
            duration_micros: start.elapsed().as_micros(),
        });
    }

    let output = match cmd.output() {
        Ok(out) => out,
        Err(e) => {
            return Err(Tripwire::trip(
                &ctx.logger,
                "TRIPWIRE_LUA_EXEC",
                FailpointType::SyntaxAnomaly,
                format!("Failed to invoke Lua engine: {e}"),
            ));
        }
```

```
    };

    let stdout = String::from_utf8_lossy(&output.stdout).to_string();
    let stderr = String::from_utf8_lossy(&output.stderr).to_string();
    let combined = if stderr.is_empty() {
        stdout
    } else {
        format!("{stdout}\n{stderr}")
    };

    let state = if output.status.success() {
        QuadState::Proceed
    } else {
        QuadState::Rollback
    };

    Ok(RunResult {
        format: RunFormat::Lua,
        state,
        output: combined,
        duration_micros: start.elapsed().as_micros(),
    })
}

/// Executes a dynamic on-the-fly transpile & run cycle for `.tape` or `.omni`.
pub fn run_transpile_on_the_fly(
    ctx: &RunContext,
    target_lang: &str,
) -> Result<RunResult, TripwireError> {
    let _start = Instant::now();
    let debugger = OmniDebugger::new(ctx.logger.clone(), 100);
    let mut assembler = OmniAssembler::new(ctx.logger.clone(), debugger);

    let assembled = match assembler.assemble_file(&ctx.resolved_input) {
        Ok(a) => a,
        Err(e) => {
            return Err(Tripwire::trip(
                &ctx.logger,
                "TRIPWIRE_TRANSPILE_ASSEMBLE",
                FailpointType::SyntaxAnomaly,
                format!("Assembly failed prior to on-the-fly transpilation: {e}"),
            ));
        }
    };

    match target_lang {
        "c" => {
            let c_code = assembled.format_c();
            let seq = TEMP_COUNTER.fetch_add(1, Ordering::SeqCst);
            let temp_c = std::env::temp_dir().join(format!("omnicode_dyn_{}_{}.c", std::process::id(), seq));
            if let Err(e) = fs::write(&temp_c, &c_code) {
                return Err(Tripwire::trip(
                    &ctx.logger,
                    "TRIPWIRE_TRANSPILE_WRITE",
                    FailpointType::SyntaxAnomaly,
                    format!("Failed to write dynamic C artifact: {e}"),
                ));
            }
            let mut sub_config = ctx.config.clone();
            sub_config.input_path = temp_c.clone();
            sub_config.format = RunFormat::C;
            let sub_ctx = RunContext {
                resolved_input: temp_c.clone(),
                format: RunFormat::C,
                config: sub_config,
                logger: ctx.logger.clone(),
            };
            let res = run_c_transpile(&sub_ctx);
            let _ = fs::remove_file(&temp_c);
            res
        }
        "lua" => {
            let lua_code = assembled.format_lua();
            let seq = TEMP_COUNTER.fetch_add(1, Ordering::SeqCst);
            let temp_lua = std::env::temp_dir().join(format!("omnicode_dyn_{}_{}.lua", std::process::id(), seq));
            if let Err(e) = fs::write(&temp_lua, &lua_code) {
                return Err(Tripwire::trip(
                    &ctx.logger,
```

```
                    "TRIPWIRE_TRANSPILE_WRITE",
                    FailpointType::SyntaxAnomaly,
                    format!("Failed to write dynamic Lua artifact: {e}"),
                ));
            }
            let mut sub_config = ctx.config.clone();
            sub_config.input_path = temp_lua.clone();
            sub_config.format = RunFormat::Lua;
            let sub_ctx = RunContext {
                resolved_input: temp_lua.clone(),
                format: RunFormat::Lua,
                config: sub_config,
                logger: ctx.logger.clone(),
            };
            let res = run_lua_transpile(&sub_ctx);
            let _ = fs::remove_file(&temp_lua);
            res
        }
        other => Err(Tripwire::trip(
            &ctx.logger,
            "TRIPWIRE_TRANSPILE_LANG",
            FailpointType::SyntaxAnomaly,
            format!("Unsupported on-the-fly transpile target '{other}' (expected 'c' or 'lua')"),
        )),
    }
}

/// Main execution pipeline dispatcher.
pub fn execute_run_pipeline(ctx: &RunContext) -> Result<RunResult, TripwireError> {
    // 1. Dynamic On-the-Fly Transpilation Check
    if let Some(ref target_lang) = ctx.config.transpile_target {
        return run_transpile_on_the_fly(ctx, target_lang);
    }

    // 2. Format Dispatcher
    match ctx.format {
        RunFormat::Omni => run_omni_source(ctx),
        RunFormat::Tape => run_tape_ribbon(ctx),
        RunFormat::C => run_c_transpile(ctx),
        RunFormat::Lua => run_lua_transpile(ctx),
        RunFormat::Auto => {
            let ft = OmniFileType::from_path(&ctx.resolved_input);
            match ft {
                Some(OmniFileType::Tape) => run_tape_ribbon(ctx),
                Some(OmniFileType::SourceReel) | Some(OmniFileType::EpisodicScroll) => run_omni_source(ctx),
                None => {
                    let ext = ctx.resolved_input.extension().and_then(|e| e.to_str()).unwrap_or("");
                    match ext {
                        "c" | "h" => run_c_transpile(ctx),
                        "lua" => run_lua_transpile(ctx),
                        _ => run_omni_source(ctx),
                    }
                }
            }
        }
    }
}

/// Convenience entry point running directly from a `RunConfig`.
pub fn run_with_config(config: RunConfig) -> Result<RunResult, String> {
    let logger = DiagnosticLogger::new(config.verbose);
    let ctx = config.open(&logger).map_err(|e| e.to_string())?;
    execute_run_pipeline(&ctx).map_err(|e| e.to_string())
}

/// Unified runner integration for the virtual terminal substrate ([`TerminalSession`]).
pub fn run_on_virtual_terminal(
    session: &mut TerminalSession,
    path_or_cmd: &str,
) -> Result<String, String> {
    let clean_path = path_or_cmd.trim();
    let config = RunConfig::new(clean_path);
    let logger = session.logger.clone();
    let ctx = config.open(&logger).map_err(|e| e.to_string())?;

    let res = execute_run_pipeline(&ctx).map_err(|e| e.to_string())?;

    // Pipe characters into the virtual terminal's output stream
```

```
    for c in res.output.chars() {
        session.process_character_echo(c).ok();
    }

    Ok(res.output)
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests & Multi-Substrate Parity Proofs )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_run_format_detection_and_names() {
        assert_eq!(RunFormat::from_path(Path::new("test.omni")), RunFormat::Omni);
        assert_eq!(RunFormat::from_path(Path::new("test.scroll")), RunFormat::Omni);
        assert_eq!(RunFormat::from_path(Path::new("test.tape")), RunFormat::Tape);
        assert_eq!(RunFormat::from_path(Path::new("test.c")), RunFormat::C);
        assert_eq!(RunFormat::from_path(Path::new("test.lua")), RunFormat::Lua);
        assert_eq!(RunFormat::from_str_name("c"), Some(RunFormat::C));
        assert_eq!(RunFormat::from_str_name("lua"), Some(RunFormat::Lua));
    }

    #[test]
    fn test_run_character_tape_on_host_and_virtual_terminal() {
        let config = RunConfig::new("lexicon/01_nouns/ideas/types/character.tape");
        let res = run_with_config(config).expect("character.tape execution must succeed");
        assert_eq!(res.state, QuadState::Proceed);
        assert!(res.output.contains("Mounted Tape"));
        assert!(res.output.contains("TRACK 2 [APP]"));
        assert!(res.output.contains("Roll 1/6"));
        assert!(res.output.contains("Execution stopped and sealed at Roll 6"));

        // Virtual terminal integration
        let mut session = TerminalSession::default();
        let vt_out = run_on_virtual_terminal(&mut session, "lexicon/01_nouns/ideas/types/character.tape")
            .expect("Virtual terminal run of character.tape must succeed");
        assert!(vt_out.contains("Mounted Tape"));
        assert!(session.read_output_char().is_some(), "Virtual terminal INTO port must contain characters");
    }

    #[test]
    fn test_run_character_omni_on_host_and_virtual_terminal() {
        let config = RunConfig::new("lexicon/01_nouns/ideas/types/character.omni");
        let res = run_with_config(config).expect("character.omni execution must succeed");
        assert_eq!(res.state, QuadState::Proceed);
        assert!(res.output.contains("OmniCode Source Reel Executed Successfully"));
        assert!(res.output.contains("+1:PROCEED"));

        // Virtual terminal integration
        let mut session = TerminalSession::default();
        let vt_out = run_on_virtual_terminal(&mut session, "lexicon/01_nouns/ideas/types/character.omni")
            .expect("Virtual terminal run of character.omni must succeed");
        assert!(vt_out.contains("OmniCode Source Reel Executed Successfully"));
        assert!(session.read_output_char().is_some(), "Virtual terminal INTO port must contain characters");
    }

    #[test]
    fn test_run_c_backbone_suite() {
        let config = RunConfig::new("lexicon/01_nouns/ideas/types/transpile
variant/tests/c/integrated/test_omni_lib.c");
        let res = run_with_config(config).expect("C backbone test suite execution must succeed");
        assert_eq!(res.state, QuadState::Proceed);
        assert!(res.output.contains("ALL C BACKBONE TESTS PASS"));
        assert!(res.output.contains("FULL 4-BLOCK PARITY SEALED IN PROCEED"));
    }

    #[test]
    fn test_run_lua_tape_suite() {
        let config = RunConfig::new("lexicon/01_nouns/ideas/types/transpile variant/tests/lua/unit/test_character.lua");
        let res = run_with_config(config).expect("Lua test tape execution must succeed");
        assert_eq!(res.state, QuadState::Proceed);
        assert!(res.output.contains("Character Process Ribbon (Lua)"));
        assert!(res.output.contains("[PASS]"));
```

```
    }

    #[test]
    fn test_run_character_transpile_c_dynamic() {
        let mut config = RunConfig::new("lexicon/01_nouns/ideas/types/character.tape");
        config.transpile_target = Some("c".to_string());
        let res = run_with_config(config).expect("character.tape transpile C must succeed");
        assert_eq!(res.state, QuadState::Proceed);
        assert!(res.output.contains("OMNICODE C TRANSPILE RUNNER: [Character]"));
        assert!(res.output.contains("All 6 rolls verified"));
    }

    #[test]
    fn test_run_character_transpile_lua_dynamic() {
        let mut config = RunConfig::new("lexicon/01_nouns/ideas/types/character.tape");
        config.transpile_target = Some("lua".to_string());
        let res = run_with_config(config).expect("character.tape transpile Lua must succeed");
        assert_eq!(res.state, QuadState::Proceed);
        assert!(res.output.contains("OMNICODE LUA TRANSPILE RUNNER: [Character]"));
        assert!(res.output.contains("All 6 rolls verified"));
    }

    #[test]
    fn test_run_number_tape_on_host_and_virtual_terminal() {
        let config = RunConfig::new("lexicon/01_nouns/ideas/types/number.tape");
        let res = run_with_config(config).expect("number.tape execution must succeed");
        assert_eq!(res.state, QuadState::Proceed);
        assert!(res.output.contains("Mounted Tape"));
        assert!(res.output.contains("TRACK 2 [APP]"));
        assert!(res.output.contains("Roll 1/8"));
        assert!(res.output.contains("Execution stopped and sealed at Roll 8"));

        // Virtual terminal integration
        let mut session = TerminalSession::default();
        let vt_out = run_on_virtual_terminal(&mut session, "lexicon/01_nouns/ideas/types/number.tape")
            .expect("Virtual terminal run of number.tape must succeed");
        assert!(vt_out.contains("Mounted Tape"));
        assert!(session.read_output_char().is_some(), "Virtual terminal INTO port must contain characters");
    }

    #[test]
    fn test_run_number_omni_on_host_and_virtual_terminal() {
        let config = RunConfig::new("lexicon/01_nouns/ideas/types/number.omni");
        let res = run_with_config(config).expect("number.omni execution must succeed");
        assert_eq!(res.state, QuadState::Proceed);
        assert!(res.output.contains("OmniCode Source Reel Executed Successfully"));
        assert!(res.output.contains("+1:PROCEED"));

        // Virtual terminal integration
        let mut session = TerminalSession::default();
        let vt_out = run_on_virtual_terminal(&mut session, "lexicon/01_nouns/ideas/types/number.omni")
            .expect("Virtual terminal run of number.omni must succeed");
        assert!(vt_out.contains("OmniCode Source Reel Executed Successfully"));
        assert!(session.read_output_char().is_some(), "Virtual terminal INTO port must contain characters");
    }

    #[test]
    fn test_run_number_transpile_c_dynamic() {
        let mut config = RunConfig::new("lexicon/01_nouns/ideas/types/number.tape");
        config.transpile_target = Some("c".to_string());
        let res = run_with_config(config).expect("number.tape transpile C must succeed");
        assert_eq!(res.state, QuadState::Proceed);
        assert!(res.output.contains("OMNICODE C TRANSPILE RUNNER: [Number]"));
        assert!(res.output.contains("All 8 rolls verified"));
    }

    #[test]
    fn test_run_number_transpile_lua_dynamic() {
        let mut config = RunConfig::new("lexicon/01_nouns/ideas/types/number.tape");
        config.transpile_target = Some("lua".to_string());
        let res = run_with_config(config).expect("number.tape transpile Lua must succeed");
        assert_eq!(res.state, QuadState::Proceed);
        assert!(res.output.contains("OMNICODE LUA TRANSPILE RUNNER: [Number]"));
        assert!(res.output.contains("All 8 rolls verified"));
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/hover/mod.rs RS 1046 LINES 3689 WORDS 43.2 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Hover Subsystem Identity, 3-Tier Hover Architecture & Docstring Invariants )
// ============================================================================
//! OmniCode Diagnostics: Cross-File Docstring & Hover Engine
//!
//! # Architecture & Cross-Scale Transparency
//! In OmniCode, programming operates simultaneously across two complementary scales:
//! - **High-Level Process Ribbons (`.tape`):** Sequential SVO sentences and rolls
//!   executing across scheduler tracks with teleological intent (`WHY`).
//! - **Low-Level Source Reels (`.omni`):** Declarative 4-Block architecture defining
//!   invariant boundaries (`<...>`), memory storehouses, and physical cells.
//!
//! # Cross-File Symbol & Invariant Inheritance (.omni -> .tape)
//! When an engineer or AI agent hovers over a symbol inside a `.tape`:
//! 1. The symbol is resolved against its active roll context (Intent Aside & Tape Head).
//! 2. The underlying low-level definition is retrieved from the companion `.omni` file
//!    (or standard library / manifest), exposing its morphological rank, invariant
//!    contracts, and block origin.
//! 3. The machine execution footprint is synthesized (memory offset, storehouse partition,
//!    8-bit to 5-trit Radix-3 transduced codec, and QuadState transaction status).
//!
//! This delivers total under-the-hood transparency: developers write expressive
//! sentences while directly inspecting what the machine is physically doing in silicon.

use std::collections::HashMap;
use std::fs;
use std::path::Path;

use crate::ast::nodes::{ASTNode, EntityNode};
use crate::core::boundaries::BoundaryType;
use crate::core::tape::Tape;
use crate::diagnostics::linter::BlockKind;
use crate::diagnostics::logger::DiagnosticLogger;
use crate::parser::lexer::Lexer;
use crate::parser::parser::Parser;


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Data Models: OmniDefinition, TapeRollContext, HoverCard & HoverEngine )
// ============================================================================

/// Ontological definition extracted from a low-level `.omni` source reel.
#[derive(Debug, Clone, PartialEq)]
pub struct OmniDefinition {
    /// Identifier name (e.g., "Manifest", "Version", "Law_Truth", "Storehouse").
    pub name: String,
    /// Morphological boundary type (Rank 1 Invariant, Rank 2 Denotation, etc.).
    pub boundary_type: BoundaryType,
    /// Ontological rank tier (1 to 6).
    pub rank: u8,
    /// Primary grammatical part of speech.
    pub part_of_speech: &'static str,
    /// First-class machine intent Aside `(...)` declared on this definition.
    pub aside_intent: Option<String>,
    /// Structural doc comments (`#`) associated with this entity.
    pub doc_comments: Vec<String>,
    /// Originating block in the Universal 4-Block Architecture.
    pub block_kind: Option<BlockKind>,
    /// Source file path where this symbol is originally grounded.
    pub source_file: String,
    /// Line number of the definition.
    pub line: usize,
    /// Inner subfields or attributes (e.g. `commandment`, `principle`, `resonance`).
    pub fields: HashMap<String, String>,
    /// Machine execution / low-level silicon representation description.
    pub machine_representation: Option<String>,
}

/// Operational roll context extracted from a high-level `.tape` process ribbon.
#[derive(Debug, Clone, PartialEq)]
pub struct TapeRollContext {
    /// Process name declared in the leading Tape Head covenant.
```

```
    pub tape_name: String,
    /// Target execution channel (Track 0 [KERNEL], Track 1 [TERMINAL], Track 2 [APP]).
    pub track: String,
    /// 1-indexed roll position on the tape.
    pub roll_number: usize,
    /// Total rolls on the active tape ribbon.
    pub total_rolls: usize,
    /// Raw text content of the active roll.
    pub raw_roll_text: String,
    /// Machine intent Aside `(...)` attached to this specific roll.
    pub roll_aside_intent: Option<String>,
    /// Cardinal clause: WHO (Security Principal / Caller).
    pub head_who: String,
    /// Cardinal clause: WHAT (Target entity / payload).
    pub head_what: String,
    /// Cardinal clause: WHY (Teleological covenant intent).
    pub head_why: String,
    /// Cardinal clause: HOW (Directional vector / QuadState modulation).
    pub head_how: String,
    /// Spatial allocation requirement in memory cells.
    pub partition_size: usize,
}

/// Complete 3-tier composite hover documentation card.
#[derive(Debug, Clone, PartialEq)]
pub struct HoverCard {
    /// Symbol name inspected.
    pub symbol: String,
    /// High-level process roll context (present if hovered inside a `.tape`).
    pub tape_context: Option<TapeRollContext>,
    /// Low-level `.omni` blueprint and invariant contract.
    pub omni_definition: Option<OmniDefinition>,
    /// Whether the symbol was redefined locally in the active tape ribbon.
    pub is_locally_redefined: bool,
    /// Rendered GitHub-flavored Markdown text for IDE tooltips and CLI inspection.
    pub markdown: String,
}

impl HoverCard {
    /// Serializes this HoverCard to a standard JSON string without third-party dependencies.
    pub fn to_json(&self) -> String {
        let tape_ctx_json = match &self.tape_context {
            Some(tc) => {
                let aside_str = match &tc.roll_aside_intent {
                    Some(a) => format!("\"{}\"", escape_json_str(a)),
                    None => "null".to_string(),
                };
                format!(
                    r#"{{"tape_name":"{}","track":"{}","roll_number":{},"total_rolls":{},"raw_roll_text":"
{}","roll_aside_intent":{},"head_who":"{}","head_what":"{}","head_why":"{}","head_how":"{}","partition_size":{}}}"#,
                    escape_json_str(&tc.tape_name),
                    escape_json_str(&tc.track),
                    tc.roll_number,
                    tc.total_rolls,
                    escape_json_str(&tc.raw_roll_text),
                    aside_str,
                    escape_json_str(&tc.head_who),
                    escape_json_str(&tc.head_what),
                    escape_json_str(&tc.head_why),
                    escape_json_str(&tc.head_how),
                    tc.partition_size,
                )
            }
            None => "null".to_string(),
        };

        let omni_def_json = match &self.omni_definition {
            Some(od) => {
                let aside_str = match &od.aside_intent {
                    Some(a) => format!("\"{}\"", escape_json_str(a)),
                    None => "null".to_string(),
                };
                let block_str = match od.block_kind {
                    Some(b) => format!("\"{}\"", b.name()),
                    None => "null".to_string(),
                };
                let doc_comments_arr = od
                    .doc_comments
```

```
                    .iter()
                    .map(|c| format!("\"{}\"", escape_json_str(c)))
                    .collect::<Vec<_>>()
                    .join(",");
                let fields_obj = od
                    .fields
                    .iter()
                    .map(|(k, v)| format!("\"{}\":\"{}\"", escape_json_str(k), escape_json_str(v)))
                    .collect::<Vec<_>>()
                    .join(",");

                format!(
                    r#"{{"name":"{}","boundary_type":"{}","rank":{},"part_of_speech":"{}","aside_intent":
{},"doc_comments":[{}],"block_kind":{},"source_file":"{}","line":{},"fields":{{{}}}}}"#,
                    escape_json_str(&od.name),
                    escape_json_str(&od.boundary_type.to_string()),
                    od.rank,
                    escape_json_str(od.part_of_speech),
                    aside_str,
                    doc_comments_arr,
                    block_str,
                    escape_json_str(&od.source_file),
                    od.line,
                    fields_obj,
                )
            }
            None => "null".to_string(),
        };

        format!(
            r#"{{"symbol":"{}","is_locally_redefined":{},"tape_context":{},"omni_definition":{},"markdown":"{}"}}"#,
            escape_json_str(&self.symbol),
            self.is_locally_redefined,
            tape_ctx_json,
            omni_def_json,
            escape_json_str(&self.markdown),
        )
    }

    /// Renders an ANSI-styled human terminal presentation of the 3-tier hover card.
    pub fn render_terminal(&self) -> String {
        let mut out = String::new();

out.push_str("\x1b[1;36m================================================================================\x1b[0m\n");
        let rank_desc = if let Some(def) = &self.omni_definition {
            def.boundary_type.rank_name()
        } else if self.is_locally_redefined {
            "Rank 2: Denotative Essence [ ... ]"
        } else {
            "Process Tape Entity"
        };
        out.push_str(&format!(
            "\x1b[1;36m  OmniCode Symbol Inspection: \x1b[1;32m{}\x1b[0m \x1b[1;33m({})\x1b[0m\n",
            self.symbol, rank_desc
        ));

out.push_str("\x1b[1;36m================================================================================\x1b[0m\n\n");

        if let Some(ctx) = &self.tape_context {
            out.push_str("\x1b[1;33m[TIER 1: HIGH-LEVEL PROCESS TAPE ROLL]\x1b[0m\n");
            out.push_str(&format!("  • Process Ribbon:    \x1b[1;37m{}\x1b[0m ({})\n", ctx.tape_name, ctx.track));
            out.push_str(&format!("  • Active Roll:       \x1b[1;37m{}/{}\x1b[0m -> `{}`\n", ctx.roll_number,
ctx.total_rolls, ctx.raw_roll_text));
            if let Some(aside) = &ctx.roll_aside_intent {
                out.push_str(&format!("  • Roll Intent Aside: \x1b[3m\"{}\"\x1b[0m\n", aside));
            }
            out.push_str(&format!("  • Covenant Head:     WHO {} -> WHAT {} (WHY: \"{}\")\n", ctx.head_who,
ctx.head_what, ctx.head_why));
            if self.is_locally_redefined {
                out.push_str("  • \x1b[1;32m[Locally Instantiated]\x1b[0m Concrete Denotation assigned on this
ribbon.\n");
            }
            out.push('\n');
        }

        out.push_str("\x1b[1;32m[TIER 2: UNDER-THE-HOOD .OMNI BLUEPRINT]\x1b[0m\n");
        if let Some(def) = &self.omni_definition {
            let block = def.block_kind.map(|b| b.name()).unwrap_or("Declarative Scope");
```

```
            out.push_str(&format!("  • Grounded Source:   \x1b[1;37m{}:{}\x1b[0m\n", def.source_file, def.line));
            out.push_str(&format!("  • 4-Block Origin:    \x1b[1;35m{}\x1b[0m\n", block));
            out.push_str(&format!("  • Ontological Rank:  Rank {} ({})\n", def.rank, def.boundary_type));
            if let Some(aside) = &def.aside_intent {
                out.push_str(&format!("  • Contract Intent:   \x1b[3m\"{}\"\x1b[0m\n", aside));
            }
            if !def.doc_comments.is_empty() {
                out.push_str("  • Doc Comments:\n");
                for c in &def.doc_comments {
                    out.push_str(&format!("      # {}\n", c));
                }
            }
            if !def.fields.is_empty() {
                out.push_str("  • Axiomatic Fields:\n");
                for (k, v) in &def.fields {
                    out.push_str(&format!("      - {}: {}\n", k, v));
                }
            }
        } else {
            out.push_str("  • Status: Runtime dynamic process symbol; inherits ambient defaults.\n");
        }
        out.push('\n');

        out.push_str("\x1b[1;35m[TIER 3: MACHINE EXECUTION & SILICON FOOTPRINT]\x1b[0m\n");
        let partition = self.tape_context.as_ref().map(|c| format!("{} cells", c.partition_size)).unwrap_or_else(||
"1024 cells".to_string());
        out.push_str(&format!("  • Partition Reservation: \x1b[1;37m{}\x1b[0m\n", partition));
        out.push_str("  • Transducer Codec:      Dense Byte <-> 5 Balanced Trits (Radix-3 [-121..+121])\n");
        out.push_str("  • Silicon Truth State:   \x1b[1;32m+1:PROCEED\x1b[0m (Deterministic commit, transactional
rollback)\n");

        out
    }
}

/// Cross-file symbol indexer and hover resolution engine.
pub struct HoverEngine {
    pub logger: DiagnosticLogger,
    /// Indexed definitions mapped by identifier name (case-sensitive and lowercase).
    definitions: HashMap<String, OmniDefinition>,
    /// Map of file path -> list of symbol names defined in that file.
    file_symbols: HashMap<String, Vec<String>>,
}

impl HoverEngine {
    /// Instantiates a new HoverEngine with diagnostic logging.
    pub fn new(logger: DiagnosticLogger) -> Self {
        Self {
            logger,
            definitions: HashMap::new(),
            file_symbols: HashMap::new(),
        }
    }

    /// Looks up an indexed definition by name.
    pub fn lookup_definition(&self, symbol: &str) -> Option<&OmniDefinition> {
        self.definitions.get(symbol).or_else(|| self.definitions.get(&symbol.to_lowercase()))
    }

    /// Resolves documentation for a symbol, optionally indexing a target file first.
    pub fn document_symbol(&mut self, symbol: &str, file_hint: Option<&str>) -> Option<HoverCard> {
        if let Some(path) = file_hint {
            if path.ends_with(".tape") {
                return self.resolve_symbol_for_tape(path, symbol, 0);
            } else {
                let _ = self.index_file(path);
            }
        }

        let def = self.lookup_definition(symbol).cloned();
        let markdown = self.format_omni_only_markdown(symbol, def.as_ref());
        Some(HoverCard {
            symbol: symbol.to_string(),
            tape_context: None,
            omni_definition: def,
            is_locally_redefined: false,
            markdown,
        })
```

```
    }
}


// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Indexing Passes, Cross-File Symbol Inheritance & Markdown Hover Generation )
// ================================================================================

impl HoverEngine {
    /// Indexes a `.omni` source reel string, extracting all entities, boundaries,
    /// docstrings, attached Asides, and block origins.
    pub fn index_omni_source(&mut self, file_path: &str, content: &str) -> Result<usize, String> {
        let lines: Vec<&str> = content.lines().collect();
        let mut current_block = None;
        let mut count = 0;

        // Pass 1: Parse block layout
        let mut line_blocks = vec![None; lines.len()];
        for (idx, line) in lines.iter().enumerate() {
            let upper = line.to_uppercase();
            if upper.contains("BLOCK 1") || upper.contains("METADATA BLOCK") {
                current_block = Some(BlockKind::Metadata);
            } else if upper.contains("BLOCK 2") || upper.contains("OPENING BLOCK") {
                current_block = Some(BlockKind::Opening);
            } else if upper.contains("BLOCK 3") || upper.contains("BODY BLOCK") {
                current_block = Some(BlockKind::Body);
            } else if upper.contains("BLOCK 4") || upper.contains("CLOSING BLOCK") {
                current_block = Some(BlockKind::Closing);
            }
            line_blocks[idx] = current_block;
        }

        // Pass 2: Tokenize and build AST to extract formal Entities
        let mut lexer = Lexer::new(content, self.logger.clone());
        let tokens = lexer.tokenize()?;
        let debugger = crate::diagnostics::debugger::OmniDebugger::new(self.logger.clone(), 10);
        let mut parser = Parser::new(tokens, debugger);

        // Best-effort parse program
        if let Ok(prog) = parser.parse_program() {
            for stmt in &prog.statements {
                if let ASTNode::Entity(entity) = stmt {
                    let def = self.build_omni_definition(entity, file_path, &lines, &line_blocks);
                    let name = def.name.clone();
                    self.insert_definition(name.clone(), def);
                    self.file_symbols
                        .entry(file_path.to_string())
                        .or_default()
                        .push(name);
                    count += 1;
                }
            }
        }

        // Pass 3: Textual line-by-line fallback for inner fields and registers
        for (idx, line) in lines.iter().enumerate() {
            let trimmed = line.trim();
            if trimmed.starts_with('#') || trimmed.is_empty() {
                continue;
            }

            // Detect Root Module: `= [Module]: < Category { Name } >`
            if (trimmed.starts_with("= [Module]:") || trimmed.starts_with("=[Module]:")) &&
!self.definitions.contains_key("Manifest") {
                if let Some(brace_start) = trimmed.find('{') {
                    if let Some(brace_end) = trimmed[brace_start..].find('}') {
                        let candidate = trimmed[brace_start + 1..brace_start + brace_end].trim();
                        if is_valid_identifier(candidate) && !self.definitions.contains_key(candidate) {
                            let aside = extract_aside_from_str(trimmed).or_else(|| {
                                if idx + 1 < lines.len() {
                                    extract_aside_from_str(lines[idx + 1])
                                } else {
                                    None
                                }
                            });
                            let doc_comments = extract_preceding_comments(&lines, idx);
                            let mut fields = HashMap::new();
```

```
                            for l in &lines[idx + 1..idx.saturating_add(35).min(lines.len())] {
                                let l_trimmed = l.trim();
                                if l_trimmed.starts_with('}') {
                                    break;
                                }
                                if let Some(colon) = l_trimmed.find(':') {
                                    let raw_k = l_trimmed[..colon].trim();
                                    let clean_k = raw_k.trim_matches(|c| c == '[' || c == ']' || c == ' ');
                                    let v = l_trimmed[colon + 1..].trim().trim_end_matches(',');
                                    fields.insert(clean_k.to_lowercase(), v.to_string());
                                }
                            }
                            let def = OmniDefinition {
                                name: candidate.to_string(),
                                boundary_type: BoundaryType::Invariant,
                                rank: 1,
                                part_of_speech: "Noun",
                                aside_intent: aside,
                                doc_comments,
                                block_kind: line_blocks[idx],
                                source_file: file_path.to_string(),
                                line: idx + 1,
                                fields,
                                machine_representation: Some("Ontological Essence (< Invariant >)".into()),
                            };
                            self.insert_definition(candidate.to_string(), def);
                            count += 1;
                        }
                    }
                }
            }

            // Detect field patterns: `name: < ... > ( ... )` or `[ Name ]: { ... }` or `name [ ... ] ( ... )`
            if let Some(colon_pos) = trimmed.find(':') {
                let raw_candidate = trimmed[..colon_pos].trim();
                let id_candidate = raw_candidate.trim_matches(|c| c == '[' || c == ']' || c == ' ');
                if is_valid_identifier(id_candidate) && !self.definitions.contains_key(id_candidate) {
                    let rest = trimmed[colon_pos + 1..].trim();
                    let mut btype = detect_boundary_type_from_str(rest);
                    if line_blocks[idx] == Some(BlockKind::Metadata) {
                        btype = BoundaryType::Invariant;
                    }
                    let aside = extract_aside_from_str(rest).or_else(|| {
                        if idx + 1 < lines.len() {
                            extract_aside_from_str(lines[idx + 1])
                        } else {
                            None
                        }
                    });
                    let doc_comments = extract_preceding_comments(&lines, idx);

                    let def = OmniDefinition {
                        name: id_candidate.to_string(),
                        boundary_type: btype,
                        rank: btype.rank(),
                        part_of_speech: "Noun",
                        aside_intent: aside,
                        doc_comments,
                        block_kind: line_blocks[idx],
                        source_file: file_path.to_string(),
                        line: idx + 1,
                        fields: HashMap::new(),
                        machine_representation: Some("Contextual Attribute / Field".into()),
                    };
                    self.insert_definition(id_candidate.to_string(), def);
                    count += 1;
                }
            }

            // Detect standalone registers: `name [ ... ] ( ... )`
            if trimmed.contains('[') && !trimmed.starts_with("roll_") && !trimmed.starts_with("verify:") {
                let id_end = trimmed.find('[').unwrap_or(0);
                let id_candidate = trimmed[..id_end].trim();
                if is_valid_identifier(id_candidate) && !self.definitions.contains_key(id_candidate) {
                    let aside = extract_aside_from_str(trimmed);
                    let doc_comments = extract_preceding_comments(&lines, idx);
                    let def = OmniDefinition {
                        name: id_candidate.to_string(),
```

```
                        boundary_type: BoundaryType::Denotation,
                        rank: BoundaryType::Denotation.rank(),
                        part_of_speech: "Noun",
                        aside_intent: aside,
                        doc_comments,
                        block_kind: line_blocks[idx],
                        source_file: file_path.to_string(),
                        line: idx + 1,
                        fields: HashMap::new(),
                        machine_representation: Some("Active State Register".into()),
                    };
                    self.insert_definition(id_candidate.to_string(), def);
                    count += 1;
                }
            }
        }

        self.logger.log(
            "HOVER",
            format!("Indexed {} symbols from .omni source: {}", count, file_path),
        );
        Ok(count)
    }

    /// Indexes a specific file from disk.
    pub fn index_file(&mut self, path: impl AsRef<Path>) -> Result<usize, String> {
        let p = path.as_ref();
        let content = fs::read_to_string(p)
            .map_err(|e| format!("Failed to read file {:?}: {e}", p))?;
        let path_str = p.to_string_lossy().to_string();
        self.index_omni_source(&path_str, &content)
    }

    /// Recursively indexes foundational files and standard libraries across the workspace.
    pub fn index_workspace(&mut self, root_dir: impl AsRef<Path>) {
        let root = root_dir.as_ref();

        // 1. Core manifest
        let manifest_path = root.join("manifest.omni");
        if manifest_path.exists() {
            let _ = self.index_file(&manifest_path);
        }

        // 2. Standard library modules in lib/core/
        let lib_core = root.join("lib/core");
        if lib_core.exists() && lib_core.is_dir() {
            if let Ok(entries) = fs::read_dir(lib_core) {
                for entry in entries.flatten() {
                    let p = entry.path();
                    if p.extension().map_or(false, |ext| ext == "omni") {
                        let _ = self.index_file(&p);
                    }
                }
            }
        }
    }

    /// Helper to store definitions in both exact and lowercase keys for flexible lookup.
    fn insert_definition(&mut self, key: String, def: OmniDefinition) {
        self.definitions.insert(key.to_lowercase(), def.clone());
        self.definitions.insert(key, def);
    }

    /// Resolves hover information for a symbol used inside a high-level `.tape` process ribbon.
    ///
    /// # Resolution Sequence
    /// 1. Check if defined or redefined in the local `.tape` ribbon.
    /// 2. If companion `.omni` exists (e.g. `foo.tape` -> `foo.omni`), resolve from companion.
    /// 3. Otherwise, resolve from the workspace symbol index (stdlib or manifest).
    /// 4. Generate the composite 3-tier Markdown HoverCard.
    pub fn resolve_symbol_for_tape(
        &mut self,
        tape_path: &str,
        symbol: &str,
        line_number: usize,
    ) -> Option<HoverCard> {
        let tape_p = Path::new(tape_path);

```

```
        // Step 1: Ensure companion .omni is indexed if it exists
        let companion_omni = tape_p.with_extension("omni");
        if companion_omni.exists() {
            let _ = self.index_file(&companion_omni);
        }

        // Step 2: Load and inspect the active Tape
        let tape = Tape::from_file(tape_p, self.logger.clone()).ok();

        // Extract Tape Context for this roll
        let mut tape_context = None;
        let mut is_locally_redefined = false;

        if let Some(ref t) = tape {
            let track_str = if tape_path.contains("kernel") || tape_path.contains("boot") {
                "Track 0 [KERNEL]"
            } else if tape_path.contains("terminal") {
                "Track 1 [TERMINAL]"
            } else {
                "Track 2 [APP]"
            };

            // Find roll matching line number that contains symbol, or matching line number, or first roll referencing
symbol
            let matched_roll = t
                .rolls
                .iter()
                .find(|r| r.line_number == line_number && r.text.contains(symbol))
                .or_else(|| t.rolls.iter().find(|r| r.line_number == line_number))
                .or_else(|| t.rolls.iter().find(|r| r.text.contains(symbol)));

            if let Some(r) = matched_roll {
                let roll_aside = extract_aside_from_str(&r.text);

                tape_context = Some(TapeRollContext {
                    tape_name: t.head.name.clone(),
                    track: track_str.to_string(),
                    roll_number: r.index + 1,
                    total_rolls: t.roll_count(),
                    raw_roll_text: r.text.clone(),
                    roll_aside_intent: roll_aside,
                    head_who: t.head.who.clone(),
                    head_what: t.head.what.clone(),
                    head_why: t.head.why.clone(),
                    head_how: t.head.how.clone(),
                    partition_size: t.head.required_partition_size,
                });
            }

            // Check if symbol was given a Denotation anywhere on this tape ribbon: `symbol [ ... ]`
            is_locally_redefined = t.rolls.iter().any(|r| {
                r.text.contains(&format!("{symbol} [")) || r.text.contains(&format!("{symbol}["))
            });
        }

        // Step 3: Lookup low-level .omni definition
        let omni_def = self
            .definitions
            .get(symbol)
            .cloned()
            .or_else(|| self.definitions.get(&symbol.to_lowercase()).cloned());

        // Build Markdown card
        let markdown = self.format_composite_markdown(
            symbol,
            tape_context.as_ref(),
            omni_def.as_ref(),
            is_locally_redefined,
        );

        Some(HoverCard {
            symbol: symbol.to_string(),
            tape_context,
            omni_definition: omni_def,
            is_locally_redefined,
            markdown,
        })
    }
```

```

    /// Resolves hover information at a specific file coordinate `(line, col)`.
    pub fn hover_at(&mut self, file_path: &str, line: usize, col: usize) -> Option<HoverCard> {
        let content = fs::read_to_string(file_path).ok()?;
        let lines: Vec<&str> = content.lines().collect();
        if line == 0 || line > lines.len() {
            return None;
        }

        let target_line = lines[line - 1];
        let symbol = extract_word_at_col(target_line, col)?;

        if file_path.ends_with(".tape") {
            self.resolve_symbol_for_tape(file_path, &symbol, line)
        } else {
            // Index the file itself if not already done
            let _ = self.index_omni_source(file_path, &content);
            let def = self
                .definitions
                .get(&symbol)
                .cloned()
                .or_else(|| self.definitions.get(&symbol.to_lowercase()).cloned());

            let markdown = self.format_omni_only_markdown(&symbol, def.as_ref());
            Some(HoverCard {
                symbol: symbol.clone(),
                tape_context: None,
                omni_definition: def,
                is_locally_redefined: false,
                markdown,
            })
        }
    }

    /// Formats the complete 3-Tier Markdown HoverCard bridging .tape down to .omni.
    fn format_composite_markdown(
        &self,
        symbol: &str,
        tape_ctx: Option<&TapeRollContext>,
        omni_def: Option<&OmniDefinition>,
        is_redefined: bool,
    ) -> String {
        let mut md = String::new();

        // ------------------------------------------------------------------------
        // HEADER: Identity & Morphological Badging
        // ------------------------------------------------------------------------
        let rank_badge = if let Some(def) = omni_def {
            format!("`{}`", def.boundary_type.rank_name())
        } else if is_redefined {
            "`Rank 2: Denotative Essence [ ... ]`".to_string()
        } else {
            "`Process Tape Entity`".to_string()
        };

        md.push_str(&format!("### `{symbol}` — {}\n\n", rank_badge));

        // ------------------------------------------------------------------------
        // TIER 1: HIGH-LEVEL PROCESS TAPE ROLL (Process Intent & Active Sentence)
        // ------------------------------------------------------------------------
        if let Some(ctx) = tape_ctx {
            md.push_str("#### 📜 1. High-Level Process Tape View\n");
            md.push_str(&format!(
                "* **Process Ribbon:** `{}` on `{}`\n",
                ctx.tape_name, ctx.track
            ));
            md.push_str(&format!(
                "* **Active Roll ({}/{}):** `{}`\n",
                ctx.roll_number, ctx.total_rolls, ctx.raw_roll_text
            ));

            if let Some(ref intent) = ctx.roll_aside_intent {
                md.push_str(&format!("* **Roll Intent Aside:** *\"{}\"*\n", intent));
            }

            md.push_str(&format!(
                "* **Tape Head Covenant:** WHO `{}` $\\to$ WHAT `{}` (Purpose: *\"{}\"*)\n",
                ctx.head_who, ctx.head_what, ctx.head_why
```

```
            ));

            if is_redefined {
                md.push_str(
                    "\n> [!NOTE]\n> **Locally Defined:** This entity receives a concrete Denotation `[ ... ]` in this tape ribbon, instantiating it into active memory.\n",
                );
            }
            md.push_str("\n---\n\n");
        }

        // ------------------------------------------------------------------
        // TIER 2: UNDER-THE-HOOD LOW-LEVEL BLUEPRINT (.omni Blueprint & Contract)
        // ------------------------------------------------------------------
        md.push_str("#### 🔍 2. Under-The-Hood (.omni Blueprint)\n");
        if let Some(def) = omni_def {
            let block_name = def
                .block_kind
                .map(|b| b.name())
                .unwrap_or("Declarative Scope");
            md.push_str(&format!(
                "* **Grounded Source:** [`{}`]({}) (Line {})\n",
                def.source_file, def.source_file, def.line
            ));
            md.push_str(&format!("* **4-Block Origin:** `{block_name}`\n"));
            md.push_str(&format!(
                "* **Ontological Hierarchy:** Rank {} (`{}`)\n",
                def.rank, def.boundary_type
            ));

            if let Some(ref intent) = def.aside_intent {
                md.push_str(&format!("* **Contract Intent Aside:** *\"{}\"*\n", intent));
            }

            if !def.doc_comments.is_empty() {
                md.push_str("* **Architectural Description:**\n");
                for comment in &def.doc_comments {
                    md.push_str(&format!("  > {comment}\n"));
                }
            }

            if !def.fields.is_empty() {
                md.push_str("* **Sub-Attributes & Invariant Axioms:**\n");
                for (k, v) in &def.fields {
                    md.push_str(&format!("  * `{k}`: `{v}`\n"));
                }
            }
        } else {
            md.push_str("* **Under-The-Hood Status:** Pure runtime process entity; inherits standard library defaults.\n");
        }
        md.push_str("\n---\n\n");

        // ------------------------------------------------------------------
        // TIER 3: MACHINE EXECUTION & SILICON EFFECT
        // ------------------------------------------------------------------
        md.push_str("#### ⚙️ 3. Machine Execution & Silicon State\n");
        let partition = tape_ctx
            .map(|c| format!("{} cells", c.partition_size))
            .unwrap_or_else(|| "1024 cells".to_string());

        md.push_str(&format!(
            "* **Memory Allocation:** Partition reservation ceiling: `{partition}`\n"
        ));
        md.push_str("* **Transducer Codec:** Dense byte $\\longleftrightarrow$ 5 balanced trits ($[-121 \\dots +121]$ Radix-3)\n");
        md.push_str("* **Execution Truth State:** `+1: PROCEED` (Deterministic commit, transactional rollback protection)\n");

        md
    }

    /// Formats a hover card when hovering directly inside a `.omni` source reel.
    fn format_omni_only_markdown(&self, symbol: &str, def: Option<&OmniDefinition>) -> String {
        let mut md = String::new();
        if let Some(d) = def {
            let block_name = d.block_kind.map(|b| b.name()).unwrap_or("Universal Scope");
            md.push_str(&format!(
```

```
                "### `{}` — `{}`\n\n",
                d.name,
                d.boundary_type.rank_name()
            ));
            md.push_str(&format!("* **Origin:** `{block_name}` (Line {})\n", d.line));
            md.push_str(&format!("* **Part of Speech:** `{}`\n", d.part_of_speech));

            if let Some(ref intent) = d.aside_intent {
                md.push_str(&format!("* **Machine Intent Aside:** *\"{}\"*\n", intent));
            }

            if !d.doc_comments.is_empty() {
                md.push_str("\n#### Architectural Docstring:\n");
                for c in &d.doc_comments {
                    md.push_str(&format!("> {c}\n"));
                }
            }

            if !d.fields.is_empty() {
                md.push_str("\n#### Invariant Fields:\n");
                for (k, v) in &d.fields {
                    md.push_str(&format!("* `{k}`: `{v}`\n"));
                }
            }
        } else {
            md.push_str(&format!("### `{symbol}`\n\n"));
            md.push_str("* **Classification:** OmniCode Identifier\n");
            md.push_str("* **Scope:** Ambient Context\n");
        }
        md
    }

    /// Internal helper converting an AST EntityNode into an OmniDefinition.
    fn build_omni_definition(
        &self,
        entity: &EntityNode,
        file_path: &str,
        lines: &[&str],
        line_blocks: &[Option<BlockKind>],
    ) -> OmniDefinition {
        let line_idx = entity.line.saturating_sub(1);
        let block_kind = line_blocks.get(line_idx).copied().flatten();
        let doc_comments = extract_preceding_comments(lines, line_idx);
        let btype = entity.definition.boundary_type;

        let mut fields = HashMap::new();
        // Extract subfields if this was a record with key-value entries
        if line_idx < lines.len() {
            for l in &lines[line_idx..line_idx.saturating_add(15).min(lines.len())] {
                let trimmed = l.trim();
                if trimmed.starts_with('}') || trimmed.starts_with(']') {
                    break;
                }
                if let Some(colon) = trimmed.find(':') {
                    let k = trimmed[..colon].trim();
                    let v = trimmed[colon + 1..].trim().trim_end_matches(',');
                    if is_valid_identifier(k) {
                        fields.insert(k.to_string(), v.to_string());
                    }
                }
            }
        }

        let aside_intent = entity
            .aside
            .as_ref()
            .or(entity.definition.aside.as_ref())
            .or(entity.context.as_ref().and_then(|c| c.aside.as_ref()))
            .map(|a| a.intent.clone())
            .or_else(|| {
                if line_idx < lines.len() {
                    extract_aside_from_str(lines[line_idx])
                } else {
                    None
                }
            });

        OmniDefinition {
```

```
            name: entity.name.clone(),
            boundary_type: btype,
            rank: btype.rank(),
            part_of_speech: "Noun",
            aside_intent,
            doc_comments,
            block_kind,
            source_file: file_path.to_string(),
            line: entity.line,
            fields,
            machine_representation: Some(format!("Ontological Essence ({})", btype)),
        }
    }
}


// ---------------------------------------------------------------------------
// HELPER UTILITIES: Token extraction, comments, and delimiters
// ---------------------------------------------------------------------------

fn escape_json_str(s: &str) -> String {
    let mut out = String::with_capacity(s.len());
    for c in s.chars() {
        match c {
            '"' => out.push_str("\\\""),
            '\\' => out.push_str("\\\\"),
            '\n' => out.push_str("\\n"),
            '\r' => out.push_str("\\r"),
            '\t' => out.push_str("\\t"),
            c => out.push(c),
        }
    }
    out
}

fn is_valid_identifier(s: &str) -> bool {
    if s.is_empty() {
        return false;
    }
    let first = s.chars().next().unwrap();
    (first.is_alphabetic() || first == '_') && s.chars().all(|c| c.is_alphanumeric() || c == '_')
}

fn extract_word_at_col(line: &str, col: usize) -> Option<String> {
    if line.is_empty() {
        return None;
    }
    let chars: Vec<char> = line.chars().collect();
    let idx = col.saturating_sub(1).min(chars.len().saturating_sub(1));

    if !chars[idx].is_alphanumeric() && chars[idx] != '_' {
        return None;
    }

    let mut start = idx;
    while start > 0 && (chars[start - 1].is_alphanumeric() || chars[start - 1] == '_') {
        start -= 1;
    }

    let mut end = idx;
    while end + 1 < chars.len() && (chars[end + 1].is_alphanumeric() || chars[end + 1] == '_') {
        end += 1;
    }

    Some(chars[start..=end].iter().collect())
}

fn extract_aside_from_str(text: &str) -> Option<String> {
    let start = text.find('(')?;
    let end = text.rfind(')')?;
    if end > start {
        Some(text[start + 1..end].trim().to_string())
    } else {
        None
    }
}

fn detect_boundary_type_from_str(s: &str) -> BoundaryType {
```

```
    if s.contains('<') && s.contains('>') {
        BoundaryType::Invariant
    } else if s.contains('{') && s.contains('}') {
        BoundaryType::Connotation
    } else if s.contains('{') && s.contains(']') {
        BoundaryType::TopDown
    } else if s.contains('[') && s.contains('}') {
        BoundaryType::BottomUp
    } else {
        BoundaryType::Denotation
    }
}

fn extract_preceding_comments(lines: &[&str], line_idx: usize) -> Vec<String> {
    let mut comments = Vec::new();
    let mut cur = line_idx;
    while cur > 0 {
        cur -= 1;
        let trimmed = lines[cur].trim();
        if trimmed.starts_with('#') {
            let clean = trimmed.trim_start_matches('#').trim();
            // Ignore boundary separator bars: ====== or ------
            if !clean.starts_with('=') && !clean.starts_with('-') && !clean.is_empty() {
                comments.push(clean.to_string());
            }
        } else {
            break;
        }
    }
    comments.reverse();
    comments
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests, Verification & Subsystem Sealing )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_hover_engine_manifest_omni_indexing() {
        let logger = DiagnosticLogger::new(false);
        let mut engine = HoverEngine::new(logger);

        let manifest_content = r#"
# ============================================================================
# BLOCK 1: METADATA BLOCK
# ============================================================================
# LAW 1: TRUTH & FIDELITY
# Unalterable prohibition against deceit.
Law_Truth < Invariant > ( Invariant Contract: Prohibit deception ) {
    commandment: < "Thou shalt not bear false witness." >,
    resonance: < +1:PROCEED >
}

# ============================================================================
# BLOCK 2: OPENING BLOCK
# ============================================================================
covenant_active [ +1 : State? ] ( Active State Register )
        "#;

        let count = engine
            .index_omni_source("manifest.omni", manifest_content)
            .expect("Indexing must succeed");

        assert!(count >= 2);

        // Verify Law_Truth extraction
        let law_def = engine.definitions.get("Law_Truth").expect("Must index Law_Truth");
        assert_eq!(law_def.boundary_type, BoundaryType::Invariant);
        assert_eq!(law_def.rank, 1);
        assert_eq!(law_def.block_kind, Some(BlockKind::Metadata));
        assert!(law_def.aside_intent.as_ref().unwrap().contains("Prohibit deception"));
        assert!(law_def.doc_comments.iter().any(|c| c.contains("TRUTH & FIDELITY")));
```

```
    }

    #[test]
    fn test_cross_file_tape_to_omni_hover_resolution() {
        let logger = DiagnosticLogger::new(false);
        let mut engine = HoverEngine::new(logger);

        // Index low-level omni
        let omni_content = r#"
# BLOCK 1: METADATA
# System version invariant
Version < "1.0.0" > ( Baseline release )
        "#;
        engine.index_omni_source("test.omni", omni_content).unwrap();

        // Synthetic tape source
        let tape_content = r#"
[Tape.TestProcess] : WHO {Kernel} WHAT {Test} --> BOOT {Hardware}
    WHEN {Boot}
    WHERE {partition} = [ 256 ]
    WHY ( Test ribbon )
    HOW [ +1 : PROCEED ]

roll_1 [ Version [ "1.0.0" ] ] ( Concrete definition in memory )
roll_2 [ commit_status [ PROCEED ] ]
        "#;

        // Temporarily write tape file to test directory
        let temp_dir = std::env::temp_dir();
        let tape_path = temp_dir.join("test.tape");
        let omni_path = temp_dir.join("test.omni");
        fs::write(&tape_path, tape_content).unwrap();
        fs::write(&omni_path, omni_content).unwrap();

        let card = engine
            .resolve_symbol_for_tape(tape_path.to_str().unwrap(), "Version", 9)
            .expect("Must resolve hover card");

        assert_eq!(card.symbol, "Version");
        assert!(card.is_locally_redefined);
        assert!(card.tape_context.is_some());
        assert!(card.markdown.contains("High-Level Process Tape View"));
        assert!(card.markdown.contains("Under-The-Hood (.omni Blueprint)"));
        assert!(card.markdown.contains("Machine Execution & Silicon State"));

        // Clean up temp files
        fs::remove_file(tape_path).ok();
        fs::remove_file(omni_path).ok();
    }
}
```